Companion Paper — Forecasting Swiss GDP (Rolling AR/ARDL Pipeline)

Maxime Fleury, Eva Macchitella & Alex Ogbonna

2026-01-05

Table of contents

1 Replication overview **1**
1.1 Execution order . . . 1

2 Function map **2**
2.1 Load functions . . . 4

3 Script 00 — Setup **4**
3.1 Purpose of the script . . . 4
3.2 Main operations . . . 4

4 Script 01 — Data acquisition **5**
4.1 Goal of the script . . . 5
4.2 Why we do it . . . 8
4.3 Key outputs . . . 8

5 Script 02 — Cleaning data **9**
5.1 Transformations . . . 9
5.1.1 Seasonal adjustment (China) . . . 9
5.1.2 Alignment and quarterly structure . . . 9
5.1.3 Missing values treatment . . . 11
5.1.4 Growth-rate construction . . . 11
5.2 External demand variable . . . 12
5.2.1 Trade weights . . . 12
5.2.2 Kalman imputation on trade weights . . . 13
5.2.3 External growth (trade-weighted partner growth) . . . 13
5.3 COVID winsorization . . . 14
5.4 Key outputs . . . 15

6 Script 03 — Preliminary tests **15**
6.1 Why these tests . . . 15
6.1.1 Stationarity . . . 15
6.1.2 Structural breaks and parameter stability . . . 16

6.1.3 Granger causality . . . 16
6.2 Stationarity tests . . . 16
6.2.1 ADF test (Augmented Dickey–Fuller) . . . 16
6.2.2 KPSS test (Kwiatkowski–Phillips–Schmidt–Shin) . . . 17
6.3 Structural break analysis . . . 18
6.3.1 Model and hypotheses . . . 18
6.3.2 Additional robustness checks . . . 19
6.4 Relationship stability tests . . . 20
6.4.1 Hypothesis and interpretation . . . 21
6.5 Granger causality tests . . . 22
6.5.1 How to interpret the results . . . 24

7 Script 04 — Rolling model estimation 24
7.1 Model set-up . . . 24
7.1.1 AR benchmark . . . 24
7.1.2 External-demand process . . . 25
7.1.3 Swiss ARDL model . . . 25
7.1.4 Lag selection and rolling estimation . . . 25
7.2 Initialization of rolling parameters . . . 26
7.3 Rolling estimation . . . 26
7.4 Exporting rolling estimation results . . . 28
7.5 Final-window diagnostics (residual checks) . . . 28

8 Script 05 — Rolling forecasting 31
8.1 Forecasting framework . . . 31
8.1.1 Recursive rolling-window forecasting . . . 31
8.1.2 Models compared . . . 32
8.1.3 Multi-horizon recursive backtesting . . . 32
8.2 Forecast Evaluation . . . 33
8.2.1 Absolute forecast evaluation . . . 33
8.2.2 Relative evaluation . . . 40
8.2.3 Export of evaluation results . . . 43
8.3 True out-of-sample forecast (2025–2027) . . . 44
8.3.1 Forecast design . . . 44
8.4 Annualization . . . 47
8.4.1 From quarterly log-growth to a quarterly level index . . . 47
8.4.2 Annual aggregation and annual growth . . . 47

9 Script 06 — Figures and density forecast 49
9.1 Swiss GDP level and growth . . . 49
9.2 Swiss GDP growth: Original vs winsorized (COVID focus) . . . 51
9.3 Individual country GDP growth rates . . . 53
9.4 External demand growth . . . 55
9.5 Evolution of Swiss trading partners' weights . . . 57
9.6 Stability of the Swiss–external GDP relationship (rolling correlations) . . . 58
9.7 Comprehensive comparison plot (Swiss GDP growth vs. external demand growth) . . . 61
9.8 Diagnostic spaghetti chart . . . 62
9.9 Forecast error distribution . . . 64

9.10 Final visualization: quarterly density forecast (ARDL fan chart) . . . 66
9.10.1 Method overview (Monte Carlo density forecast) . . . 66
9.11 Annual growth comparison: Switzerland vs. global demand . . . 69
9.11.1 Data preparation . . . 69
9.11.2 Annual growth comparison chart . . . 71
9.12 Publication-style forecast table . . . 73
9.12.1 Prepare data for the table grid . . . 73
9.12.2 Draw and export the table . . . 75

10 Appendix A — Core functions **78**
10.1 Data download & transformation functions . . . 78
10.1.1 `download_fred_data()` . . . 78
10.1.2 `impute_missing_quarters()` . . . 78
10.1.3 `calculate_growth_rate()` . . . 79
10.1.4 `clean_gdp_data()` . . . 79
10.1.5 `compute_winsor_bounds()` . . . 81
10.1.6 `winsorize_period()` . . . 81
10.2 Statistical testing and diagnostics . . . 82
10.2.1 `adf_stationarity()` . . . 82
10.2.2 `kpss_stationarity()` . . . 83
10.2.3 `bai_perron_breaks()` . . . 84
10.2.4 `rec_cusum_test()` . . . 85
10.2.5 `granger_causality()` . . . 86
10.2.6 `ljung_box_residuals()` . . . 87
10.2.7 `ljung_box_squared_residuals()` . . . 88
10.2.8 `jarque_bera_residuals()` . . . 89
10.3 Standard forecast evaluation functions . . . 90
10.3.1 `forecast_metrics()` . . . 90
10.3.2 `t_test_unbiasedness()` . . . 91
10.3.3 `mincer_zarnowitz_test()` . . . 92
10.3.4 `efficiency_test()` . . . 94
10.3.5 `msfe_ratio()` . . . 95
10.3.6 `mafe_ratio()` . . . 96
10.3.7 `dm_test_eval()` . . . 97
10.4 Rolling window estimation . . . 97
10.4.1 `estimate_rolling_models()` . . . 97
10.5 Rolling forecasting functions . . . 102
10.5.1 `get_coefficient_safe()` . . . 102
10.5.2 `multi_horizon_recursive_forecasting()` . . . 103
10.5.3 `get_forecast_simulation_dynamic()` . . . 108
10.5.4 `generate_rolling_paths()` . . . 111
10.5.5 `aggregate_growth_to_annual()` . . . 115

11 References **117**

1 Replication overview

This companion paper documents the full forecasting pipeline used to produce Swiss GDP forecasts based on a rolling ARDL framework. It is intended as a technical companion to the main paper, focusing on implementation details, reproducibility, and model mechanics rather than substantive economic interpretation.

The document serves two complementary purposes:

1. To provide the theoretical background and implementation details of every custom function defined in `R/functions.R`.
2. To explain the rationale, structure, and outputs of each script composing the forecasting pipeline (`scripts/01` to `scripts/06`).

The structure of the paper is intentionally designed to mirror the execution order of the code, following best practices in computational reproducibility and transparent forecasting pipelines.

1.1 Execution order

The forecasting pipeline is modular and fully automated. Each script performs a well-defined task and is executed sequentially in the following order:

1. `scripts/00_setup.R`
2. `R/functions.R`
3. `scripts/01_load_data.R`
4. `scripts/02_clean_data.R`
5. `scripts/03_preliminary_tests.R`
6. `scripts/04_model_estimation_rolling.R`
7. `scripts/05_forecasting_rolling.R`
8. `scripts/06_graphs.R`

The pipeline is designed to be run from the project root via the script `main.R`, which sequentially sources all required scripts.

> **Reproducibility note**
> In this document, the full pipeline can optionally be re-executed by setting `params$run_pipeline = TRUE`.
> This option is disabled by default to ensure fast compilation of the companion paper.

```
if (isTRUE(params$run_pipeline))
  source("main.R")
```

2 Function map

This project relies on a set of custom helper functions implemented in `R/functions.R`. In the main text, we describe methods at the level of the pipeline scripts (`scripts/01` to `scripts/06`) and refer to these functions only when they are used.

A complete function-by-function documentation (theory, hypotheses, implementation notes, and minimal examples) is provided in **Appendix A**.

The following table provides a high-level overview of all custom functions used in the project, their purpose, and where they enter the pipeline.

Function	Category	Signature	Used in
download_fred_data	Data acquisition	fred_id, output_filename, start_date = "2000-01-01"	01_load_data
impute_missing_quarters	Missing data handling	target_data, anchor_data, value_col	(called by other functions)
calculate_growth_rate	Transformations	variable, type = "qoq"	02_clean_data
clean_gdp_data	Data cleaning	data, date_col, gdp_col, date_format = "date", country_name	(called by other functions)
compute_winsor_bounds	Outlier handling	variable, lower_quantile = 0.05, upper_quantile = 0.95	02_clean_data
winsorize_period	Outlier handling	data, variable_name, start_year, start_quarter, end_year, end_quarter, winsor_bounds	02_clean_data
adf_stationarity	Stationarity tests	variable, lag_included = 4	03_preliminary_tests
kpss_stationarity	Stationarity tests	variable	03_preliminary_tests
bai_perron_breaks	Structural breaks / stability	data, series_name, min_segment_fraction = 0.15	03_preliminary_tests
rec_cusum_test	Structural breaks / stability	formula, data	03_preliminary_tests
granger_causality	Predictability tests	data, x, y, lags = c(2, 4, 6, 8), winsorized = FALSE	03_preliminary_tests
ljung_box_residuals	Residual diagnostics	model, lag = 10	04_model_estimation_rolling

Function	Category	Signature	Used in
ljung_box_squared_residuals	Residual diagnostics	model, lag = 5	04_model_estimation_rolling
jarque_bera_residuals	Residual diagnostics	model	04_model_estimation_rolling
forecast_metrics	Forecast accuracy metrics	y_real, y_hat	05_forecasting_rolling
t_test_unbiasedness	Forecast evaluation tests	y_real, y_hat, h = 1	05_forecasting_rolling
mincer_zarnowitz_test	Forecast evaluation tests	y_real, y_hat, h = 1	05_forecasting_rolling
efficiency_test	Forecast evaluation tests	y_real, y_hat, info	05_forecasting_rolling
msfe_ratio	Forecast accuracy metrics	e1, e2, name1 = “model1”, name2 = “model2”	05_forecasting_rolling
mafe_ratio	Forecast accuracy metrics	e1, e2, name1 = “model1”, name2 = “model2”	05_forecasting_rolling
dm_test_eval	Forecast comparison tests	e1, e2, h = 1, name1 = “model1”, name2 = “model2”	05_forecasting_rolling
estimate_rolling_models	Rolling estimation / model selection	data, y_col, x_col, window_size = 24, max_lag = 3, end_indices = NULL	04_model_estimation_rolling
get_coefficient_safe	Utilities	coefficient_table, target_date, model_type, coefficient_name	(called by other functions)
multi_horizon_recursive_forecasting	Forecasting engine	data, specifications, coefficients, y_col, x_col, max_h = 10, max_lag = 3	05_forecasting_rolling
get_forecast_simulation_dynamic	Density forecast / simulation	y_train, x_train, H = 10, n_sims = 1000, max_lag = 3	06_graphs
get_c	Utilities	helper for safe coefficient extraction inside simulation routines	(called by other functions)
generate_rolling_paths	Density forecast / simulation	data, specifications, coefficients, y_col, x_col, H = 10, max_lag = 3	06_graphs

Function	Category	Signature	Used in
aggregate_growth_to_annual	Post-processing	dates_q, g_q, years_keep = NULL, base_log = 0, agg = "sum"	05_forecasting_rolling, 06_graphs

2.1 Load functions

```
source("R/functions.R")
```

3 Script 00 — Setup

This section documents the initial setup stage of the forecasting pipeline. The script scripts/00_setup.R ensures that the computational environment is correctly configured before any data processing, estimation, or forecasting steps are executed.

3.1 Purpose of the script

The purpose of `scripts/00_setup.R` is to guarantee environment reproducibility by:

- explicitly listing all R packages required by the project,
- installing missing ones,
- loading all required packages into the active R session.

This design allows the full pipeline to be executed on a clean machine without manual intervention, while avoiding unnecessary reinstallation of already available packages.

3.2 Main operations

Concretely, the script performs the following operations:

1. Defines a vector containing the names of all packages used in the project.

```
# List of all packages needed for the project
required_packages <- c(
  "tidyverse",
  "readxl",
  "lubridate",
  "tseries",
  "forecast",
  "strucchange",
  "seasonal",
  "imputeTS",
```

```
  "patchwork",
  "zoo",
  "sandwich"
)
```

2. Checks which packages are already installed in the local R environment.

```
# Identify missing packages
missing_packages <- required_packages[
  !required_packages %in% installed.packages()[, "Package"]
]
```

3. Installs only the missing packages.

```
# Install only missing packages
if (length(missing_packages) > 0) {
  install.packages(missing_packages, dependencies = TRUE)
}
```

4. Loads all required packages into the active R session.

```
# Load all packages
invisible(lapply(required_packages, library, character.only = TRUE))
```

5. Prints a confirmation message when the setup is complete.

```
cat("All required packages are installed and loaded successfully.\n")
```

```
All required packages are installed and loaded successfully.
```

4 Script 01 — Data acquisition

This section documents the data acquisition steps implemented in `scripts/01_load_data.R`. The goal of this script is to load and store all raw macroeconomic datasets required for the Swiss GDP forecasting pipeline, without applying transformations yet. All harmonization and variable construction is deferred to `scripts/02_clean_data.R`.

4.1 Goal of the script

`scripts/01_load_data.R` loads raw quarterly (or monthly-to-quarterly) datasets from the following sources:

1. **Switzerland (SECO)** : real GDP (constant prices), seasonally and calendar adjusted (CSSA), quarterly.

```r
# Swiss GDP from State Secretariat for Economic Affairs (SECO)
# Data: Real GDP (constant prices), seasonally and calendar adjusted
# Frequency: Quarterly, from 1980
swiss_gdp <- read_csv(
  "https://www.seco.admin.ch/dam/seco/de/dokumente/Wirtschaft/Wirtschaftslage/BIP_Daten/ch_seco
) %>%
  filter(
    seas_adj == "cssa",
    type == "real",
    date >= "2000-01-01",
    structure == "gdp"
  ) %>%
  select(date, gdp = value)
```

2. **United States, Euro Area, Japan (FRED)** : real GDP series (quarterly, seasonally adjusted), downloaded through the helper function `download_fred_data()`.

```r
# All FRED series: Real GDP (constant prices), seasonally adjusted
# Frequency: Quarterly, from 2000Q1
# Download all FRED datasets using standardized function
# Ensures consistent date filtering and file naming across international sources
usa_gdp <- download_fred_data("GDPC1", "usa_gdp.csv")
euro_gdp <- download_fred_data("CLVMNACSCAB1GQEA19", "euro_gdp.csv")
japan_gdp <- download_fred_data("JPNRGDPEXP", "japan_gdp.csv")
```

3. **China (NBS, manual file)** : quarterly nominal GDP from a locally stored CSV file (`data/quarterly_china_gdp.csv`).

```r
# China GDP from National Bureau of Statistics
# Data: Nominal GDP (current prices), NOT seasonally adjusted
# Units: 100 million Yuan, raw quarterly data
# Note: Requires manual update from official Chinese statistics
china_gdp_raw <- read_csv("data/quarterly_china_gdp.csv", skip = 2) %>%
  filter(grepl("Gross Domestic Product Current Quarter", Indicators)) %>%
  select(-Indicators) %>%
  pivot_longer(everything(), names_to = "quarter_label", values_to = "gdp_china_nominal") %>%
  mutate(gdp_china_nominal = as.numeric(gdp_china_nominal)) %>%
  arrange(desc(row_number()))
```

4. **United Kingdom (ONS)** : real GDP chained volume measure (series ABMI), quarterly, downloaded via an ONS CSV endpoint.

```r
# United Kingdom GDP from Office for National Statistics
# Series: ABMI - Gross Domestic Product: chained volume measure
# Data: Real GDP, seasonally adjusted
# Units: Index, 2019 = 100
uk_gdp <- read_csv(
```

```
  "https://www.ons.gov.uk/generator?format=csv&uri=/economy/grossdomesticproductgdp/timeseries/
) %>%
  filter(
    grepl("^[0-9]{4} Q[1-4]$", Title),
    as.numeric(substr(Title, 1, 4)) >= 2000
  ) %>%
  select(quarter = Title, gdp_uk = 2)
```

5. **Swiss external trade (manual file)** : monthly exports/imports by partner from a local Excel file (`data/swiss_external_trade.xlsx`), aggregated to quarterly totals for the five main partners used in the project.

```
# Swiss foreign trade by trading partner (seasonally adjusted), since January 1997
# Data: Monthly exports and imports by partner country
# Units: Millions of CHF, nominal values
# Note: Requires manual update from OFDF

# Helper: month names in French
month_levels_french <- c(
  "Janvier", "Février", "Mars", "Avril", "Mai", "Juin",
  "Juillet", "Août", "Septembre", "Octobre", "Novembre", "Décembre"
)

# Partners to keep for analysis
partners_keep <- c("Chine", "Japon", "Zone euro", "USA", "Royaume-Uni")

# Process raw Excel data to create quarterly trade data for 5 major partners
swiss_trade_5_q <- read_excel(
  "data/swiss_external_trade.xlsx",
  sheet = "fr",
  skip  = 16,
  col_names = c(
    "partner", "partner_type", "year", "month",
    "exports_chf", "exports_change_pct",
    "imports_chf", "imports_change_pct"
  )
) %>%
  filter(partner != "Spalte1") %>%
  mutate(
    year        = as.integer(year),
    exports_chf = as.numeric(exports_chf),
    imports_chf = as.numeric(imports_chf),
    month_num   = match(month, month_levels_french),
    date        = make_date(year, month_num, 1L),
    quarter     = quarter(date)
  ) %>%
  filter(
```

```
    year >= 2000, year <= 2025,
    partner %in% partners_keep
  ) %>%
  group_by(partner, year, quarter) %>%
  summarise(
    exports_chf = sum(exports_chf, na.rm = TRUE),
    imports_chf = sum(imports_chf, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(year, quarter, partner)
```

The script also ensures a consistent time coverage by filtering most sources to the post-2000 period and prepares minimal date/quarter variables for later merging.

4.2 Why we do it

The forecasting model combines:

- **Swiss GDP growth** as the target variable y_t,
- an **external demand proxy** x_t constructed from trading partner GDP growth and Swiss trade exposure.

Therefore, we require:

- Swiss national accounts data to define y_t,
- GDP series for major partners (USA, Euro Area, Japan, UK, China) to construct partner growth rates,
- Swiss trade data by partner to build trade-weighted external demand (and potentially validate partner relevance).

Two series require manual updates because they come from locally stored files:

- `data/quarterly_china_gdp.csv` (China GDP)
- `data/swiss_external_trade.xlsx` (Swiss trade by partner)

4.3 Key outputs

At the end of this script, the following raw objects are available for the next steps of the pipeline:

- **swiss_gdp**: Swiss real GDP (quarterly, CSSA, real, post-2000)
- **usa_gdp, euro_gdp, japan_gdp**: trading partner GDP series from FRED (quarterly)
- **china_gdp_raw**: China nominal quarterly GDP from local CSV (manual update)
- **uk_gdp**: UK GDP chained volume measure from ONS (quarterly)
- **swiss_trade_5_q**: Swiss exports/imports aggregated to quarterly totals for five partners (manual update)

5 Script 02 — Cleaning data

This section documents the data preparation and feature-engineering step implemented in `scripts/02_clean_data.R`. Starting from the raw datasets loaded in Script 01, this script (i) seasonally adjusts China GDP, (ii) constructs a unified quarterly dataset in levels, (iii) imputes missing values using Kalman smoothing on levels, (iv) transforms GDP levels into quarterly growth rates using log-differences, (v) builds a trade-weighted external-demand growth measure, and (vi) applies a targeted winsorization procedure during the COVID window (2020Q2–2021Q1). The script ends by exporting three cleaned datasets used in the remaining pipeline.

5.1 Transformations

5.1.1 Seasonal adjustment (China)

China's quarterly GDP is not seasonally adjusted in the raw file. To make it comparable with other partner GDP series, we apply seasonal adjustment using the X-13ARIMA-SEATS procedure via seasonal::seas, with automatic model selection. The output is a quarterly seasonally adjusted GDP series.

```
# Apply seasonal adjustment to Chinese quarterly GDP using X-13-ARIMA.
# This method is the international standard (OECD, IMF, central banks)
# and accounts for China-specific seasonal patterns.
# The output is a quarterly, seasonally adjusted GDP series comparable to
# other international GDP measures used in the project.

china_time_series <- ts(china_gdp_raw$gdp_china_nominal,
                        start = c(2000, 1),
                        frequency = 4)

china_seas_adjusted <- seas(china_time_series)

# Build a clean quarterly dataset with consistent date, year and quarter variables
china_gdp <- data.frame(
  date = seq(yq("2000 Q1"), by = "quarter",
             length.out = length(final(china_seas_adjusted))),
  china_gdp = as.numeric(final(china_seas_adjusted))
) %>%
  mutate(
    year    = as.numeric(format(date, "%Y")),
    quarter = (as.numeric(format(date, "%m")) + 2) %/% 3
  )
```

5.1.2 Alignment and quarterly structure

All GDP series are merged into a single quarterly panel (`gdp_level_data`) with:

- a consistent `date` variable (quarterly),
- `year` and `quarter` features derived from the date,
- a common sample restriction (2000Q1 to 2025Q4).

This sample restriction reflects the intersection of data availability across all sources and ensures a balanced panel for subsequent rolling estimation.

```
# Create comprehensive dataset with all economic variables in levels.
# Includes Swiss GDP and international GDPs.
# All series aligned to quarterly frequency with consistent date structure.

# Start with Swiss GDP
gdp_level_data <- swiss_gdp %>%
  select(date, gdp) %>%
  mutate(
    year = as.numeric(format(date, "%Y")),
    quarter = (as.numeric(format(date, "%m")) + 2) %/% 3
  )

# Join international GDP series (levels)
gdp_level_data <- gdp_level_data %>%
  left_join(
    usa_gdp %>% transmute(date = as.Date(observation_date), usa_gdp = GDPC1),
    by = "date"
  ) %>%
  left_join(
    euro_gdp %>% transmute(date = as.Date(observation_date), euro_gdp = CLVMNACSCAB1GQEA19),
    by = "date"
  ) %>%
  left_join(
    japan_gdp %>% transmute(date = as.Date(observation_date), japan_gdp = JPNRGDPEXP),
    by = "date"
  ) %>%
  left_join(
    uk_gdp %>% transmute(date = yq(quarter), uk_gdp = as.numeric(gdp_uk)),
    by = "date"
  ) %>%
  left_join(
    china_gdp %>% transmute(date, china_gdp),
    by = "date"
  ) %>%
  filter(date >= as.Date("2000-01-01"), date <= as.Date("2025-12-31")) %>%
  arrange(date)
```

5.1.3 Missing values treatment

Missing values are imputed on levels, rather than on growth rates, using Kalman smoothing based on an ARIMA state-space representation (`imputeTS::na_kalman` with `model = "auto.arima"`).

In this procedure, an ARIMA model is automatically selected and cast into a state-space form, and missing observations are inferred using the Kalman filter and smoother.

Imputing levels is more stable than imputing differenced growth series and avoids injecting excessive volatility into subsequent growth-rate transformations.

```
# We impute missing values on GDP levels (more stable), then recompute growth.
# This avoids imputing growth rates directly, which can be noisy.

level_columns <- c("gdp", "usa_gdp", "euro_gdp", "japan_gdp",
                   "uk_gdp", "china_gdp")

for (col in level_columns) {
  ts_col <- ts(gdp_level_data[[col]], frequency = 4)
  gdp_level_data[[col]] <- as.numeric(na_kalman(ts_col, model = "auto.arima"))
}
```

5.1.4 Growth-rate construction

Quarterly GDP growth is computed for each series using log-differences (QoQ), implemented through the function `calculate_growth_rate()`:

$$g_t^{qoq} = 100 \times (\log(Y_t) - \log(Y_{t-1}))$$

```
# Transform level variables into growth rates for stationarity.
# Uses logarithmic differences for GDP series.
# Essential for time series analysis and econometric modeling.

gdp_growth_data <- gdp_level_data %>%
  mutate(
    swiss_gdp_growth  = calculate_growth_rate(gdp, "qoq"),
    usa_gdp_growth    = calculate_growth_rate(usa_gdp, "qoq"),
    euro_gdp_growth   = calculate_growth_rate(euro_gdp, "qoq"),
    japan_gdp_growth  = calculate_growth_rate(japan_gdp, "qoq"),
    uk_gdp_growth     = calculate_growth_rate(uk_gdp, "qoq"),
    china_gdp_growth  = calculate_growth_rate(china_gdp, "qoq")
  ) %>%
  select(date, year, quarter, ends_with("_growth"))
```

This transformation helps stabilize variance and improves stationarity properties for time-series modeling.

5.2 External demand variable

5.2.1 Trade weights

Quarterly time-varying trade weights are built from Swiss external trade data. For each partner and quarter:

- total trade is defined as exports + imports,
- the partner weight is its share of total trade across the five partner groupings (Europe, US, China, UK, Japan):

$$w_{i,t} = \frac{Trade_{i,t}}{\sum_j Trade_{j,t}}$$

```
# Calculate varying trade weights based on OFDF from 2000 to 2025 quarterly
# Weights represent relative importance of each trading partner to Swiss economy
# Based on total trade volume (exports + imports) with each partner

trade_weights_q <- swiss_trade_5_q %>%
  mutate(
    total_trade = exports_chf + imports_chf,
    partner_group = case_when(
      partner == "Zone euro"   ~ "Europe",
      partner == "USA"         ~ "US",
      partner == "Chine"       ~ "China",
      partner == "Royaume-Uni" ~ "UK",
      partner == "Japon"       ~ "Japan"
    )
  ) %>%
  group_by(year, quarter) %>%
  mutate(weight_trade = total_trade / sum(total_trade, na.rm = TRUE)) %>%
  ungroup() %>%
  select(year, quarter, partner_group, weight_trade) %>%
  pivot_wider(
    names_from  = partner_group,
    values_from = weight_trade,
    names_prefix = "weight_"
  ) %>%
  arrange(year, quarter)
```

These weights are time-varying (quarter-specific), reflecting the evolving trade exposure of Switzerland.

5.2.2 Kalman imputation on trade weights

To ensure that external-growth construction does not generate missing values, any missing weights are also imputed using Kalman smoothing with `model = "auto.arima"`.

```
# KALMAN IMPUTATION ON TRADE WEIGHTS
# Ensures complete weight series to avoid NA in external_growth
weight_columns <- grep("^weight_", names(trade_weights_q), value = TRUE)

for (col in weight_columns) {
  ts_col <- ts(trade_weights_q[[col]], frequency = 4)
  trade_weights_q[[col]] <- as.numeric(na_kalman(ts_col, model = "auto.arima"))
}
```

5.2.3 External growth (trade-weighted partner growth)

The external demand proxy is defined as the weighted average of partner GDP growth rates:

$$x_t = \sum_{i \in \{EU, US, China, UK, Japan\}} w_{i,t}\, g_{i,t}$$

This construction yields a time-varying external demand indicator consistent with Switzerland's evolving trade exposure.

```
# Create weighted external growth variable for Swiss GDP forecasting
# Represents demand conditions in Switzerland's main trading partners
# Varying weights ensures consistent economic interpretation over time

gdp_growth_data <- gdp_growth_data %>%
  left_join(trade_weights_q, by = c("year", "quarter")) %>%
  mutate(
    external_growth =
      weight_Europe * euro_gdp_growth +
      weight_US     * usa_gdp_growth +
      weight_China  * china_gdp_growth +
      weight_UK     * uk_gdp_growth +
      weight_Japan  * japan_gdp_growth
  )
```

Operationally, this becomes:

- `weight_Europe * euro_gdp_growth`
- `weight_US * usa_gdp_growth`
- `weight_China * china_gdp_growth`
- `weight_UK * uk_gdp_growth`
- `weight_Japan * japan_gdp_growth`

5.3 COVID winsorization

To reduce the influence of extreme observations during the COVID shock, which can dominate rolling estimation and model selection, growth series are winsorized only over the window 2020Q2–2021Q1.

The procedure is the following:

1. For each growth series, compute global sample quantile bounds (5% and 95%) using `compute_winsor_bounds()`.
2. Apply capping only within the COVID window using `winsorize_period()`.
3. Save the resulting winsorized versions of growth rates with suffix `_winsorized`.

```
# Winsorize growth series only during the COVID window (2020Q2-2021Q1).
# Quantile bounds are computed on the full sample for each series, then applied
# only within the selected period.
# Winsorized series are saved with suffix "_winsorized".

winsorized_gdp_data <- gdp_growth_data

growth_columns <- c("swiss_gdp_growth", "usa_gdp_growth", "euro_gdp_growth",
                    "japan_gdp_growth", "uk_gdp_growth", "china_gdp_growth",
                    "external_growth")

for (growth_rate in growth_columns) {

  bounds <- compute_winsor_bounds(
    variable = winsorized_gdp_data[[growth_rate]],
    lower_quantile = 0.05,
    upper_quantile = 0.95
  )

  winsorized_gdp_data <- winsorize_period(
    data = winsorized_gdp_data,
    variable_name = growth_rate,
    start_year = 2020, start_quarter = 2,
    end_year   = 2021, end_quarter   = 1,
    winsor_bounds = bounds
  )
}

winsorized_gdp_data <- winsorized_gdp_data %>%
  select(date, year, quarter, ends_with("_winsorized"))
```

This design preserves the original distribution outside the COVID period while limiting the leverage of extraordinary outliers in the crisis window that could bias the model selection and forecast evaluation.

5.4 Key outputs

Script 02 produces and exports three datasets:

- `data/gdp_level_data.csv`
  Quarterly GDP levels for Switzerland and partners, with imputed missing values.
- `data/gdp_growth_data.csv`
  Quarterly GDP growth rates (log-differences) and the constructed `external_growth`.
- `data/winsorized_gdp_growth.csv`
  Winsorized growth series (COVID window only), with variables ending in `_winsorized`.

```
# Save all final datasets for analysis
write_csv(gdp_level_data, "data/gdp_level_data.csv")
write_csv(gdp_growth_data, "data/gdp_growth_data.csv")
write_csv(winsorized_gdp_data, "data/winsorized_gdp_growth.csv")
```

6 Script 03 — Preliminary tests

This section documents the time-series diagnostic step implemented in `scripts/03_preliminary_tests.R`. Before estimating rolling AR and ARDL models, we run a consistent set of tests to (i) verify the time-series properties of the variables, especially stationarity, (ii) detect potential structural breaks in growth rates, (iii) assess whether the *relationship* between Swiss growth and external demand is stable over time, and (iv) test whether external demand contains predictive content for Swiss GDP growth (Granger causality). These checks provide the econometric justification for using growth transformations, targeted winsorization, and a rolling-window forecasting design.

6.1 Why these tests

6.1.1 Stationarity

Most classical inference and many forecasting/evaluation procedures rely on working with stationary processes, or at least approximately stationary series. In macro data, levels are often non-stationary, while growth rates are typically closer to stationary. We therefore test:

- GDP levels (`gdp_level_data`) to confirm non-stationarity is plausible,
- GDP growth (`gdp_growth_data`) to confirm stationarity is plausible,
- winsorized growth (`winsorized_gdp_data`) to check robustness after the COVID winsorization procedure.

6.1.2 Structural breaks and parameter stability

Even if growth is stationary, the data-generating process can change over time because of policy shifts, crises or globalization changes for instance. Structural breaks or unstable parameters motivate:

- using rolling estimation windows,
- checking stability of the Swiss–external relationship.

6.1.3 Granger causality

ARDL models assume that lags of external demand improve forecasting Swiss GDP growth. Granger tests provide a formal check of whether lagged external growth has predictive power for Swiss growth at relevant horizons (2, 4, 6, 8 quarters).

6.2 Stationarity tests

We apply both ADF and KPSS because they have opposite null hypotheses, giving a more informative diagnostic.

6.2.1 ADF test (Augmented Dickey–Fuller)

The Augmented Dickey–Fuller (ADF) test assesses whether a time series contains a unit root.

- H_0: the series has a unit root (non-stationary)
- H_1: the series is stationary

The ADF regression is typically written as:

$$\Delta y_t = c + \phi\, y_{t-1} + \sum_{k=1}^{p} \psi_k\, \Delta y_{t-k} + u_t$$

The unit-root null is:

$$H_0 : \ \phi = 0$$

and the stationary alternative is:

$$H_1 : \ \phi < 0$$

6.2.2 KPSS test (Kwiatkowski–Phillips–Schmidt–Shin)

The Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test checks whether a time series is stationary.

- H_0: the series is (trend-)stationary
- H_1: the series has a unit root (non-stationary)

The KPSS test assumes that the series can be written as:

$$y_t = d_t + r_t + \varepsilon_t$$

where d_t is a deterministic component (constant or trend), and r_t captures a possible stochastic trend.

If the variance of the random-walk component is zero, the series is stationary; otherwise, it is non-stationary.

```
level_variables <- c("gdp", "usa_gdp", "euro_gdp", "japan_gdp",
                     "uk_gdp", "china_gdp")

growth_variables <- c("swiss_gdp_growth", "usa_gdp_growth", "euro_gdp_growth",
                      "japan_gdp_growth", "uk_gdp_growth",
                      "china_gdp_growth", "external_growth")

winsor_variables <- c("swiss_gdp_growth_winsorized", "usa_gdp_growth_winsorized",
                      "euro_gdp_growth_winsorized", "japan_gdp_growth_winsorized",
                      "uk_gdp_growth_winsorized", "china_gdp_growth_winsorized",
                      "external_growth_winsorized")

if (isTRUE(params$run_pipeline)) {

  cat("\n--- LEVEL DATA ---\n")
  for (variable in level_variables) {
    cat("\nVariable:", variable, "\n")
    adf_stationarity(gdp_level_data[[variable]], lag_included = 4)
    kpss_stationarity(gdp_level_data[[variable]])
  }

  cat("\n--- GROWTH DATA ---\n")
  for (variable in growth_variables) {
    cat("\nVariable:", variable, "\n")
    adf_stationarity(gdp_growth_data[[variable]], lag_included = 4)
    kpss_stationarity(gdp_growth_data[[variable]])
  }

  cat("\n--- WINSORIZED GROWTH DATA ---\n")
  for (variable in winsor_variables) {
    cat("\nVariable:", variable, "\n")
```

```
    adf_stationarity(winsorized_gdp_data[[variable]], lag_included = 4)
    kpss_stationarity(winsorized_gdp_data[[variable]])
  }

}

# Illustrative example: Swiss GDP growth
adf_stationarity(gdp_growth_data$swiss_gdp_growth, lag_included = 4)
```

```
    Augmented Dickey-Fuller Test

data:  variable
Dickey-Fuller = -4.7984, Lag order = 4, p-value = 0.01
alternative hypothesis: stationary


Conclusion: STATIONARY (unit root rejected)
```

```
kpss_stationarity(gdp_growth_data$swiss_gdp_growth)
```

```
    KPSS Test for Level Stationarity

data:  variable
KPSS Level = 0.033462, Truncation lag parameter = 4, p-value = 0.1


Conclusion: STATIONARY (stationarity not rejected)
```

6.3 Structural break analysis

To detect potential structural changes in growth-rate dynamics, we apply the Bai–Perron multiple structural break test (via the `strucchange` package). The analysis is conducted on growth-rate series only, since the subsequent forecasting models are specified in stationary form.

6.3.1 Model and hypotheses

In line with the course presentation, the Bai–Perron framework is applied to a simple mean-shift model:

$$y_t = \mu_j + u_t, \qquad t = T_{j-1} + 1, \dots, T_j,$$

where the sample is partitioned into regimes indexed by j, each characterized by its own mean μ_j.

The break dates $\{T_j\}$ are estimated endogenously by minimizing the residual sum of squares across all admissible partitions.

To avoid overfitting, the procedure imposes a minimum segment length. In our implementation, each regime must contain at least 15% of the total sample.

The number of breaks is selected using an information criterion (BIC), following standard practice.

```
# Structural breaks indicate changes in economic relationships due to events.
# Breaks inform whether to model full sample or separate regimes.

if (isTRUE(params$run_pipeline)) {
  for (variable in growth_variables) {
    cat("\nVariable:", variable, "\n")
    bai_perron_breaks(gdp_growth_data, variable, min_segment_fraction = 0.15)
  }
}

# Illustrative example: Bai-Perron structural break test
# Swiss GDP growth (growth rates)

bai_perron_breaks(
  data = gdp_growth_data,
  series_name = "swiss_gdp_growth",
  min_segment_fraction = 0.15
)
```

```
     Optimal 1-segment partition:

Call:
breakpoints.formula(formula = y ~ 1, h = min_segment_fraction)

Breakpoints at observation number:
NA

Corresponding to breakdates:
NA

Conclusion: BIC selects 0 structural break(s)
```

6.3.2 Additional robustness checks

Because large global events can distort break selection, we also test for structural breaks over selected sub-periods. This provides a robustness check on the stability of break detection across different sample windows.

```r
# Additional tests to confirm no structural breaks in key variables.
# Testing multiple sub-periods to verify stability across different timeframes.

subperiods <- list(
  "2000-2010" = c("2000-01-01", "2010-12-31"),
  "2005-2015" = c("2005-01-01", "2015-12-31"),
  "2010-2020" = c("2010-01-01", "2020-12-31")
)

if (isTRUE(params$run_pipeline)) {
  # Swiss GDP Growth
  for (period in names(subperiods)) {
    cat("\nPeriod:", period, "\n")

    data_sub <- gdp_growth_data %>%
      filter(date >= as.Date(subperiods[[period]][1]),
             date <= as.Date(subperiods[[period]][2]))

    if (nrow(data_sub) >= 15) {
      bai_perron_breaks(data_sub, "swiss_gdp_growth", min_segment_fraction = 0.15)
    } else {
      cat("Insufficient observations\n")
    }
  }

  # External GDP Growth
  for (period in names(subperiods)) {
    cat("\nPeriod:", period, "\n")

    data_sub <- gdp_growth_data %>%
      filter(date >= as.Date(subperiods[[period]][1]),
             date <= as.Date(subperiods[[period]][2]))

    if (nrow(data_sub) >= 15) {
      bai_perron_breaks(data_sub, "external_growth", min_segment_fraction = 0.15)
    } else {
      cat("Insufficient observations\n")
    }
  }
}
```

6.4 Relationship stability tests

Even if growth rates are stationary, the relationship between Swiss GDP growth and external demand growth may vary over time.
To assess parameter stability, we apply a recursive CUSUM test based on recursive residuals,

following the approach presented in the course.

The test is applied to the following predictive regression:

$$y_t = \alpha + \beta x_t + \varepsilon_t,$$

where y_t denotes Swiss GDP growth and x_t denotes external demand growth.

6.4.1 Hypothesis and interpretation

- H_0: the regression parameters (α, β) are stable over time
- H_1: at least one parameter changes over time

The CUSUM statistic is constructed from recursive residuals and compared to time-varying critical boundaries. Crossing the confidence bands indicates a structural instability in the relationship.

```
# Testing whether the relationship between Swiss GDP growth and external GDP
# growth is stable over time. Parameter instability provides a justification
# for using a rolling window estimation strategy.


  cat("RELATIONSHIP STABILITY TEST\n")
```

```
RELATIONSHIP STABILITY TEST
```

```
  cat("\nWinsorized data:\n")
```

```

Winsorized data:
```

```
  rec_cusum_test(
    formula = swiss_gdp_growth_winsorized ~ external_growth_winsorized,
    data = winsorized_gdp_data
  )
```

```

Rec-CUSUM | Model: swiss_gdp_growth_winsorized ~ external_growth_winsorized

Empirical Fluctuation Process: Recursive CUSUM test

Call: strucchange::efp(formula = formula, data = data, type = "Rec-CUSUM")


Conclusion: Stable relationship (p = 0.2551 )
```

```r
  cat("\nRaw data:\n")
```

```
Raw data:
```

```r
  rec_cusum_test(
    formula = swiss_gdp_growth ~ external_growth,
    data = gdp_growth_data
  )
```

```
Rec-CUSUM | Model: swiss_gdp_growth ~ external_growth

Empirical Fluctuation Process: Recursive CUSUM test

Call: strucchange::efp(formula = formula, data = data, type = "Rec-CUSUM")


Conclusion: Stable relationship (p = 0.2719 )
```

6.5 Granger causality tests

To assess whether external demand improves forecasts of Swiss GDP growth, we conduct Granger causality tests at multiple lag horizons. These tests examine whether lagged values of external demand growth contain predictive information for Swiss GDP growth beyond its own past.

For a given lag length L, the unrestricted regression is:

$$y_t = c + \sum_{k=1}^{L} \phi_k y_{t-k} + \sum_{k=1}^{L} \beta_k x_{t-k} + u_t,$$

where y_t denotes Swiss GDP growth and x_t denotes external demand growth.

The null hypothesis of the Granger causality test is:

$$H_0 : \ \beta_1 = \beta_2 = \cdots = \beta_L = 0,$$

which states that lagged external demand does not improve the prediction of Swiss GDP growth.

The null hypothesis is tested using a joint significance test (Wald or F-test), as presented in the course.

We test multiple lag horizons $L \in \{2, 4, 6, 8\}$ to allow for different transmission speeds from foreign demand to domestic economic activity.

```r
# Testing multiple lag horizons (2, 4, 6, 8 quarters) on full sample periods.
# Multiple lags capture different economic transmission speeds.

if (isTRUE(params$run_pipeline)) {
# GDP growth data
  granger_causality(
    data = gdp_growth_data,
    x = "external_growth",
    y = "swiss_gdp_growth",
    lags = c(2, 4, 6, 8),
    winsorized = FALSE
  )

  # Winsorized growth data
  granger_causality(
    data = winsorized_gdp_data,
    x = "external_growth_winsorized",
    y = "swiss_gdp_growth_winsorized",
    lags = c(2, 4, 6, 8),
    winsorized = TRUE
  )

}
# Illustrative example: Granger causality test
# Lag = 4 quarters, winsorized data

granger_causality(
data = winsorized_gdp_data,
x = "external_growth_winsorized",
y = "swiss_gdp_growth_winsorized",
lags = 4,
winsorized = TRUE
)
```

```
=== Granger causality tests ===
Series: external_growth_winsorized and swiss_gdp_growth_winsorized | Winsorized: TRUE

--- Lag = 4 ---
Granger causality test

Model 1: swiss_gdp_growth_winsorized ~ Lags(swiss_gdp_growth_winsorized, 1:4) + Lags(external_g
Model 2: swiss_gdp_growth_winsorized ~ Lags(swiss_gdp_growth_winsorized, 1:4)
  Res.Df Df      F  Pr(>F)
1     90
2     94 -4 2.9794 0.02325 *
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Granger causality test

Model 1: external_growth_winsorized ~ Lags(external_growth_winsorized, 1:4) + Lags(swiss_gdp_gr
Model 2: external_growth_winsorized ~ Lags(external_growth_winsorized, 1:4)
  Res.Df Df      F Pr(>F)
1     90
2     94 -4 1.7141 0.1538

Causality: external_growth_winsorized → swiss_gdp_growth_winsorized
```

6.5.1 How to interpret the results

- **CUSUM stability:** Evidence of parameter stability supports the use of a stable forecasting relationship over time. Instability concentrated in crisis periods (e.g. COVID) motivates winsorization and rolling-window estimation.
- **Granger causality:** Rejection of the null for some lag horizons indicates that external demand has predictive content for Swiss GDP growth, providing empirical support for the ARDL forecasting framework. Failure to reject at all horizons suggests limited incremental forecasting gains relative to univariate benchmarks, an issue revisited in the forecast evaluation stage.

7 Script 04 — Rolling model estimation

This section documents the rolling estimation step implemented in `scripts/04_model_estimation_rolling.R`. The script estimates (i) a Swiss AR benchmark, (ii) an ARMA model for the external-demand growth proxy, and (iii) the main Swiss ARDL model, using a rolling window estimation and BIC-based lag selection. It exports the resulting time-varying specifications and coefficients, then refits the final-window ARDL to run basic residual diagnostics.

7.1 Model set-up

Let y_t denote Swiss GDP growth and x_t denote the external-demand growth index.
All models are estimated on stationary growth-rate series within rolling windows of fixed length.

7.1.1 AR benchmark

The benchmark model for Swiss GDP growth is an autoregressive process of order p:

$$y_t = c + \sum_{i=1}^{p} \phi_i \, y_{t-i} + \varepsilon_t,$$

where ε_t is a white-noise error term.

7.1.2 External-demand process

The external-demand growth series is modeled as an ARMA(p_x, q_x) process:

$$x_t = c_x + \sum_{i=1}^{p_x} a_i \, x_{t-i} + u_t + \sum_{j=1}^{q_x} b_j \, u_{t-j},$$

where u_t is an innovation assumed to be serially uncorrelated.

This ARMA specification is used to model the dynamic properties of external demand growth and to support recursive multi-horizon forecasting in the subsequent stage.

7.1.3 Swiss ARDL model

The main forecasting model is an autoregressive distributed lag (ARDL) specification:

$$y_t = \alpha + \sum_{i=1}^{p} \phi_i \, y_{t-i} + \theta_0 \, x_t + \sum_{j=1}^{q} \theta_j \, x_{t-j} + \varepsilon_t.$$

This specification allows contemporaneous and lagged external demand to affect Swiss GDP growth, while controlling for its own persistence.

7.1.4 Lag selection and rolling estimation

Within each rolling window, the lag orders (p, q) for the ARDL model, as well as (p_x, q_x) for the ARMA model of x_t, are selected by minimizing the Bayesian Information Criterion (BIC).

The BIC is defined as:

$$\mathrm{BIC} = -2 \log(\hat{L}) + k \log(n),$$

where $\hat{L}$ denotes the maximized likelihood, k is the number of estimated parameters, and n is the sample size within the rolling window.

The use of BIC favors parsimonious specifications in finite samples and rolling estimation settings.

7.2 Initialization of rolling parameters

The rolling estimation procedure is initialized by specifying the window size, the maximum lag orders considered, and the variables entering the models.

The window length is set to 40 quarters (10 years), which balances estimation precision with the need to allow for time variation in parameters.

Lag orders are restricted to a maximum of three, in line with the quarterly frequency and the parsimony principle emphasized in the course.

The rolling windows are defined by their ending indices, starting from the first observation for which a full window is available.

```
# Window size in quarters (40 = 10 years)
window_size <- 40

# Max lag order considered for AR / ARDL
max_lag <- 3

# Columns in the winsorized dataset
y_col <- "swiss_gdp_growth_winsorized"
x_col <- "external_growth_winsorized"

# Endpoints for rolling windows (start at the first full window)
end_indices <- window_size:nrow(winsorized_gdp_data)
```

7.3 Rolling estimation

Rolling window estimation is used to allow both model parameters and selected lag structures to evolve over time.
This approach is standard in macroeconomic forecasting when structural change, regime shifts, or time-varying transmission mechanisms are likely.

Following the notation used in the course, let H denote the rolling window length (in quarters).

At each window endpoint T, all models are estimated using the subsample:

$$\{T - H + 1, \dots, T\}.$$

Within each rolling window, the script estimates:

- an autoregressive benchmark for Swiss GDP growth,
- an ARMA specification for the external-demand growth index,
- and an ARDL forecasting model linking Swiss GDP growth to external demand.

For the ARDL model, all admissible lag combinations are evaluated within each window, and the specification minimizing BIC is retained.

The rolling estimation is implemented via the function `estimate_rolling_models()`, which stores for each window:

- the selected lag orders for each model,
- the corresponding coefficient estimates and standard errors.

```
rolling_output <- estimate_rolling_models(
  data        = winsorized_gdp_data,
  y_col       = y_col,
  x_col       = x_col,
  window_size = window_size,
  max_lag     = max_lag,
  end_indices = end_indices
)

rolling_specifications <- rolling_output$specifications
rolling_coefficients   <- rolling_output$coefficients
```

To keep the rendered companion paper readable, only a small preview of the results is displayed below.

```
print(head(rolling_specifications, 10))
```

```
         Date         Model_Type Best_P Best_Q BIC_Score     Sigma
p  2009-10-01 Swiss_AR_Benchmark      1      0  84.84052 0.6410136
p1 2009-10-01      External_ARMA      1      0  79.54786 0.5994237
1  2009-10-01         Swiss_ARDL      1      0  61.48421 0.4766481
p2 2010-01-01 Swiss_AR_Benchmark      1      0  85.38663 0.6261487
p3 2010-01-01      External_ARMA      1      0  78.12979 0.5883022
11 2010-01-01         Swiss_ARDL      1      0  58.92647 0.4604546
p4 2010-04-01 Swiss_AR_Benchmark      1      0  85.12736 0.6241453
p5 2010-04-01      External_ARMA      1      0  78.46711 0.5907543
12 2010-04-01         Swiss_ARDL      1      0  56.03873 0.4428321
p6 2010-07-01 Swiss_AR_Benchmark      1      0  84.93337 0.6224661
```

```
print(head(rolling_coefficients, 10))
```

```
                Date         Model_Type        Term   Estimate  Std_Error
ar1       2009-10-01 Swiss_AR_Benchmark         ar1 0.64448594 0.11658759
ar11      2009-10-01      External_ARMA         ar1 0.67601958 0.11232722
1         2009-10-01         Swiss_ARDL (Intercept) 0.14526091 0.09275520
11        2009-10-01         Swiss_ARDL      y_lag1 0.07985025 0.13431703
12        2009-10-01         Swiss_ARDL      x_lag0 0.73467914 0.13460762
```

```
ar12      2010-01-01 Swiss_AR_Benchmark          ar1 0.51631044 0.13296349
intercept 2010-01-01 Swiss_AR_Benchmark    intercept 0.47050766 0.19440998
ar13      2010-01-01      External_ARMA          ar1 0.70628482 0.11085653
13        2010-01-01         Swiss_ARDL (Intercept) 0.13621075 0.08954323
14        2010-01-01         Swiss_ARDL      y_lag1 0.07384849 0.12979181
```

7.4 Exporting rolling estimation results

The rolling estimation step produces two key outputs:

(i) the sequence of selected model specifications over time, and

(ii) the corresponding estimated coefficients.

These outputs are saved to disk to ensure full reproducibility of the forecasting stage and to decouple model estimation from forecast generation.

```r
write.csv(
  rolling_specifications,
  "output/rolling_model_specs.csv",
  row.names = FALSE
)

write.csv(
  rolling_coefficients,
  "output/rolling_model_coefficients.csv",
  row.names = FALSE
)
```

The exported files are:

- output/rolling_model_specs.csv: selected AR, ARMA, and ARDL specifications for each rolling window,
- output/rolling_model_coefficients.csv: corresponding estimated coefficients and standard errors.

These files are loaded directly in the forecasting script to generate recursive multi-horizon forecasts.

7.5 Final-window diagnostics (residual checks)

Even though the ARDL model is re-selected in each rolling window, it is important to verify that the final selected specification in the last window does not exhibit major residual pathologies.

We therefore re-estimate the last-window ARDL(p, q) on the most recent `window_size` observations, reconstructing the lag structure implied by the stored rolling specification, and then apply three standard residual diagnostics:

- **Ljung–Box test on residuals**
  This test examines whether the residuals from the estimated model are serially uncorrelated up to a given lag order. Formally, it tests the null hypothesis that all autocorrelations of the residuals up to lag h are jointly equal to zero.
  Passing this test (i.e. not rejecting the null) suggests that the model has adequately captured the dynamic structure of the data and that the residuals behave like white noise, which is a key requirement for reliable inference and forecasting.

- **Ljung–Box test on squared residuals**
  Applying the Ljung–Box test to *squared* residuals is a standard way to detect remaining conditional heteroskedasticity of the ARCH type. The null hypothesis is that there is no autocorrelation in squared residuals.
  Rejecting this null would indicate time-varying volatility not accounted for by the model, which could affect forecast uncertainty and motivate more complex variance modeling. Not rejecting the null supports the assumption of approximately constant conditional variance.

- **Jarque–Bera test**
  The Jarque–Bera test assesses whether the residuals are normally distributed by jointly testing whether their skewness and kurtosis match those of a normal distribution. The null hypothesis is normality of the residuals.
  While strict normality is not required for consistency of OLS estimates, approximate normality is useful for interpretation, inference, and for simulation-based forecasting methods that assume Gaussian shocks.

These diagnostics are reported for the final estimation window only and are used as a qualitative check of model adequacy rather than as formal model selection criteria.

```
# We re-estimate the ARDL model on the very last window to check residual health.
# Get parameters from the last estimated rolling window
last_specification <- tail(
  rolling_specifications[rolling_specifications$Model_Type == "Swiss_ARDL", ],
  1
)

last_p    <- last_specification$Best_P
last_q    <- last_specification$Best_Q
last_date <- last_specification$Date

# Re-create the data for this specific window
n_total <- nrow(winsorized_gdp_data)
index_start <- n_total - window_size + 1

y_win <- winsorized_gdp_data[[y_col]][index_start:n_total]
x_win <- winsorized_gdp_data[[x_col]][index_start:n_total]

# Re-fit the specific ARDL structure found by the rolling loop
n_win <- length(y_win)
y_target <- y_win[(max_lag + 1):n_win]
reg_list <- list()
```

```r
  # Construct lags based on last_p and last_q
  if (last_p > 0) {
    for (k in 1:last_p) {
      reg_list[[paste0("y_lag", k)]] <-
        y_win[(max_lag + 1 - k):(n_win - k)]
    }
  }

  reg_list[["x_lag0"]] <- x_win[(max_lag + 1):n_win]

  if (last_q > 0) {
    for (k in 1:last_q) {
      reg_list[[paste0("x_lag", k)]] <-
        x_win[(max_lag + 1 - k):(n_win - k)]
    }
  }

  df_diag <- as.data.frame(reg_list)
  df_diag$y <- y_target

  # Fit final-window ARDL
  final_fit <- lm(y ~ ., data = df_diag)

  cat("\n=== DIAGNOSTICS: FINAL ARDL WINDOW ===\n")
```

```
=== DIAGNOSTICS: FINAL ARDL WINDOW ===
```

```r
  cat("Last window end date:", as.character(last_date), "\n")
```

```
Last window end date: 2025-07-01
```

```r
  cat("Selected ARDL order: p =", last_p, ", q =", last_q, "\n\n")
```

```
Selected ARDL order: p = 1 , q = 0
```

```r
  # Autocorrelation in residuals
  ljung_box_residuals(final_fit, lag = 10)
```

```

    Box-Ljung test

data:  residuals
X-squared = 6.0324, df = 10, p-value = 0.8125
```

```
Conclusion: No significant autocorrelation
        (residuals behave as white noise)
```

```
  # Conditional heteroskedasticity
  ljung_box_squared_residuals(final_fit, lag = 5)
```

```
    Box-Ljung test

data:  squared_residuals
X-squared = 0.95208, df = 5, p-value = 0.9663
```

```
Conclusion: No evidence of conditional heteroskedasticity
        (variance is stable)
```

```
  # Normality of residuals
  jarque_bera_residuals(final_fit)
```

```
    Jarque Bera Test

data:  residuals
X-squared = 0.37422, df = 2, p-value = 0.8294
```

```
Conclusion: Residuals are normal
```

8 Script 05 — Rolling forecasting

This section documents the forecasting and evaluation stage of the project. Building on the rolling model specifications and coefficients estimated in Script 04, we conduct a recursive multi-horizon forecasting exercise, assess predictive performance on a stable pre-COVID sample, and generate out-of-sample forecasts for the period 2025–2027. Finally, quarterly growth forecasts are aggregated into annual growth rates for policy-relevant reporting.

8.1 Forecasting framework

8.1.1 Recursive rolling-window forecasting

Forecasts are generated using a rolling-window recursive forecasting framework. For each forecast origin date t, models are estimated using a rolling window of fixed length (40 quarters), and forecasts are produced for horizons:

$$h = 1, 2, \ldots, H$$

using only information available at time t.

Let y_t denote Swiss GDP growth and x_t external demand growth. For a given information set Ω_t at time t, the h-step-ahead forecast is:

$$\hat{y}_{t+h|t} = \mathbb{E}_t \left[y_{t+h} \mid \Omega_t \right].$$

Forecasts are recursive: for $h > 1$, previously forecasted values are fed back into the model whenever lagged dependent variables are required.

8.1.2 Models compared

Two models are evaluated:

- **AR benchmark**: Swiss GDP growth depends only on its own lags.
- **ARDL model**: Swiss GDP growth depends on its own lags and lagged external demand growth.

Both models are re-selected dynamically in each rolling window using the Bayesian Information Criterion (BIC).

8.1.3 Multi-horizon recursive backtesting

Using the rolling specifications and coefficients saved in Script 04, we generate recursive forecasts up to $H = 10$ quarters ahead. This produces a panel of forecasts indexed by forecast origin date and horizon.

```
max_h       <- 10
max_lag     <- 3
window_size <- 40

y_col <- "swiss_gdp_growth_winsorized"
x_col <- "external_growth_winsorized"

# Run multi-horizon recursive rolling forecasts.
# For each forecast origin, models are re-estimated on a rolling window and
# forecasts are generated recursively up to horizon max_h using only information
# available at the time of the forecast.
multi_horizon_results <- multi_horizon_recursive_forecasting(
  data      = winsorized_gdp_data,
  specifications     = rolling_specifications,
  coefficients     = rolling_coefficients,
  y_col     = y_col,
  x_col     = x_col,
```

```
  max_h     = max_h,
  max_lag   = max_lag
)
```

8.2 Forecast Evaluation

Forecast evaluation is restricted to the pre-COVID period (up to 2019Q4). Since no explicit COVID dummy or regime-switching mechanism is included in the models, the extreme volatility observed during 2020–2021 would dominate forecast errors and obscure differences in underlying predictive performance.

Restricting the evaluation sample allows us to focus on a relatively stable macroeconomic environment and to assess the models' ability to forecast under "normal" business-cycle conditions.

```
# Since we are not using a Dummy variable to handle the massive COVID shock,
# we evaluate performance only on the stable period (2010-2019).
# This prevents the 2020 volatility from destroying the error metrics.
evaluation_cutoff <- as.Date("2019-12-31")

# Filter the results for evaluation ONLY
evaluation_data <- subset(multi_horizon_results,
                          origin_date <= evaluation_cutoff)

horizons <- sort(unique(evaluation_data$horizon))
```

8.2.1 Absolute forecast evaluation

This section evaluates absolute predictive performance of the two competing models (AR vs ARDL) at each forecast horizon h. The objective is to quantify how close forecasts are to realized Swiss GDP growth, using standard loss-based accuracy measures.

Let the h-step-ahead forecast error be:

$$e_{t+h|t} = y_{t+h} - \hat{y}_{t+h|t}.$$

For each horizon h, we compute the following accuracy measures over the evaluation sample of size T_h.

8.2.1.1 Mean Forecast Error (MFE)

The Mean Forecast Error captures the average bias:

$$\text{MFE}(h) = \frac{1}{T_h} \sum_{t=1}^{T_h} e_{t+h|t}.$$

- If $\text{MFE}(h) > 0$, forecasts are on average **too low** (under-predict).

- If $\mathrm{MFE}(h) < 0$, forecasts are on average **too high** (over-predict).

8.2.1.2 Mean Squared Forecast Error (MSFE)

The Mean Squared Forecast Error penalizes large errors more heavily:

$$\mathrm{MSFE}(h) = \frac{1}{T_h} \sum_{t=1}^{T_h} e_{t+h|t}^2.$$

It corresponds to expected squared loss under quadratic preferences.

8.2.1.3 Mean Absolute Forecast Error (MAFE)

The Mean Absolute Forecast Error uses absolute loss:

$$\mathrm{MAFE}(h) = \frac{1}{T_h} \sum_{t=1}^{T_h} \left| e_{t+h|t} \right|.$$

Compared with MSFE, MAFE is less sensitive to extreme outliers.

8.2.1.4 Root Mean Squared Error (RMSE)

The Root Mean Squared Error is defined as the square root of the MSFE and is expressed in the same units as the target variable:

$$\mathrm{RMSE}(h) = \sqrt{\mathrm{MSFE}(h)}.$$

```
absolute_results <- data.frame()

for (h in horizons) {

  h_data <- subset(evaluation_data, horizon == h)
  if (nrow(h_data) < 10) next

  # METRICS (AR + ARDL)
  metrics_ardl <- forecast_metrics(h_data$y_real, h_data$y_hat_ardl)
  metrics_ar   <- forecast_metrics(h_data$y_real, h_data$y_hat_ar)

  absolute_results <- rbind(absolute_results, data.frame(
    Horizon = h,

    Bias_AR   = round(metrics_ar$mfe, 4),
    Bias_ARDL = round(metrics_ardl$mfe, 4),

    MSFE_AR   = round(metrics_ar$msfe, 4),
    MSFE_ARDL = round(metrics_ardl$msfe, 4),
```

```
    MAFE_AR   = round(metrics_ar$mae, 4),
    MAFE_ARDL = round(metrics_ardl$mae, 4),

    RMSE_AR   = round(metrics_ar$rmse, 4),
    RMSE_ARDL = round(metrics_ardl$rmse, 4),

    stringsAsFactors = FALSE
  ))
}

cat("\n--- ABSOLUTE METRICS (Pre-Covid) ---\n")
```

```
--- ABSOLUTE METRICS (Pre-Covid) ---
```

```
print(absolute_results)
```

```
   Horizon Bias_AR Bias_ARDL MSFE_AR MSFE_ARDL MAFE_AR MAFE_ARDL RMSE_AR
1        1 -0.1821   -0.0458  0.2572    0.1713  0.3826    0.3148  0.5072
2        2 -0.2776   -0.0210  0.3195    0.2091  0.4544    0.3381  0.5653
3        3 -0.2889    0.0427  0.3266    0.2438  0.4792    0.3795  0.5715
4        4 -0.3011    0.0786  0.3974    0.2391  0.5175    0.3663  0.6304
5        5 -0.3154    0.0932  0.3443    0.2288  0.4618    0.3609  0.5868
6        6 -0.2602    0.1655  0.4621    0.3078  0.5305    0.4004  0.6798
7        7 -0.2313    0.2050  0.5261    0.3570  0.5730    0.4275  0.7253
8        8 -0.1953    0.2487  0.5552    0.3456  0.5744    0.4260  0.7451
9        9 -0.1960    0.2535  0.4964    0.3343  0.5415    0.4301  0.7045
10      10 -0.1860    0.2674  0.4839    0.3675  0.5415    0.4396  0.6956
   RMSE_ARDL
1     0.4138
2     0.4572
3     0.4937
4     0.4890
5     0.4783
6     0.5548
7     0.5975
8     0.5879
9     0.5782
10    0.6062
```

Absolute accuracy measures summarize forecast performance, but they do not assess whether forecasts satisfy standard optimality conditions implied by rational forecasting. We therefore complement the analysis with a set of formal statistical tests applied horizon by horizon.

8.2.1.5 Unbiasedness t-test

A forecast is said to be unbiased if its forecast errors have zero mean. For each horizon h, we test:

$$H_0 : \mathbb{E}(e_{t+h|t}) = 0,$$

where $e_{t+h|t} = y_{t+h} - \hat{y}_{t+h|t}$.

Operationally, this hypothesis is tested using a standard t-test on the mean of forecast errors. Rejection of H_0 indicates systematic over- or under-prediction.

```
test_results_t_unbiasedness <- data.frame()

for (h in horizons) {

  h_data <- subset(evaluation_data, horizon == h)
  if (nrow(h_data) < 10) next

  t_ardl <- t_test_unbiasedness(h_data$y_real, h_data$y_hat_ardl)
  t_ar   <- t_test_unbiasedness(h_data$y_real, h_data$y_hat_ar)

  test_results_t_unbiasedness <- rbind(
    test_results_t_unbiasedness,
    data.frame(
      Horizon = h,
      t_stat_AR   = round(t_ar$t_stat, 3),
      pval_AR     = round(t_ar$p_val, 4),
      t_stat_ARDL = round(t_ardl$t_stat, 3),
      pval_ARDL   = round(t_ardl$p_val, 4),
      stringsAsFactors = FALSE
    )
  )
}

print(test_results_t_unbiasedness)
```

```
   Horizon t_stat_AR pval_AR t_stat_ARDL pval_ARDL
1        1    -2.433  0.0195      -0.704    0.4855
2        2    -3.566  0.0010      -0.290    0.7729
3        3    -3.706  0.0006       0.549    0.5859
4        4    -3.438  0.0014       1.030    0.3092
5        5    -4.032  0.0002       1.257    0.2161
6        6    -2.620  0.0124       1.976    0.0550
7        7    -2.128  0.0395       2.311    0.0261
8        8    -1.718  0.0936       2.952    0.0053
9        9    -1.832  0.0744       3.085    0.0037
10      10    -1.755  0.0869       3.109    0.0035
```

A failure to reject H_0 supports the absence of systematic bias at the corresponding forecast horizon.

8.2.1.6 Mincer–Zarnowitz regression tests

The Mincer–Zarnowitz (MZ) regression assesses whether forecasts are both unbiased and correctly scaled. For each forecast horizon h, we estimate the regression:

$$y_{t+h} = \alpha_h + \beta_h \hat{y}_{t+h|t} + \epsilon_{t+h},$$

and test the joint null hypothesis:

$$H_0 : \ \alpha_h = 0 \quad \text{and} \quad \beta_h = 1.$$

- $\alpha_h \neq 0$ indicates systematic bias.
- $\beta_h \neq 1$ indicates incorrect forecast calibration (over- or under-reaction).
- Failure to reject H_0 supports forecast optimality in the Mincer–Zarnowitz sense.

```
test_results_mz <- data.frame()

for (h in horizons) {

  h_data <- subset(evaluation_data, horizon == h)
  if (nrow(h_data) < 10) next

  mz_ardl <- mincer_zarnowitz_test(h_data$y_real, h_data$y_hat_ardl)
  mz_ar   <- mincer_zarnowitz_test(h_data$y_real, h_data$y_hat_ar)

  test_results_mz <- rbind(
    test_results_mz,
    data.frame(
      Horizon = h,

      alpha_AR   = round(mz_ar$alpha_hat, 4),
      beta_AR    = round(mz_ar$beta_hat, 4),
      F_AR       = round(mz_ar$F_stat, 3),
      pval_AR    = round(mz_ar$p_val, 4),

      alpha_ARDL = round(mz_ardl$alpha_hat, 4),
      beta_ARDL  = round(mz_ardl$beta_hat, 4),
      F_ARDL     = round(mz_ardl$F_stat, 3),
      pval_ARDL  = round(mz_ardl$p_val, 4),

      stringsAsFactors = FALSE
    )
  )
}
```

```
print(test_results_mz)
```

```
   Horizon alpha_AR beta_AR   F_AR pval_AR alpha_ARDL beta_ARDL F_ARDL
1        1   0.3927  0.0646 30.792       0     0.2022    0.4816  3.127
2        2   0.3424  0.0784 74.674       0     0.7305   -0.8057  6.099
3        3   0.2304  0.2631 62.227       0     0.9348   -1.3908 10.679
4        4   0.4695 -0.0670 63.131       0     0.6313   -0.6134  6.730
5        5   0.2191  0.2673 53.507       0     0.4648   -0.1578  6.830
6        6   0.4410  0.0414 36.315       0     0.0051    1.5246  4.227
7        7   0.5670 -0.0916 40.337       0    -0.0899    1.9999  5.852
8        8   0.8504 -0.4297 65.975       0    -0.1078    2.2399  9.409
9        9   0.6180 -0.1126 59.398       0    -0.1927    2.5813 11.159
10      10   0.5969 -0.0698 58.016       0     0.0529    1.7705  9.788
   pval_ARDL
1     0.2094
2     0.0474
3     0.0048
4     0.0346
5     0.0329
6     0.1208
7     0.0536
8     0.0091
9     0.0038
10    0.0075
```

A failure to reject the joint null hypothesis indicates that forecasts are both unbiased and correctly scaled at horizon h.

8.2.1.7 Forecast efficiency test

Beyond unbiasedness, a forecast is often considered efficient if its errors cannot be systematically predicted using information that was already available at the time the forecast was produced. Intuitively: if you can explain today's forecast errors with variables you already knew at time t, then the forecast left "easy predictability" on the table and is not informationally efficient.

Let the forecast error be:

$$e_{t+h|t} = y_{t+h} - \hat{y}_{t+h|t}.$$

A simple efficiency check regresses the forecast error on an information set available at time t. In our implementation, the information set is kept minimal and uses the model's own forecast (a standard diagnostic):

$$e_{t+h|t} = \gamma_0 + \gamma_1 \hat{y}_{t+h|t} + u_{t+h}.$$

The null hypothesis of forecast efficiency is:

$$H_0 : \gamma_1 = 0,$$

i.e. the forecast error is not predictable using the forecast itself.
We test H_0 using an F-test (with HAC correction inside the function to account for potential overlap across horizons).

If we fail to reject H_0, the forecast errors do not exhibit systematic predictability with respect to the chosen information set, supporting efficiency.

```
# Efficiency test: regress forecast errors on an information set available at t.
# H0: coefficients on information variables are jointly zero (efficiency).
# Here: info set is kept minimal (the forecast itself), to avoid overfitting.

test_results_efficiency <- data.frame()

for (h in horizons) {

  h_data <- subset(evaluation_data, horizon == h)
  if (nrow(h_data) < 10) next

  # Information sets (available at forecast origin)
  info_ar   <- data.frame(y_hat = h_data$y_hat_ar)
  info_ardl <- data.frame(y_hat = h_data$y_hat_ardl)

  # Run efficiency tests (helper handles HAC / overlap if implemented)
  efficiency_ar   <- efficiency_test(h_data$y_real, h_data$y_hat_ar,   info = info_ar)
  efficiency_ardl <- efficiency_test(h_data$y_real, h_data$y_hat_ardl, info = info_ardl)

  # Store results
  test_results_efficiency <- rbind(
    test_results_efficiency,
    data.frame(
      Horizon   = h,
      F_AR      = round(efficiency_ar$F_stat, 3),
      pval_AR   = round(efficiency_ar$p_val, 4),
      F_ARDL    = round(efficiency_ardl$F_stat, 3),
      pval_ARDL = round(efficiency_ardl$p_val, 4),
      stringsAsFactors = FALSE
    )
  )
}

# Preview (optional)
print(test_results_efficiency)
```

	Horizon	F_AR	pval_AR	F_ARDL	pval_ARDL
1	1	12.940	0.0009	1.095	0.3018
2	2	13.236	0.0008	7.578	0.0089
3	3	8.858	0.0050	12.167	0.0012
4	4	18.712	0.0001	5.656	0.0224
5	5	9.568	0.0037	3.347	0.0750
6	6	10.629	0.0023	0.561	0.4582
7	7	12.033	0.0013	2.038	0.1614
8	8	23.251	0.0000	3.838	0.0573
9	9	13.464	0.0007	7.381	0.0098
10	10	12.482	0.0011	1.431	0.2388

A large p-value (e.g. > 0.10) means we do not find evidence against efficiency at that horizon, whereas small p-values indicate that forecast errors are predictable using the chosen information set (inefficiency).

8.2.2 Relative evaluation

Absolute accuracy metrics evaluate each model in isolation. We now compare the ARDL competitor against the AR benchmark directly, horizon by horizon, using (i) loss ratios and (ii) a formal Diebold–Mariano test.

Let the forecast errors at horizon (h) be:

$$e_{t+h|t}^{(m)} = y_{t+h} - \widehat{y}_{t+h|t}^{(m)}, \qquad m \in \{\text{ARDL}, \text{AR}\}.$$

8.2.2.1 MSFE ratio

The Mean Squared Forecast Error (MSFE) at horizon (h) is:

$$\text{MSFE}^{(m)}(h) = \frac{1}{N_h} \sum_{t=1}^{N_h} \left(e_{t+h|t}^{(m)}\right)^2.$$

We report the MSFE ratio:

$$\text{MSFE Ratio}(h) = \frac{\text{MSFE}^{(\text{ARDL})}(h)}{\text{MSFE}^{(\text{AR})}(h)}.$$

The MSFE places a quadratic penalty on forecast errors. As a result, large forecast errors receive disproportionately high weight in the evaluation.

This criterion is particularly sensitive to large forecast misses, extreme events or tail realizations and periods of high volatility.

Interpretation:

- If (MSFE Ratio(h) < 1): ARDL improves squared-loss performance relative to AR.
- If (MSFE Ratio(h) > 1): AR benchmark performs better.

8.2.2.2 MAFE ratio

The Mean Absolute Forecast Error (MAFE) at horizon (h) is:

$$\mathrm{MAFE}^{(m)}(h) = \frac{1}{N_h} \sum_{t=1}^{N_h} \left| e^{(m)}_{t+h|t} \right|.$$

The MAFE ratio is:

$$\text{MAFE Ratio}(h) = \frac{\mathrm{MAFE}^{(\mathrm{ARDL})}(h)}{\mathrm{MAFE}^{(\mathrm{AR})}(h)}.$$

The MAFE uses an absolute loss function, assigning a linear penalty to forecast errors.

Compared to MSFE, MAFE treats all deviations proportionally, is less influenced by a small number of extreme errors and provides a more robust measure of typical forecast accuracy.

A MAFE ratio below one therefore indicates that the ARDL model delivers smaller errors on average, even if it does not necessarily dominate in terms of large tail errors.

8.2.2.3 Diebold–Mariano test

To test whether the two models have the same expected predictive loss, define the loss differential:

$$d_{t,h} = L\big(e^{(\mathrm{ARDL})}_{t+h|t}\big) - L\big(e^{(\mathrm{AR})}_{t+h|t}\big)\,,$$

with squared loss (L(e)=e^2) in our implementation. The null hypothesis is:

$$H_0 : \ \mathbb{E}(d_{t,h}) = 0,$$

i.e., equal predictive accuracy. Rejection suggests a statistically meaningful difference in forecast performance (accounting for serial correlation induced by multi-step/overlapping forecasts).

The Diebold–Mariano test is interpreted conditional on the chosen loss function and horizon, and its power may decrease at long horizons.

```
# Compare ARDL against AR benchmark horizon-by-horizon using:
# - MSFE ratio (squared loss)
# - MAFE ratio (absolute loss)
# - Diebold-Mariano test (equal predictive accuracy)

relative_results <- data.frame()

for (h in horizons) {

  h_data <- subset(evaluation_data, horizon == h)
  if (nrow(h_data) < 10) next

  # --- MSFE ratio ---
  msfe <- msfe_ratio(
    e1 = h_data$e_ardl,
```

```
    e2 = h_data$e_ar,
    name1 = "ARDL",
    name2 = "AR"
  )

  # --- MAFE ratio ---
  mafe <- mafe_ratio(
    e1 = h_data$e_ardl,
    e2 = h_data$e_ar,
    name1 = "ARDL",
    name2 = "AR"
  )

  # --- Diebold-Mariano test (squared loss) ---
  dm <- dm_test_eval(
    e1 = h_data$e_ardl,
    e2 = h_data$e_ar,
    h  = h,
    name1 = "ARDL",
    name2 = "AR"
  )

  relative_results <- rbind(relative_results, data.frame(
    Horizon     = h,

    MSFE_ARDL   = round(msfe$msfe1, 4),
    MSFE_AR     = round(msfe$msfe2, 4),
    MSFE_Ratio  = round(msfe$ratio, 4),   # < 1 => ARDL better under squared loss

    MAFE_ARDL   = round(mafe$mafe1, 4),
    MAFE_AR     = round(mafe$mafe2, 4),
    MAFE_Ratio  = round(mafe$ratio, 4),   # < 1 => ARDL better under absolute loss

    DM_Pval     = round(dm$p_val, 4),
    Signif_10pct = ifelse(!is.na(dm$p_val) & dm$p_val < 0.10, "YES", "NO"),

    stringsAsFactors = FALSE
  ))
}

# Keep rendering readable: print a small preview only
print(relative_results)
```

```
  Horizon MSFE_ARDL MSFE_AR MSFE_Ratio MAFE_ARDL MAFE_AR MAFE_Ratio DM_Pval
1       1    0.1713  0.2572     0.6657    0.3148  0.3826     0.8228  0.0338
2       2    0.2091  0.3195     0.6544    0.3381  0.4544     0.7439  0.0233
3       3    0.2438  0.3266     0.7463    0.3795  0.4792     0.7919  0.0986
```

```
4        4    0.2391  0.3974     0.6016    0.3663  0.5175     0.7079  0.1005
5        5    0.2288  0.3443     0.6644    0.3609  0.4618     0.7816  0.1641
6        6    0.3078  0.4621     0.6662    0.4004  0.5305     0.7549  0.1452
7        7    0.3570  0.5261     0.6786    0.4275  0.5730     0.7460  0.0456
8        8    0.3456  0.5552     0.6226    0.4260  0.5744     0.7416  0.0277
9        9    0.3343  0.4964     0.6735    0.4301  0.5415     0.7942  0.0010
10      10    0.3675  0.4839     0.7594    0.4396  0.5415     0.8118  0.0784
   Signif_10pct
1           YES
2           YES
3           YES
4            NO
5            NO
6            NO
7           YES
8           YES
9           YES
10          YES
```

A consistent pattern of ratios below one indicates that incorporating external demand improves forecast accuracy relative to the autoregressive benchmark. The Diebold–Mariano p-values provide a formal check of whether these improvements are statistically distinguishable from noise at each horizon.

8.2.3 Export of evaluation results

To ensure full transparency and reproducibility, all forecast evaluation outputs produced in Script 05 are exported as CSV files. These files contain the complete numerical results underlying the tables discussed in the companion paper and can be reused directly for figures, robustness checks, or policy reporting.

Each file corresponds to a specific evaluation dimension:

- **Absolute forecast accuracy** (bias and loss-based metrics)
- **Relative forecast performance** (ARDL vs AR)
- **Unbiasedness tests**
- **Mincer–Zarnowitz calibration tests**
- **Forecast efficiency tests**

```
# --- Absolute forecast accuracy metrics (bias, MSFE, MAFE, RMSE) ---
write.csv(
  absolute_results,
  "output/eval_absolute_accuracy_precovid.csv",
  row.names = FALSE
)

# --- Relative forecast performance (MSFE / MAFE ratios, DM tests) ---
```

```r
write.csv(
  relative_results,
  "output/eval_relative_performance_precovid.csv",
  row.names = FALSE
)

# --- Unbiasedness t-tests ---
write.csv(
  test_results_t_unbiasedness,
  "output/eval_ttest_unbiasedness_precovid.csv",
  row.names = FALSE
)

# --- Mincer-Zarnowitz regression tests ---
write.csv(
  test_results_mz,
  "output/eval_mincer_zarnowitz_precovid.csv",
  row.names = FALSE
)

# --- Forecast efficiency tests (F-tests) ---
write.csv(
  test_results_efficiency,
  "output/eval_efficiency_tests_precovid.csv",
  row.names = FALSE
)
```

All exported files are stored in the output/ directory and correspond exactly to the evaluation procedures described in this section. This guarantees that every reported result in the companion paper can be independently replicated from the provided code and data.

8.3 True out-of-sample forecast (2025–2027)

After completing the pseudo out-of-sample evaluation, we generate a forward-looking forecast for Swiss GDP growth over 2025–2027.

8.3.1 Forecast design

Let $t^\star$ denote the last rolling estimation endpoint, the last available forecast origin in `rolling_specifications`. We treat the model selected at $t^\star$, including its AR/ARDL lag orders and estimated coefficients, as the most up-to-date specification available.

We then produce forecasts for horizons:

$$h = 1, 2, \dots, H_{\text{future}},$$

where in our application:

$$H_{\text{future}} = 12$$

quarters (i.e., 3 years ahead).

Operationally, the dataset is extended by appending H_{future} future quarters with missing values for the dependent variable. The forecasting engine `multi_horizon_recursive_forecasting()` is then called once, using:

- the last-window coefficients,
- recursive forecasting for both x_t (external demand) and y_t (Swiss GDP growth),
- and the same lag structure used in rolling estimation.

Here, $t^\star$ corresponds to the last rolling window endpoint (last available forecast origin).

```
# Last rolling endpoint + corresponding specification/coefficients
last_origin <- max(as.Date(rolling_specifications$Date), na.rm = TRUE)

last_specification <- subset(
  rolling_specifications,
  as.Date(Date) == as.Date(last_origin)
)

last_coefficient <- subset(
  rolling_coefficients,
  as.Date(Date) == as.Date(last_origin)
)

cat("Last rolling origin_date:", format(last_origin), "\n")
```

```
Last rolling origin_date: 2025-07-01
```

```
# Create future quarters block and extend the dataset
H_future <- 12

last_date_in_data <- max(as.Date(winsorized_gdp_data$date), na.rm = TRUE)

future_dates <- seq(
  from = last_date_in_data,
  by   = "quarter",
  length.out = H_future + 1
)[-1]

# Create a future block with exact same columns as winsorized_gdp_data
future_block <- winsorized_gdp_data[1:H_future, ]
future_block[,] <- NA
```

```r
future_block$date <- future_dates

data_extended <- rbind(winsorized_gdp_data, future_block)

# Forecast recursively from last_origin for h = 1..H_future
future_results <- multi_horizon_recursive_forecasting(
  data           = data_extended,
  specifications = last_specification,
  coefficients   = last_coefficient,
  y_col          = y_col,
  x_col          = x_col,
  max_h          = H_future,
  max_lag        = max_lag
)

# Match each horizon to the actual forecast date (origin_date + h)
index_end <- which(as.Date(data_extended$date) == as.Date(last_origin))

# Build final forecast table (calendar dates + horizons)
final_forecast_df <- data.frame(
  Date             = as.Date(data_extended$date[index_end + future_results$horizon]),
  Horizon          = future_results$horizon,
  Forecast_ARDL    = round(future_results$y_hat_ardl, 4),
  Forecast_AR      = round(future_results$y_hat_ar, 4),
  stringsAsFactors = FALSE
)

# Keep only 2025-2027
forecast_focus <- subset(
  final_forecast_df,
  Date >= as.Date("2025-01-01") & Date <= as.Date("2027-12-31")
)

print(forecast_focus)
```

```
        Date Horizon Forecast_ARDL Forecast_AR
1 2025-10-01       1        0.2657      0.2597
2 2026-01-01       2        0.4335      0.5748
3 2026-04-01       3        0.4691      0.7012
4 2026-07-01       4        0.4766      0.7520
5 2026-10-01       5        0.4782      0.7723
6 2027-01-01       6        0.4785      0.7805
7 2027-04-01       7        0.4786      0.7838
8 2027-07-01       8        0.4786      0.7851
9 2027-10-01       9        0.4786      0.7856
```

The object forecast_focus is the main quarterly forecast table used in later steps (annualisation and

plotting). It contains forecast dates, horizons, and point forecasts from both AR and ARDL models, restricted to the policy-relevant horizon 2025–2027.

8.4 Annualization

Policy communication typically relies on annual growth rates, while our forecasting models operate on quarterly log-growth rates. We therefore convert the quarterly forecast path into annual growth numbers in three steps:

1) Combine historical quarterly growth and forecasted quarterly growth into a single series.
2) Reconstruct a quarterly level index by cumulating log-growth rates.

3) Aggregate quarterly levels into annual totals by summing quarters, keeping only complete years, then compute annual growth rates.

8.4.1 From quarterly log-growth to a quarterly level index

Let g_t denote quarterly growth in log-differences (expressed in percent), i.e.

$$g_t = 100(\log Y_t - \log Y_{t-1}).$$

Define quarterly log-levels $\ell_t = \log Y_t$. Then:

$$\ell_t = \ell_{t-1} + \frac{g_t}{100}.$$

Starting from an arbitrary baseline ℓ_0 (we set `base_log = 0`), we reconstruct the level path recursively:

$$\ell_t = \ell_0 + \sum_{s=1}^{t} \frac{g_s}{100}, \qquad Y_t = \exp(\ell_t).$$

8.4.2 Annual aggregation and annual growth

Let $Y_{y,q}$ denote the reconstructed quarterly level within year y and quarter $q \in \{1, 2, 3, 4\}$. Annual GDP is approximated by the sum of quarterly levels:

$$Y_y^{\text{annual}} = \sum_{q=1}^{4} Y_{y,q}.$$

Annual growth is then:

$$G_y = \frac{Y_y^{\text{annual}}}{Y_{y-1}^{\text{annual}}} - 1.$$

In the pipeline this is implemented by the helper function `aggregate_growth_to_annual()`, which (i) reconstructs levels from growth rates, (ii) aggregates to annual totals, and (iii) returns annual growth rates, keeping only years with four quarters available.

```
# Quarterly growth rates (Δlog × 100) are combined for history and forecast,
# converted back into quarterly levels via cumulative growth,
# and aggregated into annual growth rates using complete-year information only.

# Combine historical and forecasted quarterly growth rates
growth_all <- c(
  winsorized_gdp_data[[y_col]],
  final_forecast_df$Forecast_ARDL
)

dates_all <- c(
  as.Date(winsorized_gdp_data$date),
  as.Date(final_forecast_df$Date)
)

# Annualise growth rates using the custom aggregation function
annual_forecast <- aggregate_growth_to_annual(
  dates_q    = dates_all,
  g_q        = growth_all,
  years_keep = c(2025, 2026, 2027),
  base_log   = 0
)

# Format results for reporting
annual_forecast <- annual_forecast |>
  transform(annual_growth_pct = round(annual_growth * 100, 2)) |>
  select(year, annual_growth_pct)

cat("\n--- Annual Swiss GDP Forecast ---\n")
```

```
--- Annual Swiss GDP Forecast ---
```

```
print(annual_forecast)
```

```
  year annual_growth_pct
1 2025              1.38
2 2026              1.13
3 2027              1.93
```

```
write.csv(
  annual_forecast,
  "output/annual_gdp_forecast_2025_2027.csv",
  row.names = FALSE
)
```

The resulting file output/annual_gdp_forecast_2025_2027.csv contains annual Swiss GDP growth forecasts (in percent) for 2025, 2026, and 2027, derived consistently from the quarterly ARDL forecast path.

9 Script 06 — Figures and density forecast

This script produces descriptive plots in levels and growth, data-quality checks (winsorization), the external-demand components (partner growth and weights), stability diagnostics (rolling correlations), forecast stability (spaghetti chart), forecast error diagnostics (distribution), and a density forecast for 2025–2027 (fan chart). All outputs are exported to figures/ and output/.

9.1 Swiss GDP level and growth

We begin with a simple time-series overview of Swiss real GDP in levels and growth rates. Approximative ecession periods are shaded to provide economic context.

```
# Define Swiss recession periods for visualization context
recession_periods <- data.frame(
start = as.Date(c("2008-01-01", "2020-01-01")),
end = as.Date(c("2009-12-31", "2020-12-31"))
)

# Plot 1: GDP Level
plot_gdp_level <- ggplot(gdp_level_data, aes(x = date, y = gdp)) +
geom_rect(
data = recession_periods,
aes(xmin = start, xmax = end, ymin = -Inf, ymax = Inf),
fill = "grey70", alpha = 0.2, inherit.aes = FALSE
) +
geom_line(color = "#1f77b4", linewidth = 0.8) +
labs(
title = "Swiss Real GDP Level",
subtitle = "Original series with recession periods highlighted",
x = NULL,
y = "GDP (CHF)"
) +
theme_minimal() +
theme(
plot.title = element_text(size = 12, face = "bold", hjust = 0),
```

```
plot.subtitle = element_text(size = 10, hjust = 0),
axis.text = element_text(size = 10)
)

# Plot 2: GDP Growth
plot_gdp_growth <- ggplot(gdp_growth_data, aes(x = date, y = swiss_gdp_growth)) +
geom_rect(
data = recession_periods,
aes(xmin = start, xmax = end, ymin = -Inf, ymax = Inf),
fill = "grey70", alpha = 0.2, inherit.aes = FALSE
) +
geom_line(color = "#d62728", linewidth = 0.8) +
geom_hline(yintercept = 0, linetype = "dashed", color = "black", alpha = 0.5) +
labs(
title = "Swiss Real GDP Growth Rate",
subtitle = "Quarter-over-quarter growth with recession periods highlighted",
x = NULL,
y = "QoQ Growth Rate"
) +
theme_minimal() +
theme(
plot.title = element_text(size = 12, face = "bold", hjust = 0),
plot.subtitle = element_text(size = 10, hjust = 0),
axis.text = element_text(size = 10)
)

# Combine plots vertically and display
combined_plot <- plot_gdp_level / plot_gdp_growth
print(combined_plot)

ggsave("figures/swiss_gdp_level_growth_plot.png", combined_plot,
width = 10, height = 8, dpi = 300)
```

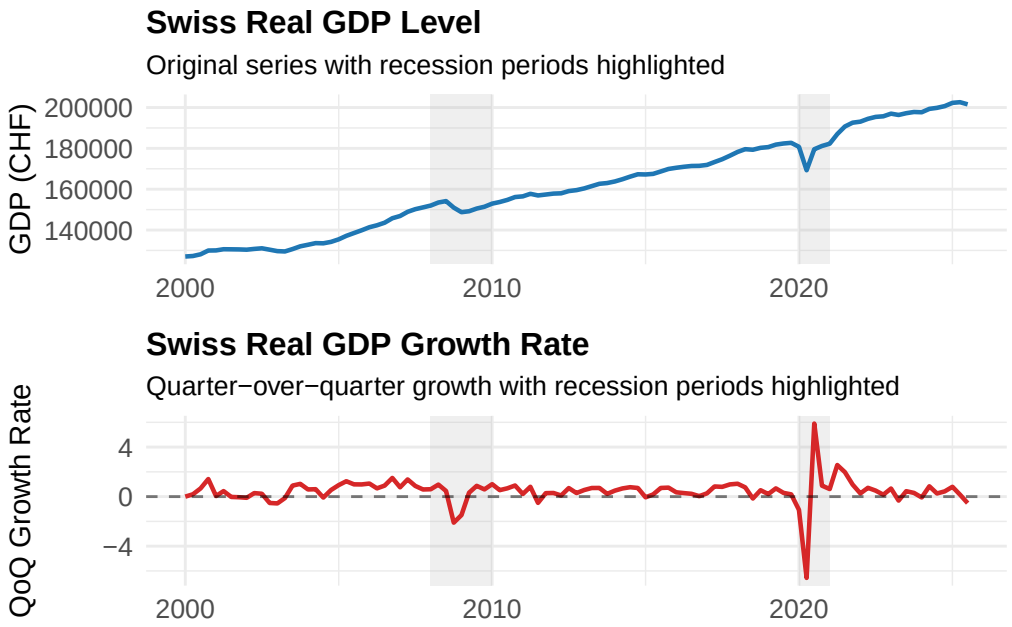


Figure 1: Swiss Real GDP Level and Growth (with recession periods highlighted)

9.2 Swiss GDP growth: Original vs winsorized (COVID focus)

The goal of this figure is to visually document the impact of our targeted COVID winsorization on Swiss GDP quarterly growth. As explained in Script 02, we do not winsorize the full sample. Instead, we cap extreme observations only during the COVID window, because the COVID shock generates outliers that can dominate rolling estimation and distort model selection in small samples.

This plot focuses on 2019–2022 and overlays:

- the original Swiss GDP QoQ growth rate (solid line), and
- the winsorized Swiss GDP QoQ growth rate (dashed line).

The shaded area highlights the COVID disruption period (2020–2021) for visual context.

Interpretation:

- when the dashed and solid lines overlap, winsorization does not alter the data;
- divergences indicate quarters where the original growth rate was outside the 5th–95th percentile bounds and therefore capped.

```
covid_focus_period <- gdp_growth_data %>%
  filter(year >= 2019 & year <= 2022) %>%
  left_join(winsorized_gdp_data %>% select(date, swiss_gdp_growth_winsorized),
            by = "date")

winsorization_comparison_plot <- ggplot(covid_focus_period, aes(x = date)) +
  geom_rect(
```

```
    data = data.frame(start = as.Date("2020-01-01"),
                      end = as.Date("2021-12-31")),
    aes(xmin = start, xmax = end, ymin = -Inf, ymax = Inf),
    fill = "grey70", alpha = 0.2, inherit.aes = FALSE
  ) +
  # First plot winsorized series (dashed line), then original series
  geom_line(aes(y = swiss_gdp_growth_winsorized, color = "Winsorized 5%-95%"),
            linetype = "dashed", linewidth = 0.8) +
  geom_line(aes(y = swiss_gdp_growth, color = "Original"), linewidth = 0.8) +
  geom_hline(yintercept = 0,
             linetype = "dashed", color = "black", alpha = 0.5) +
  scale_color_manual(
    name = "Series",
    values = c("Original" = "#d62728", "Winsorized 5%-95%" = "#d62728"),
    guide = guide_legend(override.aes = list(linetype = c("solid", "dashed"),
                                             linewidth = c(0.8, 0.8)))
  ) +
  labs(
    title = "Swiss GDP Growth Rate: Original vs Winsorized (2019-2022)",
    subtitle = "Dashed line shows winsorized values during COVID period
    (5th-95th percentiles)",
    x = NULL,
    y = "QoQ Growth Rate"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold", hjust = 0),
    plot.subtitle = element_text(size = 10, hjust = 0),
    axis.text = element_text(size = 10),
    legend.position = "bottom"
  )

print(winsorization_comparison_plot)

ggsave("figures/winsorization_comparison_covid_focus.png",
       winsorization_comparison_plot,
       width = 10, height = 6, dpi = 300)
```

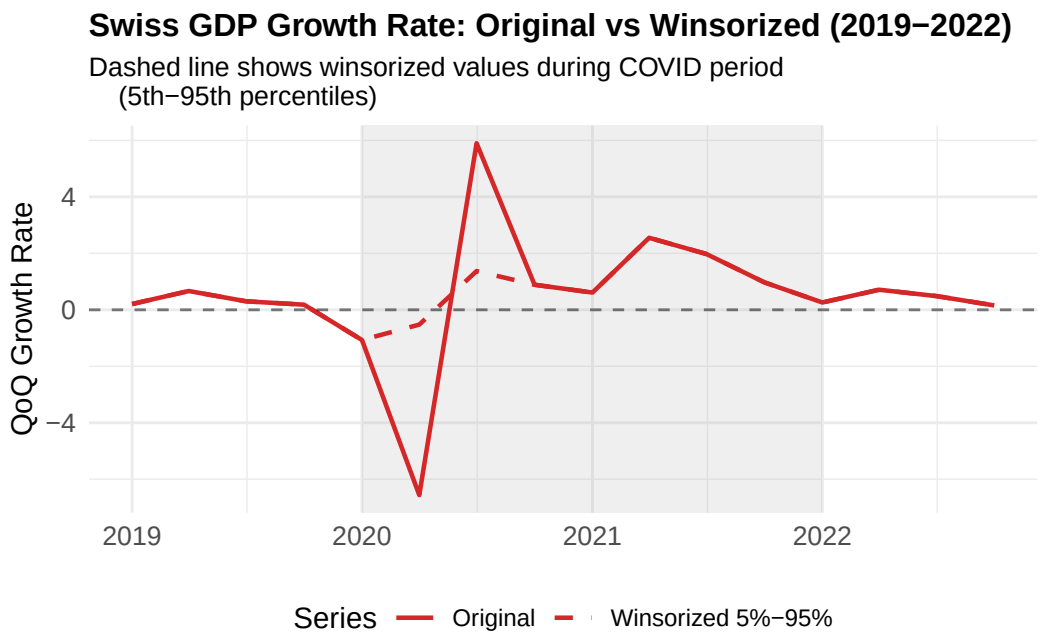


Figure 2: Swiss GDP Growth Rate: Original vs Winsorized (2019–2022). Dashed line shows winsorized values during the COVID period (5th–95th percentiles).

9.3 Individual country GDP growth rates

This section plots quarterly GDP growth rates for Switzerland's main trading partners: the Euro Area, the United States, China, Japan, and the United Kingdom. These series constitute the building blocks of the external demand indicator used in the forecasting model.

The objective of this figure is twofold. First, it provides a descriptive overview of the cyclical behavior of partner economies relative to major global shocks, such as the Global Financial Crisis and the COVID-19 recession. Second, it allows a visual assessment of cross-country heterogeneity in growth dynamics, which motivates the use of time-varying trade weights when aggregating foreign demand.

Recession periods are shaded to facilitate comparison across countries and to highlight periods of synchronized downturns.

```
plot_country_growth <- gdp_growth_data %>%
  select(
    date,
    euro_gdp_growth,
    usa_gdp_growth,
    china_gdp_growth,
    japan_gdp_growth,
    uk_gdp_growth
  ) %>%
  pivot_longer(
```

```
    -date,
    names_to = "country",
    values_to = "growth"
  ) %>%
  mutate(country = case_when(
    country == "euro_gdp_growth"  ~ "Euro Area",
    country == "usa_gdp_growth"   ~ "United States",
    country == "china_gdp_growth" ~ "China",
    country == "japan_gdp_growth" ~ "Japan",
    country == "uk_gdp_growth"    ~ "United Kingdom"
  )) %>%
  ggplot(aes(x = date, y = growth, color = country)) +
  geom_rect(
    data = recession_periods,
    aes(xmin = start, xmax = end, ymin = -Inf, ymax = Inf),
    fill = "grey70",
    alpha = 0.2,
    inherit.aes = FALSE
  ) +
  geom_line(linewidth = 0.7) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed",
    color = "black",
    alpha = 0.5
  ) +
  labs(
    title = "Individual Country GDP Growth Rates",
    subtitle = "Quarterly growth rates of Switzerland's main trading partners",
    x = NULL,
    y = "QoQ Growth Rate",
    color = "Country"
  ) +
  theme_minimal() +
  theme(
    plot.title    = element_text(size = 12, face = "bold", hjust = 0),
    plot.subtitle = element_text(size = 10, hjust = 0),
    axis.text     = element_text(size = 10),
    legend.position = "bottom"
  )

print(plot_country_growth)

ggsave(
  "figures/country_growth_rates_plot.png",
  plot_country_growth,
  width = 10,
```

```
  height = 6,
  dpi = 300
)
```

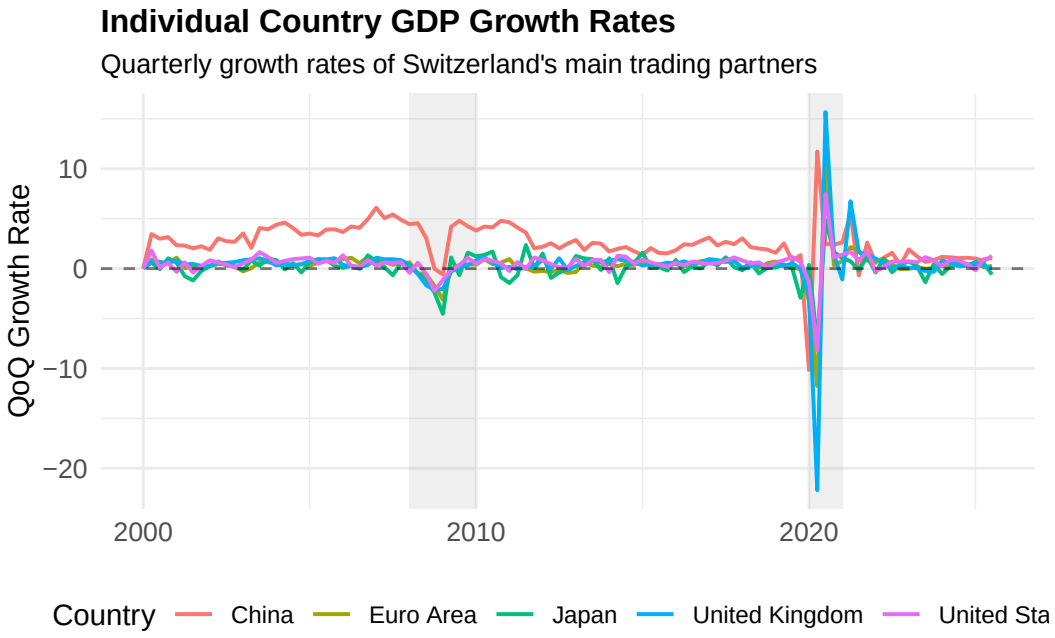


Figure 3: Quarterly GDP growth rates of Switzerland's main trading partners.

9.4 External demand growth

This section plots the constructed *external demand growth* indicator used as the main exogenous driver in the Swiss ARDL model. The variable is defined as a time-varying trade-weighted average of quarterly GDP growth rates across Switzerland's five main trading partners (Euro Area, United States, China, United Kingdom, Japan). Trade weights are computed from quarterly Swiss bilateral trade flows (exports + imports) and therefore reflect the evolving external exposure of the Swiss economy over time.

The figure serves as a validation and communication tool. Visually, it confirms that the external growth index exhibits strong comovement during major global downturns (e.g., 2008–2009 and 2020), while still displaying distinct dynamics compared to Swiss GDP growth. Since this series is later winsorized and fed into rolling ARDL models, plotting it at this stage provides a transparent diagnostic of its scale, volatility, and business-cycle alignment.

As in previous figures, recession periods are shaded to provide macroeconomic context.

```
plot_external_growth <- ggplot(gdp_growth_data, aes(x = date, y = external_growth)) +
  geom_rect(
    data = recession_periods,
    aes(xmin = start, xmax = end, ymin = -Inf, ymax = Inf),
```

```
    fill = "grey70", alpha = 0.2, inherit.aes = FALSE
  ) +
  geom_line(color = "#2ca02c", linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black", alpha = 0.5) +
  labs(
    title = "Swiss External Demand Growth Rate",
    subtitle = "Time-varying weighted average of trading partners' GDP growth\nWeights based on
    x = NULL,
    y = "Quarterly Growth Rate"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold", hjust = 0),
    plot.subtitle = element_text(size = 10, hjust = 0),
    axis.text = element_text(size = 10)
  )

print(plot_external_growth)

ggsave("figures/external_growth_plot.png", plot_external_growth,
       width = 10, height = 6, dpi = 300)
```

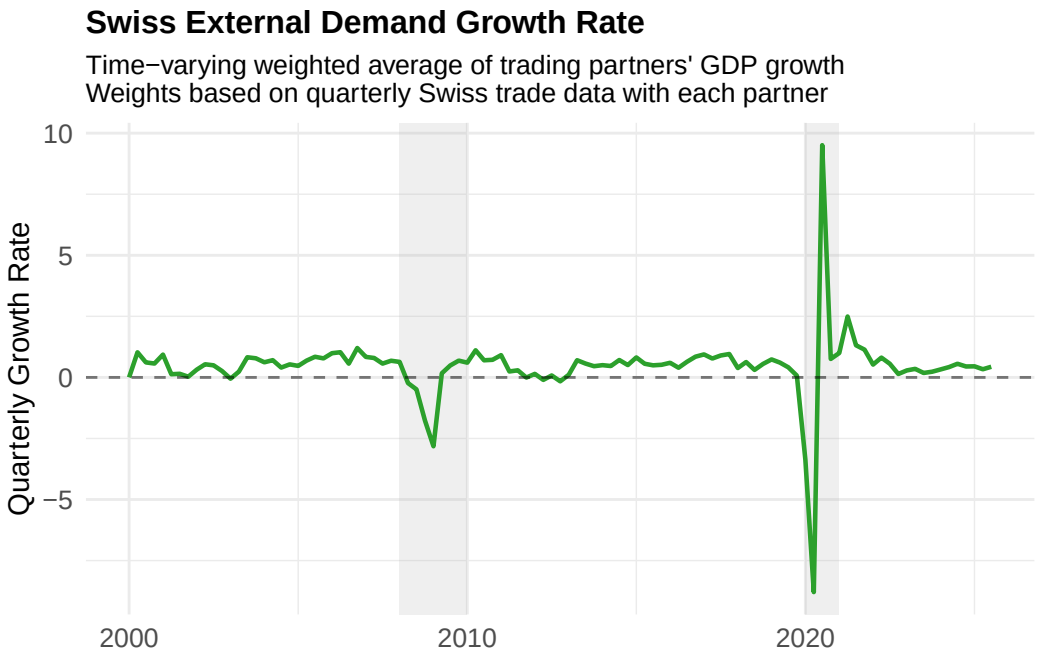


Figure 4: Swiss external demand growth: time-varying trade-weighted average of partner GDP growth.

9.5 Evolution of Swiss trading partners' weights

This section visualizes the quarterly *trade weights* used to build the external demand index. For each quarter, we compute total bilateral trade with each partner as exports + imports, then convert these totals into shares that sum to one across the five partner groups (Europe, US, China, UK, Japan). Because these weights vary over time, they capture changes in Switzerland's external exposure (e.g., rising Chinese importance, shifting composition across regions).

Plotting the weights is important for two reasons:

1. Economic interpretation: it shows which partners dominate the external-demand proxy at different points in time.
2. Data validation: it helps verify that weights evolve smoothly and do not contain suspicious jumps, especially after Kalman imputation was applied to fill any missing quarterly observations.

We use a stacked area chart to display the composition of weights over time.

```
# Create date column and reshape in one step
weights_plot_data <- trade_weights_q %>%
  mutate(date = as.Date(paste(year, quarter * 3 - 2, "01", sep = "-"))) %>%
  pivot_longer(
    cols = c(weight_Europe, weight_US, weight_China, weight_UK, weight_Japan),
    names_to = "region",
    values_to = "weight",
    names_prefix = "weight_"
  ) %>%
  mutate(region = factor(region, levels = c("Europe", "US", "China", "UK", "Japan")))

# Create area plot showing composition over time
plot_weights_area <- ggplot(weights_plot_data, aes(x = date, y = weight, fill = region)) +
  geom_area(alpha = 0.8, color = "white", linewidth = 0.2) +
  scale_fill_manual(
    values = c(
      "Europe" = "#1f77b4",
      "US"     = "#ff7f0e",
      "China"  = "#d62728",
      "UK"     = "#9467bd",
      "Japan"  = "#2ca02c"
    )
  ) +
  labs(
    title = "Evolution of Swiss Trading Partners' Weights",
    subtitle = "Quarterly trade weights used for external demand calculation",
    x = NULL,
    y = "Weight (proportion)",
    fill = "Region"
  ) +
  theme_minimal() +
```

```
  theme(
    plot.title = element_text(size = 12, face = "bold", hjust = 0),
    plot.subtitle = element_text(size = 10, hjust = 0),
    axis.text = element_text(size = 10),
    legend.position = "bottom"
  )

print(plot_weights_area)

ggsave("figures/trade_weights_area.png", plot_weights_area,
       width = 10, height = 6, dpi = 300)
```

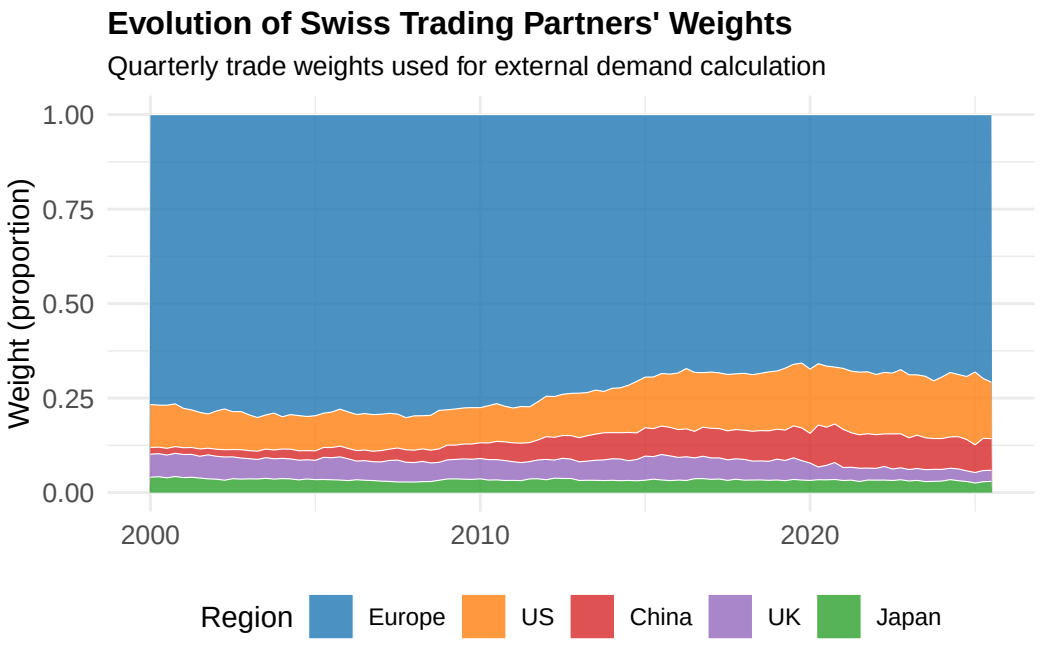


Figure 5: Quarterly evolution of trade weights used in the external demand index.

9.6 Stability of the Swiss–external GDP relationship (rolling correlations)

A core motivation for using a rolling-window forecasting design is that the relationship between Swiss GDP growth and external demand may not be stable over time. Even if each series is approximately stationary in growth rates, the link between them can strengthen or weaken across business-cycle regimes, globalization phases, or crisis episodes.

To visualize this stability, we compute rolling correlations between:

- Swiss GDP growth (`swiss_gdp_growth`)
- external demand growth (`external_growth`)

using three window lengths:

- **8 quarters (2 years)**: short-run comovement
- **24 quarters (6 years)**: medium-run comovement
- **40 quarters (10 years)**: long-run comovement (aligned with our rolling estimation window)

We then (i) identify the first date where all three correlations are available (so the plot has no missing segments), (ii) restrict to a comparison period up to 2015-12-31 (as in your script), and (iii) plot the three rolling correlation series.

```
# 1. Calculate correlations
gdp_growth_data <- gdp_growth_data %>%
  mutate(
    rolling_cor_8  = zoo::rollapply(data = cbind(swiss_gdp_growth,
                                                 external_growth),
                                    width = 8,
                                    FUN = function(x) cor(x[,1], x[,2]),
                                    by.column = FALSE,
                                    fill = NA, align = "right"),
    rolling_cor_24 = zoo::rollapply(data = cbind(swiss_gdp_growth,
                                                 external_growth),
                                    width = 24,
                                    FUN = function(x) cor(x[,1], x[,2]),
                                    by.column = FALSE, fill = NA,
                                    align = "right"),
    rolling_cor_40 = zoo::rollapply(data = cbind(swiss_gdp_growth,
                                                 external_growth),
                                    width = 40,
                                    FUN = function(x) cor(x[,1], x[,2]),
                                    by.column = FALSE,
                                    fill = NA, align = "right")
  )

# 2. Find the first date where all three correlations have non-NA values
first_complete_date <- gdp_growth_data %>%
  filter(!is.na(rolling_cor_8) & !is.na(rolling_cor_24)
         & !is.na(rolling_cor_40)) %>%
  summarise(first_date = min(date)) %>%
  pull(first_date)

cat(paste("First date with all three correlations available:",
          format(first_complete_date, "%Y-%m-%d"), "\n"))
```

```
First date with all three correlations available: 2009-10-01
```

```
# 3. Filter for period when all three lines are present and reshape for ggplot
plot_data_corr <- gdp_growth_data %>%
  filter(date >= first_complete_date & date <= as.Date("2015-12-31")) %>%
  select(date, rolling_cor_8, rolling_cor_24, rolling_cor_40) %>%
```

```
  pivot_longer(cols = starts_with("rolling_cor"),
               names_to = "window",
               values_to = "correlation") %>%
  mutate(window_label = case_when(
    window == "rolling_cor_8"  ~ "Short-term (8Q)",
    window == "rolling_cor_24" ~ "Medium-term (24Q)",
    window == "rolling_cor_40" ~ "Long-term (40Q)"
  ))

# 4. Create ggplot visualization
plot_rolling_corr <- ggplot(plot_data_corr, aes(x = date, y = correlation,
                                                color = window_label)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c(
    "Short-term (8Q)"    = "#d62728",  # Red
    "Medium-term (24Q)"  = "#1f77b4",  # Blue
    "Long-term (40Q)"    = "#2ca02c"   # Green
  )) +
  labs(
    title = "Stability of Swiss-External GDP Relationship",
    subtitle = paste0(
      "Rolling correlation coefficients across different time horizons\n",
      "Comparison period: ", format(first_complete_date, "%Y-%m"), " to 2015-12"
    ),
    x = NULL,
    y = "Correlation Coefficient",
    color = "Window Size"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    plot.subtitle = element_text(size = 10),
    legend.position = "bottom"
  )

print(plot_rolling_corr)

ggsave("figures/rolling_correlation_stability.png", plot_rolling_corr,
       width = 10, height = 6, dpi = 300)
```

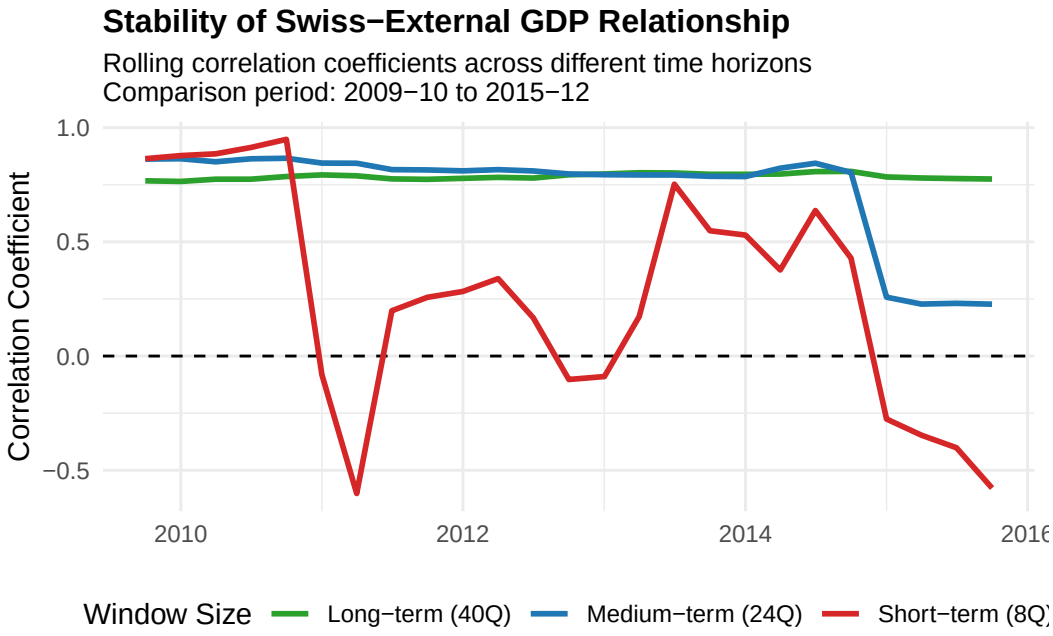


Figure 6: Rolling correlations between Swiss GDP growth and external demand growth (8Q, 24Q, 40Q).

9.7 Comprehensive comparison plot (Swiss GDP growth vs. external demand growth)

Before moving to model estimation and forecasting diagnostics, it is useful to visually compare the two core series that drive the project:

- Swiss GDP growth
- External demand growth

This "comprehensive" figure simply stacks the two time series vertically, using the exact same recession shading and visual style as the individual charts. The goal is to make comovements (and potential divergences) immediately visible, and to provide an intuitive benchmark for what the ARDL model is trying to capture.

```
comprehensive_plot <- (plot_gdp_growth + labs(title = "A) Swiss GDP Growth")) /
  (plot_external_growth + labs(title = "B) External Demand Growth"))

print(comprehensive_plot)

ggsave("figures/comprehensive_comparison_plot.png", comprehensive_plot,
       width = 12, height = 10, dpi = 300)
```

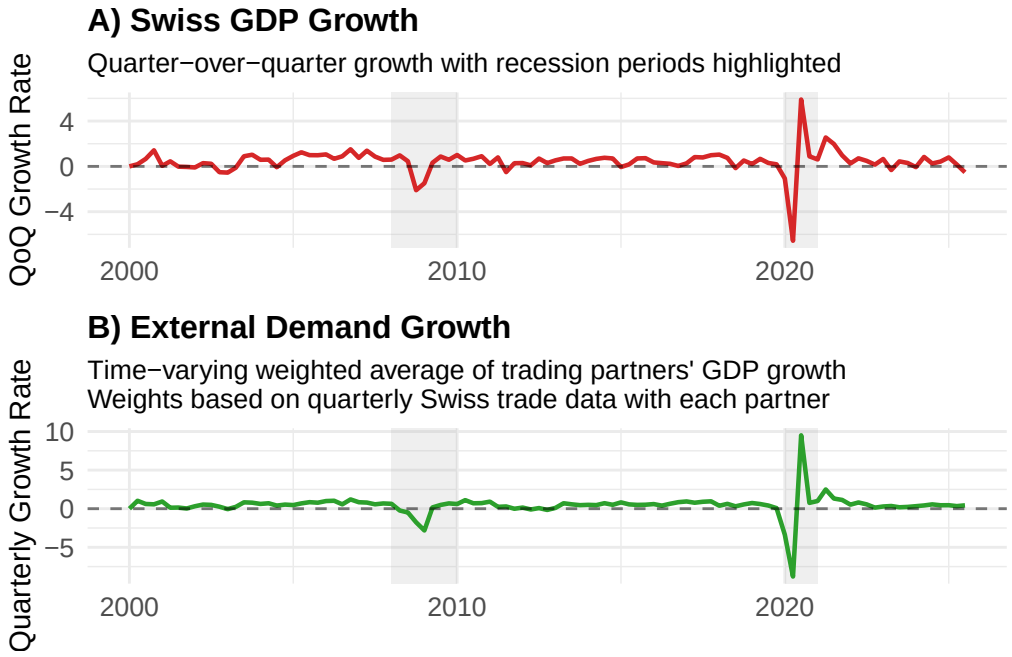


Figure 7: Comprehensive comparison: Swiss GDP growth (top) vs external demand growth (bottom).

9.8 Diagnostic spaghetti chart

A key concern in rolling-window forecasting is whether the model produces stable forecast paths over time, or whether small changes in the estimation window lead to wildly different projections. To visualize this, we generate a spaghetti chart:

- Each **blue line** is an 8-quarter-ahead ARDL forecast path produced from a specific rolling origin date.
- The **black line** is the realized (winsorized) Swiss GDP growth series.
- If forecast paths are broadly clustered and move smoothly, this suggests the rolling ARDL procedure is not overly sensitive to small sample changes.
- If forecast paths are extremely dispersed or unstable, this may signal model fragility, structural instability, or weak signal in the external demand driver.

This diagnostic relies on the helper function `generate_rolling_paths()`, which reconstructs rolling forecast trajectories using the saved rolling specifications and coefficients from Script 04.

```
rolling_paths <- generate_rolling_paths(
  data           = winsorized_gdp_data,
  specifications = rolling_specifications,
  coefficients   = rolling_coefficients,
  y_col          = y_col,
  x_col          = x_col,
  H              = 8,
  max_lag        = max_lag
)
```

```r
plot_spaghetti <- ggplot() +
  geom_line(
    data = rolling_paths,
    aes(x = plot_date, y = forecast_value, group = origin_date),
    color = "steelblue",
    alpha = 0.4,
    linewidth = 0.6
  ) +
  geom_line(
    data = winsorized_gdp_data %>% filter(date >= as.Date("2010-01-01")),
    aes(x = date, y = .data[[y_col]]),
    color = "black",
    linewidth = 1.2
  ) +
  labs(
    title = "Forecast Stability Analysis (Spaghetti Chart)",
    subtitle = "Blue Lines: 8-quarter rolling ARDL forecasts | Black Line: Realized GDP",
    caption = "Source: Rolling Window Estimation (W=40)",
    x = NULL,
    y = "GDP Growth (QoQ %)"
  ) +
  coord_cartesian(
    ylim = c(-3, 3),
    xlim = as.Date(c("2010-01-01", "2024-12-31"))
  ) +
  theme_minimal() +
  theme(
    plot.title    = element_text(face = "bold", size = 14),
    plot.subtitle = element_text(size = 10, color = "gray30")
  )

print(plot_spaghetti)

if (!dir.exists("figures")) dir.create("figures", recursive = TRUE)

ggsave(
  filename = "figures/diagnostic_spaghetti_chart.png",
  plot = plot_spaghetti,
  width = 10,
  height = 6,
  dpi = 300
)
```

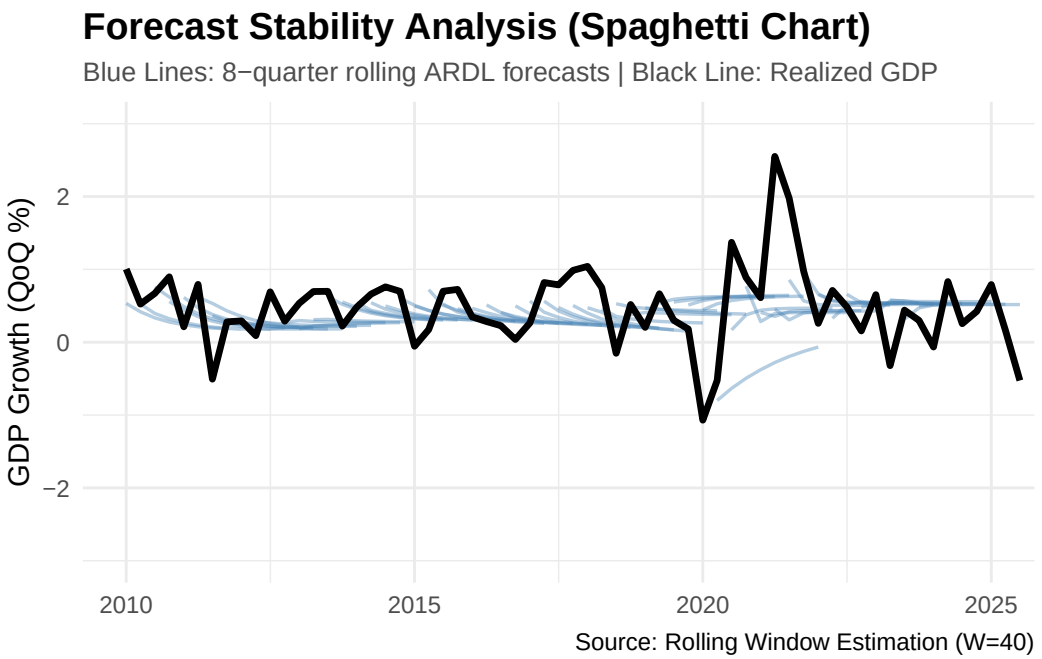


Figure 8: Spaghetti chart diagnostic: rolling 8-quarter ARDL forecast paths (blue) vs realized Swiss GDP growth (black).

9.9 Forecast error distribution

Beyond average accuracy metrics (MSFE/MAFE), it is useful to inspect the distribution of forecast errors to understand whether the model exhibits systematic bias and whether errors are approximately symmetric around zero.

Here we focus on two representative horizons:

- h = 1 (one-quarter-ahead): short-term forecast performance.
- h = 4 (one-year-ahead): medium-run performance, where forecast uncertainty and overlap effects are more visible.

For each horizon, we plot: - a histogram (scaled to density), - a kernel density estimate, - a vertical dashed red line for the mean forecast error (bias), - a vertical black line at zero.

A mean error close to zero suggests little systematic bias, while a clear shift away from zero indicates persistent under- or over-forecasting.

```
# Filter data for specific horizons
error_data <- multi_horizon_results %>%
  filter(horizon %in% c(1, 4)) %>%
  mutate(horizon_label = paste0("Horizon h=", horizon,
                                " (", horizon/4, " Year)"))

# Compute means for vertical lines
error_means <- error_data %>%
```

```r
  group_by(horizon_label) %>%
  summarise(mean_error = mean(e_ardl, na.rm = TRUE))

plot_errors <- ggplot(error_data, aes(x = e_ardl)) +
  geom_histogram(aes(y = ..density..), bins = 20, fill = "lightblue",
                 color = "white", alpha = 0.7) +
  geom_density(color = "#1f77b4", linewidth = 1) +
  geom_vline(data = error_means, aes(xintercept = mean_error),
             color = "red", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = 0, color = "black", linewidth = 0.5) +
  facet_wrap(~horizon_label, scales = "fixed") +
  labs(
    title = "Forecast Error Distribution (Bias Analysis)",
    subtitle = "Dashed red line indicates Mean Bias. Black line is Zero.",
    x = "Forecast Error (pp)",
    y = "Density"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold"),
        strip.text = element_text(size = 12))

print(plot_errors)
ggsave("figures/diagnostic_error_distribution.png",
       plot_errors, width = 10, height = 6)
```

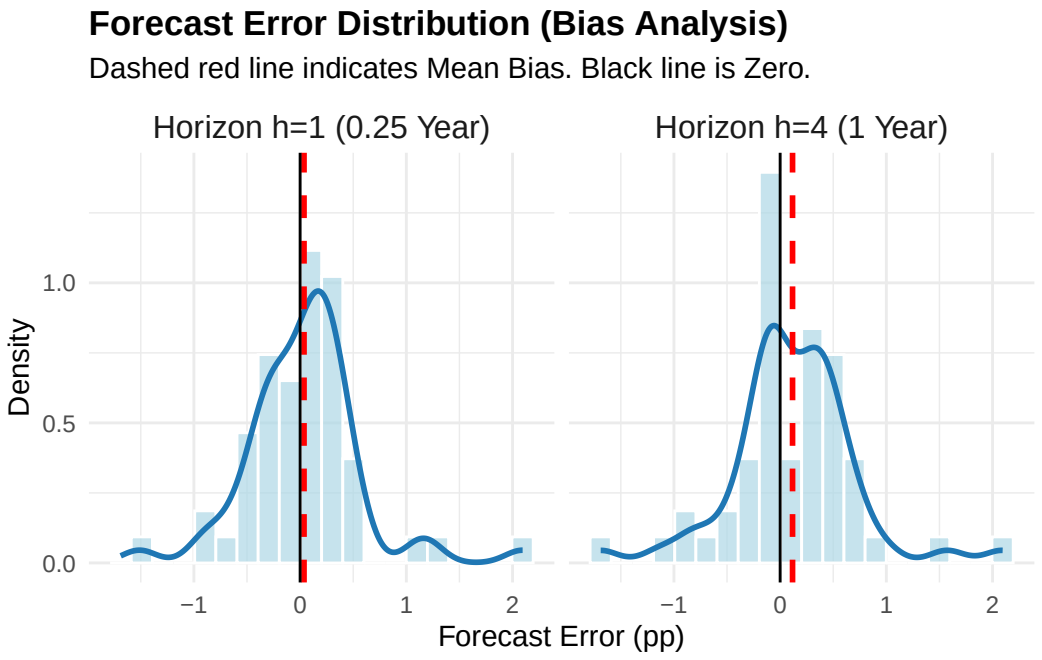


Figure 9: Forecast error distributions for ARDL at horizons h=1 and h=4. Red dashed line is mean bias; black line is zero.

9.10 Final visualization: quarterly density forecast (ARDL fan chart)

Point forecasts are informative, but policy users often need a sense of uncertainty around the baseline. We therefore produce a density forecast for Swiss GDP growth using a Monte Carlo simulation approach and visualize it as a **fan chart** (mean path + 90% interval).

9.10.1 Method overview (Monte Carlo density forecast)

We generate a predictive distribution for y_{t+h} (Swiss GDP growth) in four steps:

1. Last training window: take the last rolling estimation window (size $W = 40$ quarters) as the conditioning sample.
2. Simulate external demand paths: fit an ARMA model to external growth x_t and simulate future trajectories $x_{t+1}, \dots, x_{t+H}$.
3. Model selection: select the best ARDL(p, q) specification by BIC on the last training window.
4. Recursive simulation: simulate Swiss GDP growth recursively under the ARDL dynamics with Gaussian shocks calibrated to the residual variance.

From the simulated paths we extract: - the mean forecast at each horizon, - the 5th and 95th percentiles, forming a 90% predictive interval.

```
# We produce a density forecast for Swiss GDP growth using Monte Carlo simulation:
# 1) Fit an ARMA to external growth (x) and simulate future x paths
# 2) Select best ARDL(p,q) by BIC on the last training window
# 3) Simulate future Swiss GDP growth recursively (y) with Gaussian shocks
# 4) Plot mean forecast + 90% interval (5% / 95%)

# ------------------------------------------------------------------------
# A) Last training window
# ------------------------------------------------------------------------
n_total   <- nrow(winsorized_gdp_data)
train_idx <- (n_total - window_size + 1):n_total

y_train_last <- winsorized_gdp_data[[y_col]][train_idx]
x_train_last <- winsorized_gdp_data[[x_col]][train_idx]

# ------------------------------------------------------------------------
# B) Monte Carlo simulation (density forecast)
# ------------------------------------------------------------------------
sim_future <- get_forecast_simulation_dynamic(
  y_train = y_train_last,
  x_train = x_train_last,
  H       = 12,
  n_sims  = 5000,
  max_lag = max_lag
)
```

```r
# ------------------------------------------------------------------------
# C) Build forecast table (mean + 90% band)
# ------------------------------------------------------------------------
last_hist_date <- tail(as.Date(winsorized_gdp_data$date), 1)
last_hist_val  <- tail(winsorized_gdp_data[[y_col]], 1)

future_dates <- seq(from = last_hist_date, by = "quarter", length.out = 13)[-1]

final_forecast_density <- data.frame(
  Date     = future_dates,
  Horizon  = 1:12,
  Mean     = as.numeric(sim_future$mean),
  Lower_90 = as.numeric(sim_future$lower),
  Upper_90 = as.numeric(sim_future$upper),
  stringsAsFactors = FALSE
)

# Anchor row for continuity
plot_fan_df <- final_forecast_density |>
  dplyr::add_row(
    Date     = last_hist_date,
    Horizon  = 0,
    Mean     = last_hist_val,
    Lower_90 = last_hist_val,
    Upper_90 = last_hist_val,
    .before = 1
  ) |>
  dplyr::arrange(Date)

# Export (table)
if (!dir.exists("output")) dir.create("output", recursive = TRUE)
write.csv(
  final_forecast_density,
  "output/final_forecast_density_2025_2027_ardl.csv",
  row.names = FALSE
)

# ------------------------------------------------------------------------
# D) Fan chart plot (pretty version)
# ------------------------------------------------------------------------
if (!requireNamespace("scales", quietly = TRUE)) install.packages("scales")

hist_plot_df <- winsorized_gdp_data |>
  dplyr::filter(as.Date(date) >= as.Date("2020-01-01"))

RED_MAIN  <- "#d62728"
RED_LIGHT <- "#d62728"
```

```
plot_fan_chart <- ggplot2::ggplot() +
  # Historical series
  ggplot2::geom_line(
    data = hist_plot_df,
    ggplot2::aes(x = as.Date(date), y = .data[[y_col]]),
    linewidth = 1.1,
    color = "black"
  ) +
  # 90% interval ribbon
  ggplot2::geom_ribbon(
    data = plot_fan_df,
    ggplot2::aes(x = Date, ymin = Lower_90, ymax = Upper_90),
    fill  = RED_LIGHT,
    alpha = 0.12
  ) +
  # Mean forecast
  ggplot2::geom_line(
    data = plot_fan_df,
    ggplot2::aes(x = Date, y = Mean),
    linewidth = 1.3,
    color = RED_MAIN
  ) +
  ggplot2::labs(
    title    = "Swiss GDP Growth Density Forecast (2025-2027)",
    subtitle = "ARDL model - Monte Carlo simulation (mean + 90% interval)",
    x = NULL,
    y = "QoQ Growth ($\\Delta \\log \\times 100$)"
  ) +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::theme(
    plot.title    = ggplot2::element_text(face = "bold", size = 14),
    plot.subtitle = ggplot2::element_text(size = 10, color = "gray35"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(plot_fan_chart)

ggplot2::ggsave(
  filename = "figures/final_forecast_fan_chart_ardl_density.png",
  plot     = plot_fan_chart,
  width    = 10,
  height   = 6,
  dpi      = 300
)
```

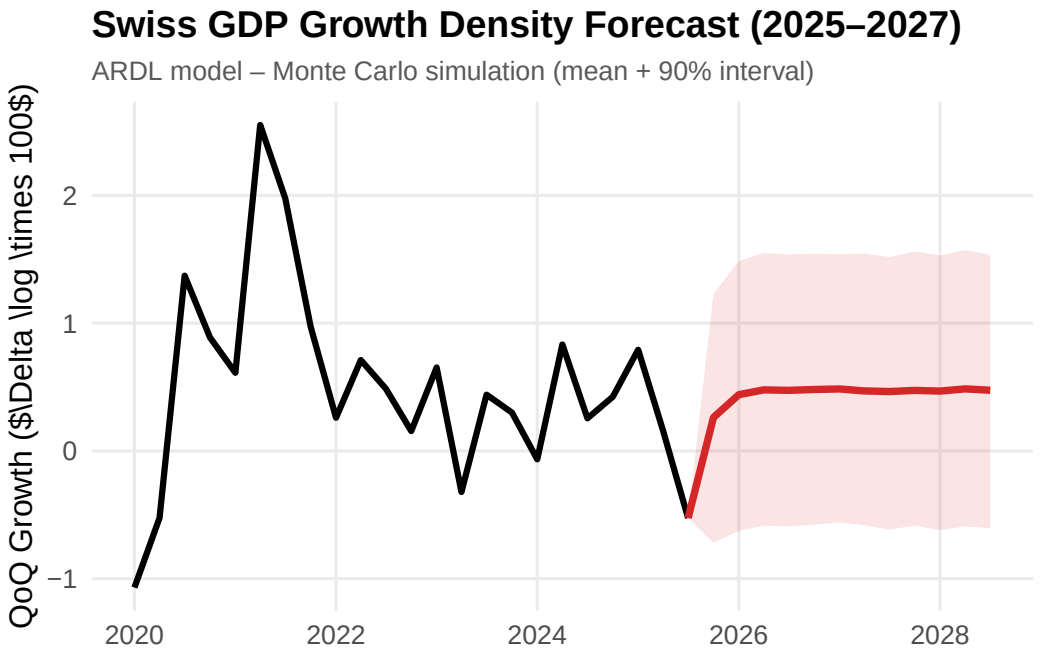


Figure 10: Swiss GDP growth density forecast (ARDL fan chart): mean forecast and 90% predictive interval from Monte Carlo simulation.

9.11 Annual growth comparison: Switzerland vs. global demand

While the forecasting model is estimated at the quarterly frequency, policy discussions are typically framed in terms of annual growth rates. This section therefore aggregates quarterly growth rates into annual growth for both Swiss GDP and global demand, combining realized data with model-based forecasts, and presents the results in a single comparative chart.

The section is organized into two parts:

- Data preparation: construction of annual growth series from quarterly data.
- Visualization: side-by-side comparison of annual growth rates.

9.11.1 Data preparation

Quarterly growth rates are expressed as $\Delta \log \times 100$. To obtain annual growth rates, we:

1. Convert quarterly growth rates into a quarterly index via cumulative growth.
2. Aggregate the index into annual growth rates using complete calendar years only.
3. Apply the same transformation to both Swiss GDP growth (y_t) and global demand growth (x_t), ensuring comparability.

Swiss GDP forecasts are taken from the ARDL point forecasts, while global demand forecasts are obtained from an ARIMA model estimated on historical external growth.

```r
# ----------------------------
# A) SWISS GDP: History + Forecast (ARDL point forecast)
# ----------------------------
swiss_full <- dplyr::bind_rows(
  winsorized_gdp_data |>
    dplyr::select(date, q_growth = .data[[y_col]]) |>
    dplyr::mutate(type = "History"),
  final_forecast_df |>
    dplyr::select(date = Date, q_growth = Forecast_ARDL) |>
    dplyr::mutate(type = "Forecast")
) |>
  dplyr::arrange(date) |>
  dplyr::distinct(date, .keep_all = TRUE)

swiss_annual <- aggregate_growth_to_annual(
  dates_q  = swiss_full$date,
  g_q      = swiss_full$q_growth,
  base_log = 0
) |>
  dplyr::mutate(Variable = "Swiss GDP")

# ----------------------------
# B) EXTERNAL DEMAND: History + Forecast (ARIMA)
# ----------------------------
ext_model <- forecast::auto.arima(
  winsorized_gdp_data[[x_col]],
  seasonal   = FALSE,
  stationary = TRUE,
  ic         = "bic"
)

ext_fc <- forecast::forecast(ext_model, h = 12)

ext_full <- dplyr::bind_rows(
  winsorized_gdp_data |>
    dplyr::select(date, q_growth = .data[[x_col]]) |>
    dplyr::mutate(type = "History"),
  data.frame(
    date     = swiss_full$date[swiss_full$type == "Forecast"],
    q_growth = as.numeric(ext_fc$mean),
    type     = "Forecast"
  )
) |>
  dplyr::arrange(date) |>
  dplyr::distinct(date, .keep_all = TRUE)

ext_annual <- aggregate_growth_to_annual(
```

```
  dates_q  = ext_full$date,
  g_q      = ext_full$q_growth,
  base_log = 0
) |>
  dplyr::mutate(Variable = "Global Demand")

# ----------------------------
# C) COMBINE + LABELS
# ----------------------------
combined_annual <- dplyr::bind_rows(swiss_annual, ext_annual) |>
  dplyr::filter(year >= 2020) |>
  dplyr::mutate(
    Status    = ifelse(year < 2025, "Realized", "Forecast"),
    Variable  = factor(Variable, levels = c("Global Demand", "Swiss GDP")),
    label_pct = sprintf("%.1f%%", annual_growth * 100)
  )
```

9.11.2 Annual growth comparison chart

The figure below presents annual growth rates for Swiss GDP and global demand side by side. Realized values are shown with full opacity, while forecasted values are displayed with reduced opacity. A vertical dashed line separates historical observations from forecasts.

```
# Colors (match names exactly to Variable levels)
clean_colors <- c("Swiss GDP" = "red", "Global Demand" = "gray60")

# Automatic cutoff (between last realized year and first forecast year)
years_sorted <- sort(unique(combined_annual$year))
cut_year     <- 2024
cut_pos      <- which(years_sorted == cut_year) + 0.5

p_combined <- ggplot2::ggplot(
  combined_annual,
  ggplot2::aes(x = factor(year), y = annual_growth * 100)
) +
  ggplot2::geom_col(
    ggplot2::aes(fill = Variable, alpha = Status),
    position = ggplot2::position_dodge(width = 0.8),
    width = 0.7
  ) +
  ggplot2::geom_vline(
    xintercept = cut_pos,
    linetype = "dashed",
    color = "gray30",
    linewidth = 0.8
  ) +
```

```
  ggplot2::geom_text(
    ggplot2::aes(label = label_pct, group = Variable),
    position = ggplot2::position_dodge(width = 0.8),
    vjust = -0.4,
    size = 2.8,
    fontface = "bold"
  ) +
  ggplot2::scale_fill_manual(values = clean_colors) +
  ggplot2::scale_alpha_manual(values = c("Realized" = 1.0, "Forecast" = 0.45), guide = "none") +
  ggplot2::scale_y_continuous(
    limits = c(-3, 6),
    breaks = seq(-3, 6, by = 1),
    expand = ggplot2::expansion(mult = c(0, 0.06)),
    labels = function(x) paste0(x, "%")
  ) +
  ggplot2::labs(
    title    = "Annual Growth: Switzerland vs. Global Demand",
    subtitle = "Swiss GDP annual growth (red); external aggregated demand (grey).",
    x = NULL,
    y = "Annual Growth (%)",
    fill = NULL
  ) +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::theme(
    legend.position = "bottom",
    plot.title = ggplot2::element_text(face = "bold", size = 14),
    plot.subtitle = ggplot2::element_text(size = 10, color = "gray35"),
    axis.text = ggplot2::element_text(size = 11),
    panel.grid.major.x = ggplot2::element_blank()
  )

print(p_combined)

# Save
ggplot2::ggsave(
  filename = "output/graph_combined_annual_growth.png",
  plot = p_combined,
  width = 10,
  height = 6,
  dpi = 300
)
```

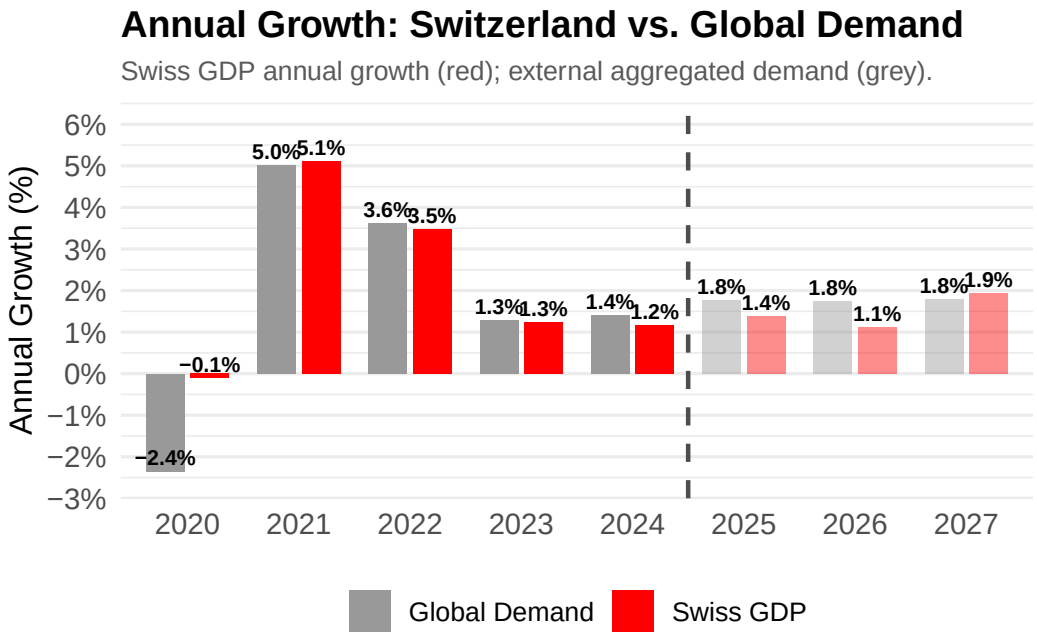


Figure 11: Annual growth comparison: Swiss GDP (red) vs. global demand (grey). Forecast years are shown with reduced opacity.

9.12 Publication-style forecast table

To complement the annual bar chart, we also export a publication-style table summarizing annual growth rates for Switzerland and global demand. The goal is to produce a clean output that can be pasted directly into a report or slide deck.

The table is constructed manually (tiles + text) in order to control:

- a dark header row,
- alternating row shading,
- italic formatting for forecast values,
- consistent spacing and a short caption note.

9.12.1 Prepare data for the table grid

We first transform `combined_annual` into a "grid" dataset where each cell has:

- an (x, y) position in the table,
- a label (formatted percent),
- a font style (plain vs italic),
- a background type (header / year column / data cell).

```r
# A) Base labels + style flags
table_base <- combined_annual |>
  dplyr::mutate(
    label      = sprintf("%.1f%%", annual_growth * 100),
    font_face  = ifelse(Status == "Forecast", "italic", "plain"),
    text_color = "black"
  ) |>
  dplyr::select(year, Variable, label, font_face, text_color)

# B) Data cells (columns: 2 = Swiss GDP, 3 = Global Demand)
cells_data <- table_base |>
  dplyr::mutate(
    x_pos = dplyr::case_when(
      Variable == "Swiss GDP"      ~ 2,
      Variable == "Global Demand"  ~ 3,
      TRUE                         ~ NA_real_
    ),
    y_pos     = - (year - min(year) + 1),  # top data row starts at -1
    cell_type = "data"
  ) |>
  dplyr::filter(!is.na(x_pos)) |>
  dplyr::select(x_pos, y_pos, label, font_face, text_color, cell_type)

# C) Year column (column 1)
years_vec <- sort(unique(table_base$year))

cells_year <- data.frame(
  x_pos = 1,
  y_pos = - (years_vec - min(years_vec) + 1),
  label = as.character(years_vec),
  font_face = "bold",
  text_color = "black",
  cell_type = "year_col",
  stringsAsFactors = FALSE
)

# D) Header row (y = 0)
cells_header <- data.frame(
  x_pos = c(1, 2, 3),
  y_pos = 0,
  label = c("Year", "Swiss GDP", "Global Demand"),
  font_face = "bold",
  text_color = "white",
  cell_type = "header",
  stringsAsFactors = FALSE
)
```

```
# Combine all cells
final_table_plot_data <- dplyr::bind_rows(cells_header, cells_year, cells_data)
```

9.12.2 Draw and export the table

We then draw the table using geom_tile() for backgrounds and geom_text() for labels. Forecast values are italicized automatically via the font_face column. Finally, we export the figure as both PNG and PDF.

```
# Compute y-limits dynamically (no hard-coded -8.5)
y_min <- min(final_table_plot_data$y_pos) - 0.6
y_max <- 1

p_table_fancy <- ggplot2::ggplot(final_table_plot_data,
                                 ggplot2::aes(x = x_pos, y = y_pos)) +

  # A) Background tiles: header dark, data rows alternating
  ggplot2::geom_tile(
    ggplot2::aes(fill = interaction(cell_type, factor(y_pos %% 2 == 0))),
    width = 0.95, height = 0.9
  ) +
  ggplot2::scale_fill_manual(
    values = c(
      "header.TRUE"    = "#2C3E50",
      "header.FALSE"   = "#2C3E50",
      "data.TRUE"      = "gray95",
      "data.FALSE"     = "white",
      "year_col.TRUE"  = "gray95",
      "year_col.FALSE" = "white"
    ),
    guide = "none"
  ) +

  # B) Text
  ggplot2::geom_text(
    ggplot2::aes(label = label, fontface = font_face, color = text_color),
    size = 4.5
  ) +
  ggplot2::scale_color_identity() +

  # C) Separator line under header
  ggplot2::geom_segment(
    ggplot2::aes(x = 0.55, xend = 3.45, y = -0.45, yend = -0.45),
    linewidth = 1,
    color = "#2C3E50"
  ) +
```

```
  # D) Scales
  ggplot2::scale_x_continuous(limits = c(0.5, 3.5)) +
  ggplot2::scale_y_continuous(limits = c(y_min, y_max)) +

  # E) Titles
  ggplot2::labs(
    title = "Macroeconomic Growth Outlook (2020-2027)",
    subtitle = "Annual year-over-year growth rates for Switzerland vs. Global Demand",
    caption = "Note: Italics indicate model forecasts."
  ) +

  # F) Theme
  ggplot2::theme_void() +
  ggplot2::theme(
    plot.title    = ggplot2::element_text(size = 16,
                                          face = "bold",
                                          hjust = 0.5,
                                          margin = ggplot2::margin(b = 5)),
    plot.subtitle = ggplot2::element_text(size = 11,
                                          color = "gray40",
                                          hjust = 0.5,
                                          margin = ggplot2::margin(b = 15)),
    plot.caption  = ggplot2::element_text(size = 9,
                                          color = "gray50",
                                          hjust = 1,
                                          margin = ggplot2::margin(t = 10)),
    plot.margin   = ggplot2::margin(1, 1, 1, 1, "cm"),
    plot.background = ggplot2::element_rect(fill = "white", color = NA)
  )

print(p_table_fancy)
```

Macroeconomic Growth Outlook (2020–2027)

Annual year−over−year growth rates for Switzerland vs. Global Demand

Year	Swiss GDP	Global Demand
2020	−0.1%	−2.4%
2021	5.1%	5.0%
2022	3.5%	3.6%
2023	1.3%	1.3%
2024	1.2%	1.4%
2025	*1.4%*	*1.8%*
2026	*1.1%*	*1.8%*
2027	*1.9%*	*1.8%*

Note: Italics indicate model forecasts.

Figure 12: Publication-style table: annual growth rates for Switzerland and global demand. Italics indicate forecasts.

```
ggplot2::ggsave(
  filename = "output/table_forecast_professional.png",
  plot = p_table_fancy,
  width = 7,
  height = 6,
  dpi = 300
)

ggplot2::ggsave(
  filename = "output/table_forecast_professional.pdf",
  plot = p_table_fancy,
  width = 7,
  height = 6
)
```

10 Appendix A — Core functions

This appendix documents the helper functions implemented in `R/functions.R`.
For each function, we provide (when relevant): **purpose**, **theory/definition** (with equations in ...), and **implementation notes** (only if useful).

Then I reproduce the **exact function code** as used in the project.

10.1 Data download & transformation functions

10.1.1 `download_fred_data()`

This function download a time series from FRED (as CSV) and restrict the sample to observations from `start_date` onward.

- Uses the FRED "fredgraph.csv" endpoint.
- Assumes the date column is named `observation_date` after `read_csv()`.

```
download_fred_data <- function(fred_id, outputput_filename,
                               start_date = "2000-01-01") {
  fred_url <- paste0("https://fred.stlouisfed.org/graph/fredgraph.csv?id=",
                     fred_id)
  data <- read_csv(fred_url) %>%
    filter(observation_date >= start_date)

  return(data)
}
```

10.1.2 `impute_missing_quarters()`

Purpose
Impute missing quarterly observations using Kalman smoothing. The function aligns a target time series with a reference ("anchor") series by adding missing rows and imputing the corresponding missing values.

Implementation notes

- If `anchor_data` contains more rows than `target_data`, we append the required number of `NA` rows to `target_data`.

- Missing values are then filled using `imputeTS::na_kalman(..., model = "StructTS")`.

```r
impute_missing_quarters <- function(target_data, anchor_data, value_col) {
  missing_rows <- nrow(anchor_data) - nrow(target_data)

  if (missing_rows > 0) {
    target_data <- bind_rows(
      target_data,
      tibble(!!sym(value_col) := rep(NA, missing_rows))
    )
    target_data[[value_col]] <- na_kalman(target_data[[value_col]], model = "StructTS")
  }

  return(target_data)
}
```

10.1.3 `calculate_growth_rate()`

Purpose
Compute quarterly (QoQ) or yearly (YoY) growth rates using log-differences. The growth rate is set to zero whenever it cannot be computed (first observation for QoQ; first four observations for YoY).

Theory / definition
Let x_t be the level series.

- **QoQ growth**: $g_t = 100 \cdot (\log x_t - \log x_{t-1})$
- **YoY growth**: $g_t = 100 \cdot (\log x_t - \log x_{t-4})$

Implementation notes
- The function returns a vector of the same length as `variable`.
- It pads with zeros where the log-difference is undefined.

```r
calculate_growth_rate <- function(variable, type = "qoq") {
  log_var <- log(variable)
  if (type == "qoq") {
    growth <- c(0, diff(log_var)) * 100
  } else if (type == "yoy") {
    growth <- c(rep(0, 4), diff(log_var, 4)) * 100
  }
  return(growth)
}
```

10.1.4 `clean_gdp_data()`

Purpose
Clean and standardize international GDP series into a harmonized quarterly growth dataset.

The function: - converts different date formats into a unified `Date` variable, - computes quarterly growth rates using log-differences, - returns a dataset with a single country-specific growth variable.

Definition
Quarterly GDP growth is computed as:

$$g_t = 100 \cdot (\log \mathrm{GDP}_t - \log \mathrm{GDP}_{t-1})$$

Implementation notes

- Supports three input date formats:
  - `"date"`: already a standard date column,
  - `"quarter"`: quarterly strings parsed using `lubridate::yq()`,
  - `"china"`: China-specific quarterly format using `gdp_china_sa`.
- The output column is renamed to `{country_name}_gdp_growth`.

```
clean_gdp_data <- function(data, date_col, gdp_col,
                           date_format = "date", country_name) {

  if (date_format == "date") {
    data_clean <- data %>%
      rename(date = {{date_col}}, gdp_temp = {{gdp_col}}) %>%
      mutate(
        date = as.Date(date),
        growth_rate = calculate_growth_rate(gdp_temp, "qoq")
      )

  } else if (date_format == "quarter") {
    data_clean <- data %>%
      rename(gdp_temp = {{gdp_col}}) %>%
      mutate(
        gdp_temp = as.numeric(gdp_temp),
        date = lubridate::yq(quarter),
        growth_rate = calculate_growth_rate(gdp_temp, "qoq")
      )

  } else if (date_format == "china") {
    data_clean <- data %>%
      mutate(
        date = lubridate::yq(paste0("20", str_sub(quarter, -2), " ",
                                    str_sub(quarter, 1, 1))),
        growth_rate = calculate_growth_rate(gdp_china_sa, "qoq")
      )
  }

  data_clean %>%
    select(date, growth_rate) %>%
```

```
    rename(!!paste0(country_name, "_gdp_growth") := growth_rate)
}
```

10.1.5 `compute_winsor_bounds()`

Purpose
Compute the lower and upper quantile bounds used for winsorization of a numeric series.
These bounds are typically used to cap extreme observations (outliers) at chosen quantiles (default: 5% and 95%).

Theory
Let X be the series. The winsorization bounds are: - lower bound: $q_\alpha(X)$ with $\alpha = 0.05$ - upper bound: $q_{1-\alpha}(X)$ with $1 - \alpha = 0.95$

So we return:

$$\text{lower} = q_{0.05}(X), \quad \text{upper} = q_{0.95}(X)$$

```
compute_winsor_bounds <- function(variable,
                                  lower_quantile = 0.05,
                                  upper_quantile = 0.95) {

  lower_bound <- as.numeric(quantile(variable, lower_quantile, na.rm = TRUE))
  upper_bound <- as.numeric(quantile(variable, upper_quantile, na.rm = TRUE))

  return(c(lower_bound = lower_bound, upper_bound = upper_bound))
}
```

10.1.6 `winsorize_period()`

Purpose
Apply winsorization to a variable **only within a specified year–quarter window**.
Observations outside this window remain unchanged, while values within the window are capped at pre-computed winsorization bounds.

Theory
Let x_t denote the original series and (L, U) the winsorization bounds.
For observations t in the selected period:

$$x_t^{\text{wins}} = \min\big(\max(x_t, L), U\big)$$

Outside the period, $x_t^{\text{wins}} = x_t$.

Implementation notes
- The original series is preserved. - A new column `{variable_name}_winsorized` is added to the dataset. - Period selection is based on `(year, quarter)` comparisons.

```r
winsorize_period <- function(data,
                             variable_name,
                             start_year, start_quarter,
                             end_year, end_quarter,
                             winsor_bounds) {

  data <- data[order(data$year, data$quarter), ]

  period_index <- which(
    (data$year > start_year | (data$year == start_year &
                               data$quarter >= start_quarter)) &
      (data$year < end_year   | (data$year == end_year
                               & data$quarter <= end_quarter))
  )

  original_values <- data[[variable_name]]
  winsorized_values <- original_values

  winsorized_values[period_index] <- pmin(
    pmax(original_values[period_index], winsor_bounds["lower_bound"]),
    winsor_bounds["upper_bound"]
  )

  data[[paste0(variable_name, "_winsorized")]] <- winsorized_values
  return(data)
}
```

10.2 Statistical testing and diagnostics

10.2.1 `adf_stationarity()`

Purpose
Perform an Augmented Dickey–Fuller (ADF) test to assess whether a time series contains a unit root.
The null hypothesis is that the series is **non-stationary**.

Definition
The ADF test is based on the regression:

$$\Delta y_t = \alpha + \beta y_{t-1} + \sum_{i=1}^{k} \gamma_i \Delta y_{t-i} + \varepsilon_t$$

- H_0: $\beta = 0$ (unit root, non-stationary)
- H_1: $\beta < 0$ (stationary)

Rejection of H_0 implies stationarity.

Implementation notes

- Uses `tseries::adf.test()`.
- The lag order k is controlled by `lag_included`.
- The function prints the test output and a short textual conclusion.

```
adf_stationarity <- function(variable,
                             lag_included = 4) {

  test <- tseries::adf.test(variable, k = lag_included)
  print(test)

  if (test$p.value < 0.05) {
    cat("\nConclusion: STATIONARY (unit root rejected)\n")
  } else {
    cat("\nConclusion: NON-STATIONARY (unit root not rejected)\n")
  }
}
```

10.2.2 `kpss_stationarity()`

Purpose
Run a KPSS test to assess whether a time series is **stationary**.
Unlike the ADF test, the null hypothesis of the KPSS test is that the series **is stationary**.

Definition
The KPSS test decomposes the series as:

$$y_t = \mu_t + \varepsilon_t, \quad \mu_t = \mu_{t-1} + u_t$$

where u_t is a random walk component.

- H_0: the series is stationary
- H_1: the series is non-stationary

Implementation notes

- Uses `tseries::kpss.test()`.
- The function prints both the test output and a short textual conclusion.

```
kpss_stationarity <- function(variable) {

  test <- tseries::kpss.test(variable)
  print(test)

  if (test$p.value > 0.05) {
    cat("\nConclusion: STATIONARY (stationarity not rejected)\n")
  } else {
    cat("\nConclusion: NON-STATIONARY (stationarity rejected)\n")
```

```
  }
}
```

10.2.3 `bai_perron_breaks()`

Purpose

Detect **structural breaks in the mean** of a time series using the Bai–Perron methodology. The function estimates multiple breakpoints and selects the optimal number of breaks using the Bayesian Information Criterion (BIC).

Theory / definition
The model tested is:

$$y_t = \mu_j + \varepsilon_t \quad \text{for } t = T_{j-1} + 1, \dots, T_j$$

where μ_j is a regime-specific mean.

- H_0: no structural breaks (constant mean)
- H_1: one or more breaks in the mean

The Bai–Perron procedure:

- estimates breakpoints subject to a minimum segment length,
- selects the number of breaks by minimizing BIC.

Implementation notes

- Uses `strucchange::breakpoints()` with model $y_t \sim 1$.
- `min_segment_fraction` enforces a minimum regime length as a fraction of the sample.
- Break dates are mapped back to calendar dates using the `date` column.

```
bai_perron_breaks <- function(data, series_name,
                              min_segment_fraction = 0.15) {
  y <- data[[series_name]]

  bp_test <- strucchange::breakpoints(y ~ 1, h = min_segment_fraction)
  print(bp_test)

  bic_values <- BIC(bp_test)
  optimal_breaks <- which.min(bic_values) - 1

  cat("\nConclusion: BIC selects", optimal_breaks, "structural break(s)\n")

  if (optimal_breaks > 0) {
    conf_interval <- confint(bp_test, breaks = optimal_breaks)
    break_index <- conf_interval$confint[, "breakpoints"]
    cat("Break date(s):", as.character(data$date[break_index]), "\n")
  }
}
```

10.2.4 `rec_cusum_test()`

Purpose
Test the **stability of regression parameters over time** using the recursive CUSUM (Rec-CUSUM) procedure.
The test evaluates whether the relationship specified by a regression model remains stable throughout the sample.

This diagnostic is used to assess whether a predictive relationship (e.g. Swiss GDP on external demand) exhibits **structural instability** that could undermine forecasting performance.

Definition
Consider the linear regression model:

$$y_t = x_t^\top \beta + \varepsilon_t$$

estimated recursively using observations $1, \dots, t$.

The recursive CUSUM statistic is constructed from the cumulative sum of standardized recursive residuals:

$$W_t = \sum_{i=k+1}^{t} \frac{\hat{\varepsilon}_i}{\hat{\sigma}}$$

where k is the number of regressors.

- H_0: regression parameters are stable over time
- H_1: at least one parameter changes over time

If the CUSUM process crosses the critical boundaries, parameter stability is rejected.

Implementation notes

- Uses `strucchange::efp()` with `type = "Rec-CUSUM"`.
- The test is applied to a regression specified via a formula (e.g. `y ~ x`).
- A p-value is obtained via `strucchange::sctest()`.
- The function prints both the test object and a concise stability conclusion.

```
rec_cusum_test <- function(formula, data) {

  cat("\nRec-CUSUM | Model:", deparse(formula), "\n")

  test <- strucchange::efp(formula, data = data, type = "Rec-CUSUM")
  print(test)

  p_value <- strucchange::sctest(test)$p.value

  if (p_value < 0.05) {
    cat("\nConclusion: UNSTABLE relationship (p =", round(p_value, 4), ")\n")
  } else {
    cat("\nConclusion: Stable relationship (p =", round(p_value, 4), ")\n")
  }
}
```

10.2.5 `granger_causality()`

Purpose
Run **Granger causality tests** between two time series (in both directions) for several lag orders. The goal is to check whether past values of one variable contain **predictive information** about the other variable beyond what is already in its own past.

This is useful in the project to assess whether (for example) external demand helps forecast Swiss GDP growth, and/or whether Swiss GDP growth helps forecast external demand.

Definition
Let y_t and x_t be two stationary series. The Granger causality test compares:

- **Restricted model** (no x lags):

$$y_t = \alpha + \sum_{i=1}^{p} \phi_i y_{t-i} + u_t$$

- **Unrestricted model** (with x lags):

$$y_t = \alpha + \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{i=1}^{p} \theta_i x_{t-i} + u_t$$

The null hypothesis for "x does not Granger-cause y" is:

$$H_0 : \ \theta_1 = \theta_2 = \cdots = \theta_p = 0$$

If the F-test rejects H_0, we conclude that x **Granger-causes** y (in the predictive sense).

Implementation notes

- Uses `lmtest::grangertest()` for each lag order in `lags`.
- Runs the test in both directions:
  - $x \to y$ (does x help predict y?)
  - $y \to x$ (does y help predict x?)
- Prints the test output and a short qualitative conclusion (none / one-way / bidirectional).

```
granger_causality <- function(data, x, y, lags = c(2, 4, 6, 8),
                              winsorized = FALSE) {

  cat("\n=== Granger causality tests ===\n")
  cat("Series:", x, "and", y, "| Winsorized:", winsorized, "\n")

  for (lag in lags) {
    cat("\n--- Lag =", lag, "---\n")

    test_x_to_y <- lmtest::grangertest(
      stats::as.formula(paste(y, "~", x)),
```

```
      data = data,
      order = lag
    )
    print(test_x_to_y)

    test_y_to_x <- lmtest::grangertest(
      stats::as.formula(paste(x, "~", y)),
      data = data,
      order = lag
    )
    print(test_y_to_x)

    p_x_to_y <- test_x_to_y$`Pr(>F)`[2]
    p_y_to_x <- test_y_to_x$`Pr(>F)`[2]

    if (p_x_to_y < 0.05 && p_y_to_x >= 0.05) {
      cat("\nCausality:", x, "→", y, "\n")
    } else if (p_x_to_y >= 0.05 && p_y_to_x < 0.05) {
      cat("\nCausality:", y, "→", x, "\n")
    } else if (p_x_to_y < 0.05 && p_y_to_x < 0.05) {
      cat("\nCausality: Bidirectional (", x, " ", y, ")\n", sep = "")
    } else {
      cat("\nCausality: None\n")
    }
  }
}
```

10.2.6 `ljung_box_residuals()`

Purpose
Test whether a model's residuals exhibit **autocorrelation** (serial correlation).
In a well-specified forecasting model, residuals should behave like **white noise**: no predictable structure left.

Definition
The Ljung–Box test is a portmanteau test for autocorrelation up to lag m.

Let $\hat{\rho}_k$ be the sample autocorrelation of residuals at lag k, and n be the sample size. The Ljung–Box statistic is:

$$Q = n(n+2)\sum_{k=1}^{m}\frac{\hat{\rho}_k^2}{n-k}$$

- H_0: no autocorrelation up to lag m (residuals are white noise)
- H_1: autocorrelation exists at one or more lags $\leq m$

A small p-value (e.g., < 0.05) implies rejecting H_0 → evidence of autocorrelation.

Implementation notes

- Uses `stats::Box.test(..., type = "Ljung-Box")`.
- The lag order m is controlled by `lag` (default 10).
- Prints the test output and a short conclusion.

```
ljung_box_residuals <- function(model, lag = 10) {

  residuals <- resid(model)

  test <- Box.test(residuals, lag = lag, type = "Ljung-Box")
  print(test)

  if (test$p.value > 0.05) {
    cat("\nConclusion: No significant autocorrelation
        (residuals behave as white noise)\n")
  } else {
    cat("\nConclusion: Significant autocorrelation detected in residuals\n")
  }
}
```

10.2.7 `ljung_box_squared_residuals()`

Purpose
Test for **conditional heteroskedasticity** in model residuals by checking for autocorrelation in **squared residuals**.
This diagnostic is commonly used to detect ARCH-type effects, which indicate time-varying volatility.

Definition
Let $\hat{\varepsilon}_t$ denote the residuals from a fitted model.
If the variance is constant over time, the squared residuals $\hat{\varepsilon}_t^2$ should exhibit no autocorrelation.

The Ljung–Box statistic applied to squared residuals is:

$$Q = n(n+2)\sum_{k=1}^{m}\frac{\hat{\rho}_k^2}{n-k},$$

where $\hat{\rho}_k$ is the autocorrelation of $\hat{\varepsilon}_t^2$ at lag k.

- H_0: no autocorrelation in squared residuals (homoskedasticity)
- H_1: autocorrelation in squared residuals (conditional heteroskedasticity)

Implementation notes

- Uses `stats::Box.test()` applied to $\hat{\varepsilon}_t^2$.

- The lag order m is controlled by `lag` (default 5).
- Prints the test output and a short diagnostic conclusion.

```
ljung_box_squared_residuals <- function(model, lag = 5) {

  squared_residuals <- resid(model)^2

  test <- Box.test(squared_residuals, lag = lag, type = "Ljung-Box")
  print(test)

  if (test$p.value > 0.05) {
    cat("\nConclusion: No evidence of conditional heteroskedasticity
        (variance is stable)\n")
  } else {
    cat("\nConclusion: Evidence of conditional heteroskedasticity detected\n")
  }
}
```

10.2.8 `jarque_bera_residuals()`

Purpose
Test whether model residuals are **normally distributed** using the Jarque–Bera test.
Normality of residuals is a standard assumption underlying many inferential procedures and is particularly relevant for density forecasting and simulation-based inference.

Theory
The Jarque–Bera (JB) test is based on the skewness and kurtosis of the residuals.

Let S denote sample skewness and K sample kurtosis. The JB statistic is:

$$\text{JB} = \frac{n}{6}\left(S^2 + \frac{(K-3)^2}{4}\right),$$

which asymptotically follows a $\chi^2(2)$ distribution.

- H_0: residuals are normally distributed
- H_1: residuals are not normally distributed

Implementation notes

- Uses `tseries::jarque.bera.test()`.
- The function prints the test output and a short normality conclusion.

```r
jarque_bera_residuals <- function(model) {

  residuals <- resid(model)

  test <- tseries::jarque.bera.test(residuals)
  print(test)

  if (test$p.value > 0.05) {
    cat("\nConclusion: Residuals are normal\n")
  } else {
    cat("\nConclusion: Residuals are not normal\n")
  }
}
```

10.3 Standard forecast evaluation functions

10.3.1 `forecast_metrics()`

Purpose
Compute standard **point forecast evaluation metrics** comparing realized values and forecasts. These measures summarize forecast accuracy and bias and are used throughout the project to compare competing models.

Definition
Let y_t denote the realized value and $\hat{y}_t$ the forecast. Define the forecast error:

$$e_t = y_t - \hat{y}_t.$$

The function computes:

- **Mean Forecast Error (MFE)**:

$$\text{MFE} = \frac{1}{n}\sum_{t=1}^{n} e_t$$

  (measures average bias)

- **Mean Absolute Error (MAE)**:

$$\text{MAE} = \frac{1}{n}\sum_{t=1}^{n} |e_t|$$

- **Mean Squared Forecast Error (MSFE)**:

$$\text{MSFE} = \frac{1}{n}\sum_{t=1}^{n} e_t^2$$

- **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\text{MSFE}}$$

Only observations where both y_t and $\hat{y}_t$ are available are used.

Implementation notes

- Enforces equal-length inputs via `stopifnot()`.
- Automatically removes missing values pairwise.
- Returns a named list containing all metrics and the effective sample size n.

```
forecast_metrics <- function(y_real, y_hat) {
  stopifnot(length(y_real) == length(y_hat))

  index <- which(!is.na(y_real) & !is.na(y_hat))

  e <- y_real[index] - y_hat[index]

  n    <- length(e)
  mfe  <- mean(e)
  mae  <- mean(abs(e))
  msfe <- mean(e^2)
  rmse <- sqrt(msfe)

  return(list(n = n,
              mfe = mfe,
              mae = mae,
              msfe = msfe,
              rmse = rmse))
}
```

10.3.2 `t_test_unbiasedness()`

Purpose

Test **forecast unbiasedness** using a HAC-corrected t-test.
The null hypothesis is that the expected forecast error is zero.
The HAC correction accounts for serial correlation induced by **overlapping multi-step forecasts**.

Definition
Let y_t be the realized value and $\hat{y}_t$ the h-step-ahead forecast.
Define the forecast error:

$$e_t = y_t - \hat{y}_t.$$

Forecast unbiasedness implies:

$$\mathbb{E}[e_t] = 0.$$

This is tested via the regression:

$$e_t = \alpha + u_t,$$

with null hypothesis:

$$H_0 : \ \alpha = 0.$$

Because forecast errors are serially correlated when forecasts overlap, inference is based on a **Newey–West HAC covariance estimator** with lag length:

$$L = h - 1.$$

Implementation notes
- Uses a constant-only regression estimated by OLS. - Applies `sandwich::NeweyWest()` with `prewhite = FALSE` and finite-sample adjustment. - Returns the estimated bias, t-statistic, p-value, and HAC lag length.

```
t_test_unbiasedness <- function(y_real, y_hat, h = 1) {

  index <- which(!is.na(y_real) & !is.na(y_hat))
  e <- y_real[index] - y_hat[index]
  n <- length(e)

  # Regression: e_t = alpha + u_t for HAC NW variance
  fit <- lm(e ~ 1)

  # HAC lag for overlapping forecasts (rule of thumb): L = h - 1
  L <- max(h - 1, 0)

  vcov_hac <- sandwich::NeweyWest(fit, lag = L, prewhite = FALSE, adjust = TRUE)
  coeff_test <- lmtest::coeftest(fit, vcov. = vcov_hac)

  alpha_hat <- coef(fit)[1]
  t_stat <- coeff_test[1, "t value"]
  p_val  <- coeff_test[1, "Pr(>|t|)"]

  return(list(n = n,
              mfe = as.numeric(alpha_hat),
              t_stat = as.numeric(t_stat),
              p_val = as.numeric(p_val),
              hac_lag = L))
}
```

10.3.3 `mincer_zarnowitz_test()`

Purpose
Test **forecast optimality** using the Mincer–Zarnowitz (MZ) regression with **HAC (Newey–West)** standard errors.
Under an optimal (unbiased and correctly scaled) forecast, the realized value should lie on the 45-degree line against the forecast.

Definition
Let y_t be the realized value and $\hat{y}_t$ the forecast. The MZ regression is:

$$y_t = \alpha + \beta \hat{y}_t + u_t.$$

Forecast optimality (in this linear sense) implies:

$$H_0 : \ \alpha = 0 \quad \text{and} \quad \beta = 1.$$

Because multi-step forecasts can generate **overlapping errors**, inference uses a Newey–West HAC covariance matrix with lag length:

$$L = h - 1.$$

The joint hypothesis ($\alpha = 0,\ \beta = 1$) is tested using a **Wald test**:

$$W = (R\hat{\theta} - r)^\top \left(R\widehat{\mathrm{Var}}(\hat{\theta}) R^\top \right)^{-1} (R\hat{\theta} - r),$$

where $\hat{\theta} = (\hat{\alpha}, \hat{\beta})^\top$,
$R = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, and $r = (0, 1)^\top$.
Under H_0, $W \sim \chi^2(2)$.

Implementation notes
- Fits `lm(y ~ f)` on non-missing matched pairs. - Uses `sandwich::NeweyWest()` with `lag = h-1`.
- Computes a Wald statistic and a $\chi^2(2)$ p-value.

```
mincer_zarnowitz_test <- function(y_real, y_hat, h = 1) {

  index <- which(!is.na(y_real) & !is.na(y_hat))
  y <- y_real[index]
  f <- y_hat[index]
  n <- length(y)

  mz_model <- lm(y ~ f)

  # HAC lag for overlapping forecasts: L = h - 1
  L <- max(h - 1, 0)

  vcov_hac <- sandwich::NeweyWest(mz_model,
                                  lag = L,
                                  prewhite = FALSE,
                                  adjust = TRUE)

  alpha_hat <- coef(mz_model)[1]
  beta_hat  <- coef(mz_model)[2]

  # Wald test of: alpha = 0 and beta = 1
  R <- rbind(c(1, 0),
```

```
              c(0, 1))
  r <- c(0, 1)

  diff <- as.matrix(R %*% coef(mz_model) - r)
  V    <- R %*% vcov_hac %*% t(R)

  W_stat <- as.numeric(t(diff) %*% solve(V) %*% diff)
  p_val  <- 1 - pchisq(W_stat, df = 2)

  return(list(
    n = n,
    alpha_hat = as.numeric(alpha_hat),
    beta_hat  = as.numeric(beta_hat),
    F_stat    = as.numeric(W_stat),
    p_val     = as.numeric(p_val)
  ))
}
```

10.3.4 `efficiency_test()`

Purpose
Test **forecast efficiency**: whether forecast errors can be predicted using an information set available at forecast time.
If a forecast is efficient, then (conditional on the information set) forecast errors should be unpredictable.

Definition
Let $e_t = y_t - \hat{y}_t$ be the forecast error and let Z_t denote an information set (one or more predictors). We estimate:

$$e_t = \alpha + Z_t^\top \delta + u_t.$$

Forecast efficiency implies:

$$H_0 : \ \delta = 0$$

(i.e., no element of Z_t has predictive power for e_t).

The function implements this as an **F-test** of joint significance of all regressors in Z_t.

Implementation notes
- Matches non-missing pairs of $(y_t, \hat{y}_t)$ and aligns `info` on the same index. - `info` can be a vector, matrix, or data frame; it is coerced to a data frame. - The returned p-value is computed from the model's overall F-statistic.

```
efficiency_test <- function(y_real, y_hat, info) {

  index <- which(!is.na(y_real) & !is.na(y_hat))
  e <- y_real[index] - y_hat[index]
  info_df <- as.data.frame(info)[index, , drop = FALSE]
```

```
  efficient_model <- lm(
    as.formula(paste("e ~", paste(names(info_df), collapse = " + "))),
    data = cbind(e = e, info_df)
  )

  fstat <- summary(efficient_model)$fstatistic
  p_val <- 1 - pf(fstat[1], fstat[2], fstat[3])

  return(list(
    F_stat = as.numeric(fstat[1]),
    df1    = as.numeric(fstat[2]),
    df2    = as.numeric(fstat[3]),
    p_val  = as.numeric(p_val)
  ))
}
```

10.3.5 `msfe_ratio()`

Purpose
Compute the **MSFE ratio** (Mean Squared Forecast Error) to compare predictive accuracy between two models under **squared loss**.

Definition
Let $e_t^{(1)}$ and $e_t^{(2)}$ be forecast errors from model 1 and model 2, aligned over the same evaluation sample.
The mean squared forecast errors are:

$$\text{MSFE}_1 = \frac{1}{T}\sum_{t=1}^{T}\left(e_t^{(1)}\right)^2, \qquad \text{MSFE}_2 = \frac{1}{T}\sum_{t=1}^{T}\left(e_t^{(2)}\right)^2.$$

The MSFE ratio is:

$$R = \frac{\text{MSFE}_1}{\text{MSFE}_2}.$$

Interpretation: - $R < 1$: model 1 has lower MSFE (better under squared loss), - $R > 1$: model 2 performs better.

Implementation notes
- Drops any period where either error series is missing (`NA`) to ensure fair comparison. - Returns MSFE values for both models and the ratio.

```
msfe_ratio <- function(e1, e2, name1 = "model1", name2 = "model2") {

  stopifnot(length(e1) == length(e2))
  index <- which(!is.na(e1) & !is.na(e2))
  e1 <- e1[index]; e2 <- e2[index]
```

```
  msfe1 <- mean(e1^2)
  msfe2 <- mean(e2^2)
  ratio <- msfe1 / msfe2

  return(list(msfe1 = msfe1,
              msfe2 = msfe2,
              ratio = ratio))
}
```

10.3.6 `mafe_ratio()`

Purpose
Compute the **MAFE ratio** (Mean Absolute Forecast Error) to compare predictive accuracy between two models under **absolute loss**.

Definition
Let $e_t^{(1)}$ and $e_t^{(2)}$ be forecast errors from model 1 and model 2, aligned over the same evaluation sample.
The mean absolute forecast errors are:

$$\text{MAFE}_1 = \frac{1}{T}\sum_{t=1}^{T}\left|e_t^{(1)}\right|, \qquad \text{MAFE}_2 = \frac{1}{T}\sum_{t=1}^{T}\left|e_t^{(2)}\right|.$$

The MAFE ratio is:

$$R = \frac{\text{MAFE}_1}{\text{MAFE}_2}.$$

Interpretation: - $R < 1$: model 1 has lower MAFE (better under absolute loss), - $R > 1$: model 2 performs better.

Implementation notes
- Ensures both error vectors have the same length. - Removes missing values pairwise before computing losses. - Returns MAFE values for both models and their ratio.

```
mafe_ratio <- function(e1, e2, name1 = "model1", name2 = "model2") {

  stopifnot(length(e1) == length(e2))
  index <- which(!is.na(e1) & !is.na(e2))
  e1 <- e1[index]; e2 <- e2[index]

  mafe1 <- mean(abs(e1))
  mafe2 <- mean(abs(e2))
  ratio <- mafe1 / mafe2

  return(list(mafe1 = mafe1,
              mafe2 = mafe2,
              ratio = ratio))
}
```

10.3.7 `dm_test_eval()`

Purpose
Perform a **Diebold–Mariano (DM) test** to assess whether two competing forecasts have **equal predictive accuracy** under squared loss.
This test provides formal statistical inference to complement MSFE or MAFE comparisons.

Definition
Let $e_t^{(1)}$ and $e_t^{(2)}$ denote forecast errors from two models.
Define the loss differential under squared loss:

$$d_t = \left(e_t^{(1)}\right)^2 - \left(e_t^{(2)}\right)^2.$$

The null hypothesis of equal predictive accuracy is:

$$H_0 : \ \mathbb{E}[d_t] = 0.$$

The Diebold–Mariano statistic is constructed as:

$$DM = \frac{\bar{d}}{\sqrt{\widehat{\mathrm{Var}}(\bar{d})}},$$

where $\widehat{\mathrm{Var}}(\bar{d})$ is a HAC variance estimator that accounts for serial correlation induced by h-step-ahead forecasts.

Under H_0, the statistic is asymptotically standard normal.

Implementation notes
- Uses `forecast::dm.test()` with `power = 2` (squared loss). - The forecast horizon h determines the HAC correction. - Returns the DM statistic and the associated p-value.

```
dm_test_eval <- function(e1, e2, h = 1, name1 = "model1", name2 = "model2") {

  stopifnot(length(e1) == length(e2))
  index <- which(!is.na(e1) & !is.na(e2))
  e1 <- e1[index]; e2 <- e2[index]

  dm_res <- forecast::dm.test(e1, e2, h = h, power = 2)

  return(list(dm_stat = as.numeric(dm_res$statistic),
              p_val = as.numeric(dm_res$p.value)))
}
```

10.4 Rolling window estimation

10.4.1 `estimate_rolling_models()`

Purpose
Estimate, for each rolling window, three models used in the forecasting pipeline:

1. **Swiss AR benchmark**: y_t modeled as an AR(p) selected by BIC (implemented via `auto.arima` with `q = 0`).

2. **External ARMA model**: x_t modeled as an ARMA(p, q) selected by BIC (via `auto.arima`).

3. **Swiss ARDL(p, q)**: y_t regressed on its own lags and contemporaneous/lagged external demand, with (p, q) selected by BIC via an explicit loop.

The function stores, for each origin date:

- the selected specifications (p, q) and fit statistics (BIC and residual scale),
- the estimated coefficients and their standard errors.

Definition
Let y_t be Swiss GDP growth and x_t the external demand growth.

1) **Swiss AR benchmark**

$$y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + u_t, \qquad u_t \sim (0, \sigma^2).$$

2) **External ARMA**

$$x_t = c + \sum_{i=1}^{p} a_i x_{t-i} + \sum_{j=1}^{q} b_j \varepsilon_{t-j} + \varepsilon_t.$$

3) **Swiss ARDL(p, q)**
Using contemporaneous x_t and q lags:

$$y_t = \alpha + \sum_{i=1}^{p} \phi_i y_{t-i} + \theta_0 x_t + \sum_{j=1}^{q} \theta_j x_{t-j} + u_t.$$

For each rolling window, the function selects (p, q) by minimizing BIC over:

$$p \in \{1, \dots, \text{max_lag}\}, \quad q \in \{0, \dots, \text{max_lag}\}.$$

Implementation notes

- Rolling windows are defined by a fixed `window_size` and window end indices `end_indices`.

- BIC is used consistently for model selection:
  - `auto.arima(..., ic = "bic")` for AR and ARMA,
  - `BIC(lm(...))` inside the ARDL grid search.

- To align regressors, ARDL estimation uses a trimmed target:
  - y is taken from indices $(\text{max_lag} + 1) : n$,
  - regressors are built so that all lags are available.

- The function stores:
  - `specifications`: selected (p, q), BIC, and σ per origin date,
  - `coefficients`: coefficient estimates and standard errors per origin date.

```
estimate_rolling_models <- function(data, y_col, x_col, window_size = 24,
                                    max_lag = 3, end_indices = NULL) {

  if (is.null(end_indices))
    end_indices <- window_size:nrow(data)

  # Store selected model specifications per window date
  specifications <- data.frame(
    Date = as.Date(character()),
    Model_Type = character(),
    Best_P = integer(),
    Best_Q = integer(),
    BIC_Score = numeric(),
    Sigma = numeric(),
    stringsAsFactors = FALSE)

  # Store coefficient estimates per window date
  coefficients <- data.frame(
    Date = as.Date(character()),
    Model_Type = character(),
    Term = character(),
    Estimate = numeric(),
    Std_Error = numeric(),
    stringsAsFactors = FALSE
  )

  for (index_end in end_indices) {
    index_start <- index_end - window_size + 1
    if (index_start < 1) next

    origin_date <- as.Date(data$date[index_end])
    y_win <- data[[y_col]][index_start:index_end]
    x_win <- data[[x_col]][index_start:index_end]

    # ------------------------------------------------------------------------
    # Swiss AR Benchmark
    # ------------------------------------------------------------------------
    model_ar <- auto.arima(y_win, max.p = max_lag, max.q = 0,
                           seasonal = FALSE, stationary = TRUE, ic = "bic")

    order_ar <- arimaorder(model_ar)

    specifications <- rbind(specifications,
```

```
                          data.frame(Date = origin_date,
                                     Model_Type = "Swiss_AR_Benchmark",
                                     Best_P = order_ar[1],
                                     Best_Q = order_ar[3],
                                     BIC_Score = model_ar$bic,
                                     Sigma = sqrt(model_ar$sigma2)))

  coefficient_ar <- coef(model_ar)
  se_ar <- sqrt(diag(model_ar$var.coef))

  if(length(coefficient_ar)>0) {
    for(k in seq_along(coefficient_ar))
      coefficients <- rbind(coefficients,
                            data.frame(Date=origin_date,
                                       Model_Type="Swiss_AR_Benchmark",
                                       Term=names(coefficient_ar)[k],
                                       Estimate=coefficient_ar[k],
                                       Std_Error=se_ar[k]))
  }

  # ------------------------------------------------------------------------
  # EXTERNAL ARMA
  # ------------------------------------------------------------------------
  model_external_arma <- auto.arima(x_win, max.p = max_lag, max.q = max_lag,
                                    seasonal = FALSE,
                                    stationary = TRUE, ic = "bic")

  order_external_arma <- arimaorder(model_external_arma)

  specifications <- rbind(specifications,
                          data.frame(Date = origin_date,
                                     Model_Type = "External_ARMA",
                                     Best_P = order_external_arma[1],
                                     Best_Q = order_external_arma[3],
                                     BIC_Score = model_external_arma$bic,
                                     Sigma = sqrt(model_external_arma$sigma2)))

  coefficient_external_arma <- coef(model_external_arma)
  se_ext <- sqrt(diag(model_external_arma$var.coef))

  if(length(coefficient_external_arma)>0) {
    for(k in seq_along(coefficient_external_arma))
      coefficients <- rbind(coefficients,
                            data.frame(Date=origin_date,
                                       Model_Type="External_ARMA",
                                       Term=names(coefficient_external_arma)[k],
                                       Estimate=coefficient_external_arma[k],
```

```
                                        Std_Error=se_ext[k]))
}

# ------------------------------------------------------------------------
# SWISS ARDL(p,q) via BIC loop
# ------------------------------------------------------------------------
best_bic <- Inf
best_fit <- NULL
best_p <- NA
best_q <- NA

n_win <- length(y_win)
y_target <- y_win[(max_lag + 1):n_win]

for (p in 1:max_lag) {
  for (q in 0:max_lag) {

    reg_list <- list()

    # lagged y terms
    for (k in 1:p) {
      reg_list[[paste0("y_lag", k)]] <- y_win[(max_lag + 1 - k):(n_win - k)]
    }

    # contemporaneous x and lagged x terms
    reg_list[["x_lag0"]] <- x_win[(max_lag + 1):n_win]
    if (q > 0) {
      for (k in 1:q) {
        reg_list[[paste0("x_lag", k)]] <- x_win[(max_lag + 1 - k):(n_win - k)]
      }
    }

    df_tmp <- as.data.frame(reg_list)
    df_tmp$y <- y_target

    fit <- lm(y ~ ., data = df_tmp)
    bic_fit <- BIC(fit)

    if (bic_fit < best_bic) {
      best_bic <- bic_fit
      best_fit <- fit
      best_p <- p
      best_q <- q
    }
  }
}
```

```
    if (!is.null(best_fit)) {
      specifications <- rbind(specifications,
                              data.frame(Date = origin_date,
                                         Model_Type = "Swiss_ARDL",
                                         Best_P = best_p,
                                         Best_Q = best_q,
                                         BIC_Score = best_bic,
                                         Sigma = sigma(best_fit)))

      sum_ardl <- summary(best_fit)$coefficients
      for (tm in rownames(sum_ardl))
        coefficients <- rbind(coefficients,
                              data.frame(Date=origin_date,
                                         Model_Type="Swiss_ARDL",
                                         Term=tm,
                                         Estimate=sum_ardl[tm, "Estimate"],
                                         Std_Error=sum_ardl[tm, "Std. Error"]))
    }
  }
  return(list(specifications = specifications, coefficients = coefficients))
}
```

10.5 Rolling forecasting functions

10.5.1 `get_coefficient_safe()`

Purpose
Safely extract a specific coefficient estimate from the rolling `coefficients` table at a given `target_date` and `model_type`.
If the coefficient is missing (e.g., because a term was not selected in that rolling window), the function returns `0` to prevent downstream code from breaking when constructing forecasts.

Definition
Let $\hat{\beta}_{d,m,k}$ denote the estimated coefficient for:

- date d,
- model type m,
- term (coefficient name) k.

This function implements:

$$\hat{\beta}_{d,m,k} = \begin{cases} \text{stored estimate} & \text{if the coefficient exists in the table,} \\ 0 & \text{otherwise.} \end{cases}$$

Implementation notes

- The matching is done jointly on `Date`, `Model_Type`, and `Term`.

- Returning `0` is a deliberate design choice: it treats non-selected coefficients as absent regressors in the forecast recursion.

```
get_coefficient_safe <- function(coefficient_table,
                                 target_date,
                                 model_type,
                                 coefficient_name) {

  estimate <- coefficient_table$Estimate[
    as.Date(coefficient_table$Date) == as.Date(target_date) &
      coefficient_table$Model_Type == model_type &
      coefficient_table$Term == coefficient_name
  ]

  if (length(estimate) == 0) return(0)
  return(estimate)
}
```

10.5.2 `multi_horizon_recursive_forecasting()`

Purpose
Generate **multi-horizon recursive forecasts** from each rolling-window origin date. For every origin date stored in `specifications`, the function:

1) Forecasts the external driver x using the estimated **External ARMA** model coefficients at that origin,

2) Produces Swiss GDP forecasts y using:

- a **Swiss AR benchmark**, and
- a **Swiss ARDL(p,q)** model (feeding in the forecasted x),

3) Stores realized values, forecasts, and forecast errors for horizons $h = 1, \dots, H$.

The output is a long data frame with one row per `(origin_date, horizon)`.

Definition

Let $h \in \{1, \dots, H\}$ denote the forecast horizon and let t be the origin index.

(1) External ARMA recursion (deterministic here)
In general:

$$x_{t+h} = c_x + \sum_{i=1}^{p_x} \phi_i^{(x)} x_{t+h-i} + \sum_{j=1}^{q_x} \psi_j^{(x)} \varepsilon_{t+h-j} + \varepsilon_{t+h}.$$

In this implementation, the future innovations are set to zero (deterministic recursion):

$$\hat{x}_{t+h} = c_x + \sum_{i=1}^{p_x} \phi_i^{(x)} \hat{x}_{t+h-i},$$

because the MA buffer is initialized at 0 and kept at 0.

(2) Swiss AR benchmark

$$\hat{y}_{t+h}^{AR} = c_y + \sum_{i=1}^{p_y} \phi_i^{(y)} \hat{y}_{t+h-i}^{AR}.$$

(3) Swiss ARDL(p,q)

$$\hat{y}_{t+h}^{ARDL} = \alpha + \sum_{i=1}^{p} \phi_i \hat{y}_{t+h-i}^{ARDL} + \theta_0 \hat{x}_{t+h} + \sum_{j=1}^{q} \theta_j \hat{x}_{t+h-j}.$$

Forecast errors

$$e_{t+h}^{AR} = y_{t+h} - \hat{y}_{t+h}^{AR}, \qquad e_{t+h}^{ARDL} = y_{t+h} - \hat{y}_{t+h}^{ARDL}.$$

Implementation notes

- Coefficients are retrieved at each origin date via `get_coefficient_safe()`, which returns `0` if a term is absent.
- The function uses buffers ordered as $(t, t-1, t-2, \dots)$ for recursive simulation.
- If a model includes fewer than `max_lag` lags (e.g., $p < max_lag$), missing lag coefficients effectively behave as zero.
- MA terms for the external ARMA are included structurally, but future shocks are fixed at zero via `buffer_e = 0`.

```
multi_horizon_recursive_forecasting <- function(data,
                                                specifications,
                                                coefficients,
                                                y_col,
                                                x_col,
                                                max_h = 10,
                                                max_lag = 3) {

  origin_dates <- unique(as.Date(specifications$Date))

  rows <- vector("list", length(origin_dates) * max_h)
  position <- 0

  # Loop over forecast origins
  for (origin_date in origin_dates) {

    index_end <- which(as.Date(data$date) == origin_date)

    if (length(index_end) == 0) next
    if (index_end < max_lag) next
```

```
  if ((index_end + max_h) > nrow(data)) next


  # ============================================================================
  # A) EXTERNAL MODEL (ARMA) COEFFICIENTS AT ORIGIN DATE
  # ============================================================================
  constant_external <- get_coefficient_safe(coefficients,
                                    origin_date,
                                    "External_ARMA",
                                    "intercept")

  if(constant_external==0)
    constant_external <- get_coefficient_safe(coefficients,
                                      origin_date,
                                      "External_ARMA",
                                      "mean")

  ar_external <- sapply(1:max_lag, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "External_ARMA",
                  paste0("ar", k)))

  ma_external <- sapply(1:max_lag, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "External_ARMA",
                  paste0("ma", k)))

  # ============================================================================
  # B) SWISS AR BENCHMARK COEFFICIENTS AT ORIGIN DATE
  # ============================================================================
  constant_ar <- get_coefficient_safe(coefficients,
                        origin_date,
                        "Swiss_AR_Benchmark",
                        "intercept")

  if(constant_ar==0)
    constant_ar <- get_coefficient_safe(coefficients,
                          origin_date,
                          "Swiss_AR_Benchmark",
                          "mean")

  phi_ar <- sapply(1:max_lag, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "Swiss_AR_Benchmark",
```

```
                paste0("ar", k)))

# ============================================================================
# C) SWISS ARDL COEFFICIENTS + (p,q) SELECTION AT ORIGIN DATE
# ============================================================================
constant_ardl <- get_coefficient_safe(coefficients,
                                origin_date,
                                "Swiss_ARDL",
                                "(Intercept)")

ardl_theta0 <- get_coefficient_safe(coefficients,
                         origin_date,
                         "Swiss_ARDL",
                         "x_lag0")


sp <- subset(specifications,
             as.Date(Date) == as.Date(origin_date)
             & Model_Type == "Swiss_ARDL")


p <- ifelse(nrow(sp)==0, 0, sp$Best_P[1])
q <- ifelse(nrow(sp)==0, 0, sp$Best_Q[1])

phi_ardl <- if(p>0)
  sapply(1:p, function(k) get_coefficient_safe(coefficients,
                                               origin_date,
                                               "Swiss_ARDL",
                                               paste0("y_lag", k)))
else numeric(0)
theta_ardl <- if(q>0)
  sapply(1:q, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "Swiss_ARDL",
                  paste0("x_lag", k)))
else numeric(0)

# ============================================================================
# D) INITIAL BUFFERS (ORDER: t, t-1, t-2, ...)
# ============================================================================
hist_x <- rev(data[[x_col]]
              [(index_end-max_lag+1):index_end])

hist_y <- rev(data[[y_col]]
              [(index_end-max_lag+1):index_end])
```

```r
  # External recursion buffers
  buffer_x_external<- hist_x
  buffer_e <- rep(0, max_lag)

  # Swiss AR buffers
  buffer_y_ar <- hist_y

  # Swiss ARDL buffers
  buffer_y_ardl <- hist_y
  buffer_x_ardl <- hist_x

  # ============================================================================
  # E) RECURSIVE FORECASTING OVER HORIZONS h = 1..max_h
  # ============================================================================
  for (h in 1:max_h) {

    # External forecast x̂_{t+h}
    # Deterministic ARMA recursion: MA part is zero if buffer_e == 0
    x_hat <- constant_external + sum(ar_external *
                                       buffer_x_external) + sum(ma_external * buffer_e)

    # Swiss AR benchmark forecast ŷ^{AR}_{t+h}
    y_hat_ar <- constant_ar + sum(phi_ar * buffer_y_ar)

    # Swiss ARDL forecast ŷ^{ARDL}_{t+h}
    ar_part <- if (p > 0) sum(phi_ardl * buffer_y_ardl[1:p]) else 0
    dl_part <- if (q > 0) sum(theta_ardl * buffer_x_ardl[1:q]) else 0

    y_hat_ardl <- constant_ardl + ar_part + (ardl_theta0 *
                                               x_hat) + dl_part

    # Realized value & errors
    y_real <- data[[y_col]][index_end + h]

    position <- position + 1
    rows[[position]] <- data.frame(
      origin_date = origin_date,
      horizon     = h,
      y_real      = y_real,
      y_hat_ar    = y_hat_ar,
      y_hat_ardl  = y_hat_ardl,
      e_ar        = y_real - y_hat_ar,
      e_ardl      = y_real - y_hat_ardl,
      stringsAsFactors = FALSE
    )

    # Update buffers (recursive)
```

```
      buffer_x_external <- c(x_hat, buffer_x_external[1:(max_lag - 1)])
      buffer_e <- c(0, buffer_e[1:(max_lag - 1)])
      buffer_y_ar <- c(y_hat_ar, buffer_y_ar[1:(max_lag - 1)])
      buffer_y_ardl <- c(y_hat_ardl, buffer_y_ardl[1:(max_lag - 1)])
      buffer_x_ardl <- c(x_hat,      buffer_x_ardl[1:(max_lag - 1)])
    }
  }
  #Bind rows into final data.frame ----
  if (position == 0) return(data.frame())
  do.call(rbind, rows[1:position])
}
```

10.5.3 `get_forecast_simulation_dynamic()`

Purpose

Produce a **density forecast** (fan chart) for Swiss GDP growth using Monte Carlo simulation with a dynamic external driver:

1) Fit an **ARMA** model to the external series x (via BIC) and simulate n_{sims} future paths $x_{t+1:t+H}$.

2) Select the best **ARDL(p,q)** specification for y on the training sample (BIC over $p = 1, \dots, \text{max_lag}$ and $q = 0, \dots, \text{max_lag}$).

3) Simulate $y_{t+1:t+H}$ **recursively** given each simulated x path, adding Gaussian shocks with variance equal to the ARDL residual variance.

4) Return the simulated mean path and a central **90% interval** (5th / 95th percentiles).

Definition

Let the selected ARDL be:

$$y_t = \alpha + \sum_{i=1}^{p} \phi_i y_{t-i} + \theta_0 x_t + \sum_{j=1}^{q} \theta_j x_{t-j} + u_t, \qquad u_t \sim \mathcal{N}(0, \sigma^2).$$

Given simulated external paths $\{x_{t+h}^{(s)}\}_{h=1}^{H}$ for simulation s, the recursive simulation for y is:

$$y_{t+h}^{(s)} = \alpha + \sum_{i=1}^{p} \phi_i \, y_{t+h-i}^{(s)} + \theta_0 \, x_{t+h}^{(s)} + \sum_{j=1}^{q} \theta_j \, x_{t+h-j}^{(s)} + \varepsilon_{t+h}^{(s)}, \quad \varepsilon_{t+h}^{(s)} \sim \mathcal{N}(0, \sigma^2).$$

The fan chart summaries at each horizon h are:

$$\hat{\mu}_h = \frac{1}{n_{\text{sims}}} \sum_{s=1}^{n_{\text{sims}}} y_{t+h}^{(s)}, \qquad \hat{q}_{0.05,h},\ \hat{q}_{0.95,h} \text{ from the empirical distribution of } \{y_{t+h}^{(s)}\}_{s=1}^{n_{\text{sims}}}.$$

Implementation notes

- External simulations use `simulate(model_external_arma, nsim = H)` from the fitted `auto.arima()` object.

- ARDL selection is performed by explicit BIC looping over (p, q); the final simulation still builds coefficient vectors of length `max_lag`, with unused lags implicitly set to zero.

- Shocks are i.i.d. Gaussian with standard deviation `sigma(best_fit)`; this makes the forecast distribution conditional on the ARDL Gaussian error assumption.

```
get_forecast_simulation_dynamic <- function(y_train,
                                            x_train,
                                            H = 10,
                                            n_sims = 1000,
                                            max_lag = 3) {

  n <- length(y_train)

  # ------------------------------------------------------------------------
  # A) External model: ARMA(x) + simulated future paths
  # ------------------------------------------------------------------------
  model_external_arma <- auto.arima(x_train,
                                    max.p = max_lag,
                                    max.q = max_lag,
                                    ic = "bic",
                                    seasonal = FALSE,
                                    stationary = TRUE)

  simulation_external <- t(replicate(n_sims,
                                     simulate(model_external_arma, nsim = H)))

  # ------------------------------------------------------------------------
  # B) Select best ARDL(p,q) by BIC (p = 1..max_lag, q = 0..max_lag)
  # ------------------------------------------------------------------------
  best_bic <- Inf
  best_fit <- NULL
  best_p <- NA
  best_q <- NA

  y_target <- y_train[(max_lag + 1):n]

  for (p in 1:max_lag) {
    for (q in 0:max_lag) {

      reg_list <- list()

      # y lags
      for (k in 1:p) {
        reg_list[[paste0("y_lag", k)]] <- y_train[(max_lag + 1 - k):(n - k)]
```

```
    }

    # x contemporaneous + lags
    reg_list[["x_lag0"]] <- x_train[(max_lag + 1):n]
    if (q > 0) {
      for (k in 1:q) {
        reg_list[[paste0("x_lag", k)]] <- x_train[(max_lag + 1 - k):(n - k)]
      }
    }

    df_t <- as.data.frame(reg_list)
    df_t$y <- y_target

    fit <- lm(y ~ ., data = df_t)
    bic_val <- BIC(fit)

    if (bic_val < best_bic) {
      best_bic <- bic_val
      best_fit <- fit
      best_p <- p
      best_q <- q
    }  # end if

  }    # end for q
}      # end for p

if (is.null(best_fit)) stop("No valid ARDL model found (best_fit is NULL).")

# ------------------------------------------------------------------------
# C) Extract coefficients + residual volatility (sigma)
# ------------------------------------------------------------------------
cf  <- coef(best_fit)
sig <- sigma(best_fit)

get_c <- function(n) ifelse(n %in% names(cf), cf[n], 0)

alpha <- get_c("(Intercept)")
ardl_theta0 <- get_c("x_lag0")

phi   <- sapply(1:max_lag, function(k) get_c(paste0("y_lag", k)))
theta <- sapply(1:max_lag, function(k) get_c(paste0("x_lag", k)))

# ------------------------------------------------------------------------
# D) Monte Carlo simulation of y over H horizons
# ------------------------------------------------------------------------
sim_mat <- matrix(NA, n_sims, H)
```

```
  last_y <- rev(tail(y_train, max_lag))
  last_x <- rev(tail(x_train, max_lag))

  for (s in 1:n_sims) {

    x_fut  <- simulation_external[s, ]
    y_fut  <- numeric(H)
    shocks <- rnorm(H, 0, sig)

    buf_y <- last_y
    buf_x <- last_x

    for (h in 1:H) {
      curr_x <- x_fut[h]

      curr_y <- alpha + sum(phi * buf_y) +
        (ardl_theta0 * curr_x) + sum(theta * buf_x) + shocks[h]

      y_fut[h] <- curr_y

      # Update buffers
      buf_y <- c(curr_y, buf_y[1:(max_lag - 1)])
      buf_x <- c(curr_x, buf_x[1:(max_lag - 1)])
    }

    sim_mat[s, ] <- y_fut
  }

  # ------------------------------------------------------------------------
  # E) Fan chart summary
  # ------------------------------------------------------------------------
  list(
    mean   = colMeans(sim_mat),
    lower  = apply(sim_mat, 2, quantile, 0.05),
    upper  = apply(sim_mat, 2, quantile, 0.95),
    best_p = best_p,
    best_q = best_q,
    bic    = best_bic,
    sigma  = sig
  )
}
```

10.5.4 `generate_rolling_paths()`

Purpose
Generate **spaghetti forecast paths** from the rolling-window estimation output. For each rolling

forecast origin date (each window endpoint), the function:

1) Reconstructs the **external** x forecast path $x_{t+1}, \ldots, x_{t+H}$ using the stored `External_ARMA` coefficients at that origin.

2) Feeds the resulting x path into the **Swiss ARDL(p,q)** recursion using the stored `Swiss_ARDL` coefficients (and the (p, q) selected at that origin).

3) Stores each origin-specific forecast path (one row per horizon) for plotting.

Definition

At a given origin date t, the external process is forecasted deterministically with an ARMA recursion:

$$\hat{x}_{t+h} = c_x + \sum_{i=1}^{P} a_i \, \hat{x}_{t+h-i} + \sum_{j=1}^{Q} b_j \, \hat{\varepsilon}_{t+h-j}.$$

In this implementation, the MA innovations buffer is initialized to zero and kept at zero (deterministic recursion), so:

$$\hat{\varepsilon}_{t+h-j} = 0 \quad \Rightarrow \quad \hat{x}_{t+h} = c_x + \sum_{i=1}^{P} a_i \, \hat{x}_{t+h-i}.$$

Given $\hat{x}_{t+h}$, the Swiss ARDL(p,q) forecast is computed recursively as:

$$\hat{y}_{t+h} = c_y + \sum_{i=1}^{p} \phi_i \, \hat{y}_{t+h-i} + \theta_0 \, \hat{x}_{t+h} + \sum_{j=1}^{q} \theta_j \, \hat{x}_{t+h-j}.$$

The function returns a long data frame containing:

$$(\text{origin_date},\ h,\ \text{plot_date},\ \hat{y}_{t+h}).$$

Implementation notes

- Coefficients are retrieved with `get_coefficient_safe()`, which returns 0 if a term is missing (e.g., not selected in that window).
- The chosen (p, q) for each origin is read from `specifications` where `Model_Type == "Swiss_ARDL"`.
- Buffers are stored in reverse chronological order: `(t, t-1, t-2, ...)`, updated at each horizon with the newest forecast.

```
generate_rolling_paths <- function(data,
                                   specifications,
                                   coefficients,
                                   y_col,
                                   x_col,
                                   H = 10,
                                   max_lag = 3) {
```

```r
origin_dates <- unique(as.Date(specifications$Date))

rows <- vector("list", length(origin_dates) * H)
pos <- 0

for (origin_date in origin_dates) {

  index_end <- which(as.Date(data$date) == as.Date(origin_date))
  if (length(index_end) == 0) next
  if (index_end < max_lag) next
  if ((index_end + H) > nrow(data)) next

  # ------------------------------------------------------------------------
  # A) External ARMA coefficients (at origin)
  # ------------------------------------------------------------------------
  const_x <- get_coefficient_safe(coefficients,
                           origin_date,
                           "External_ARMA",
                           "intercept")

  if (const_x == 0) const_x <- get_coefficient_safe(coefficients,
                                                    origin_date,
                                                    "External_ARMA",
                                                    "mean")

  ar_x <- sapply(1:max_lag, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "External_ARMA",
                  paste0("ar", k)))

  ma_x <- sapply(1:max_lag, function(k)
    get_coefficient_safe(coefficients,
                  origin_date,
                  "External_ARMA",
                  paste0("ma", k)))

  # ------------------------------------------------------------------------
  # B) Swiss ARDL (p,q) selection + coefficients (at origin)
  # ------------------------------------------------------------------------
  sp <- subset(specifications,
               as.Date(Date) == as.Date(origin_date)
               & Model_Type == "Swiss_ARDL")

  p <- ifelse(nrow(sp) == 0, 0, sp$Best_P[1])
  q <- ifelse(nrow(sp) == 0, 0, sp$Best_Q[1])
```

```r
const_y <- get_coefficient_safe(coefficients,
                          origin_date,
                          "Swiss_ARDL",
                          "(Intercept)")

theta0  <- get_coefficient_safe(coefficients,
                          origin_date,
                          "Swiss_ARDL",
                          "x_lag0")

phi_y <- if (p > 0) sapply(1:p, function(k)
  get_coefficient_safe(coefficients,
                origin_date,
                "Swiss_ARDL",
                paste0("y_lag", k)))
else numeric(0)

theta_x <- if (q > 0) sapply(1:q, function(k)
  get_coefficient_safe(coefficients,
                origin_date,
                "Swiss_ARDL",
                paste0("x_lag", k)))
else numeric(0)

# ------------------------------------------------------------------------
# C) Initialize buffers (t, t-1, t-2, ...)
# ------------------------------------------------------------------------
x_buf <- rev(data[[x_col]][(index_end - max_lag + 1):index_end])
y_buf <- rev(data[[y_col]][(index_end - max_lag + 1):index_end])
e_buf <- rep(0, max_lag)  # deterministic MA part

# For ARDL we will use:
# - y lags: y_buf[1:p]
# - x lags: x_buf[1:q] and contemporaneous x_hat via theta0
y_buf_ardl <- y_buf
x_buf_ardl <- x_buf

# ------------------------------------------------------------------------
# D) Recursive path over horizons h = 1..H
# ------------------------------------------------------------------------
for (h in 1:H) {

  # External x_hat
  x_hat <- const_x + sum(ar_x * x_buf) + sum(ma_x * e_buf)

  # ARDL y_hat
  ar_part <- if (p > 0) sum(phi_y * y_buf_ardl[1:p]) else 0
```

```
      dl_part <- if (q > 0) sum(theta_x * x_buf_ardl[1:q]) else 0

      y_hat <- const_y + ar_part + theta0 * x_hat + dl_part

      # Store
      pos <- pos + 1
      rows[[pos]] <- data.frame(
        origin_date     = as.Date(origin_date),
        horizon         = h,
        plot_date       = as.Date(data$date[index_end + h]),
        forecast_value  = y_hat,
        stringsAsFactors = FALSE
      )

      # Update buffers
      x_buf <- c(x_hat, x_buf[1:(max_lag - 1)])
      e_buf <- c(0,     e_buf[1:(max_lag - 1)])

      y_buf_ardl <- c(y_hat, y_buf_ardl[1:(max_lag - 1)])
      x_buf_ardl <- c(x_hat, x_buf_ardl[1:(max_lag - 1)])
    }
  }

  if (pos == 0) return(data.frame())
  do.call(rbind, rows[1:pos])
}
```

10.5.5 `aggregate_growth_to_annual()`

Purpose
Convert a quarterly **log-growth** series (in percent) into **annual growth rates** by:

1) Reconstructing a quarterly **level index** from the cumulative log-growth,

2) Aggregating quarterly levels into an annual level (currently: **sum over the 4 quarters**),

3) Keeping only **complete years** (exactly 4 quarters),

4) Computing annual growth as the year-over-year percentage change in the annual level.

This is used to transform quarterly growth forecasts into annual growth numbers suitable for reporting.

Definition

Let g_t denote quarterly log-growth (in percent), i.e.

$$g_t = 100 \cdot (\log L_t - \log L_{t-1}).$$

Reconstruct the (index) level series using cumulative growth:

$$\log L_t = \text{base_log} + \sum_{s \le t} \frac{g_s}{100}, \qquad L_t = \exp(\log L_t)\,.$$

Aggregate quarterly levels into an annual level A_y. With **sum aggregation** (as implemented):

$$A_y = \sum_{q=1}^{4} L_{y,q}.$$

Annual growth is then:

$$\text{annual_growth}_y = \frac{A_y - A_{y-1}}{A_{y-1}}.$$

Implementation notes

- The function assumes `g_q` is already in **percent** (e.g., $100\Delta \log$).
- The quarterly level is an **index** (scale depends on `base_log`), but annual growth is scale-invariant.
- Only years with **exactly 4 quarterly observations** are kept (`n_quarters == 4`).
- `years_keep` optionally filters the output to a selected set of years.
- Argument `agg` is currently present but **not used** in the code (annual aggregation is always `sum(...)`).

```r
aggregate_growth_to_annual <- function(dates_q,
                                       g_q,
                                       years_keep = NULL,
                                       base_log = 0,
                                       agg = "sum") {
  df <- data.frame(date = as.Date(dates_q), g = as.numeric(g_q))
  df <- df[order(df$date), ]
  df$level <- exp(base_log + cumsum(df$g / 100))
  df$year  <- as.integer(format(df$date, "%Y"))

  annual <- df %>%
    dplyr::group_by(year) %>%
    dplyr::summarise(
      annual_level = sum(level, na.rm = TRUE),
      n_quarters   = dplyr::n(),
      .groups = "drop"
    )

  annual <- annual[annual$n_quarters == 4, ]
  annual$annual_growth <- c(NA, diff(annual$annual_level) / head(annual$annual_level, -1))
```

```
  if (!is.null(years_keep)) annual <- annual[annual$year %in% years_keep, ]
  rownames(annual) <- NULL
  return(annual)
}
```

11 References

- Diebold, F. X., & Mariano, R. S. (1995). Comparing Predictive Accuracy.
- Mincer, J., & Zarnowitz, V. (1969). The Evaluation of Economic Forecasts.
- Newey, W., & West, K. (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix.